

Framework

[5-10 mins] Ask Clarifying questions:

1. What are the most important features for this product?
2. What are the success metrics? DAU, number of retries per session (small as possible), satisfaction indicators such as copy text, thumbs up, or end of session and returning the next day.

[10-15 mins] Propose High Level and Get Buy In:

1. Enthusiasm and propose to work together on this.
2. Back of Envelope calculations for various numbers if necessary
 - a. How many users do we expect per day and what is the average QPS? Do we expect to handle failovers?
 - b. Latency
 - i. RAM read is 300x faster than disk read (30ms), 100x faster than network read (10ms)
 - ii. Worldwide roundtrip is 150 ms, which is 300x faster than a roundtrip in the same data center
 - c. Downtime in a year: 99% or 0 9's → 3.5 days, 2 9's → 1 hour, 3 9's → 5 mins, 4 9's → 30 seconds
 - d. X example:
 - i. 300 MAU, 50% DAU → 150m, 2 posts per day avg, 10% has media, Data stored 5 years
 - ii. QPS: $150m * 2 / (24 * 3600) = 3000 - 4000$, peak is 2x so 7000
 - iii. Storage: text less than 0.001% → mostly media, 1mb → 30TB per day → 60 TB for 5 years
3. Draw an initial high level blueprint, though suggest multiple approaches if possible
 - a. APIs, web servers, data stores, cache, CDN, message queue, etc.
 - b. Do we need API endpoints and database schemas? Where to go deeper? Ask for feedback and communicate
4. Talk through a few concrete use-cases

[15-25 mins] Design deep dive:

1. Prioritize which components to go into first.
2. For senior candidates, it's likely about system performance such as bottlenecks and resource estimations.
3. For each type of problem, there's usually one or two interesting topics that's important to dig into.
4. Signal strengths and what you know, don't go into unnecessary algorithm details

[5 mins] Wrap up:

1. Recap the design, refresh their memory
2. Identify potential improvements that can be made

- a. Error cases
 - b. Operational issues such as monitoring and rollout
 - c. Handling next scale curve (going from 1m → 10m → 100m)
3. Thank interviewer for their insights and collaboration

ML System Design

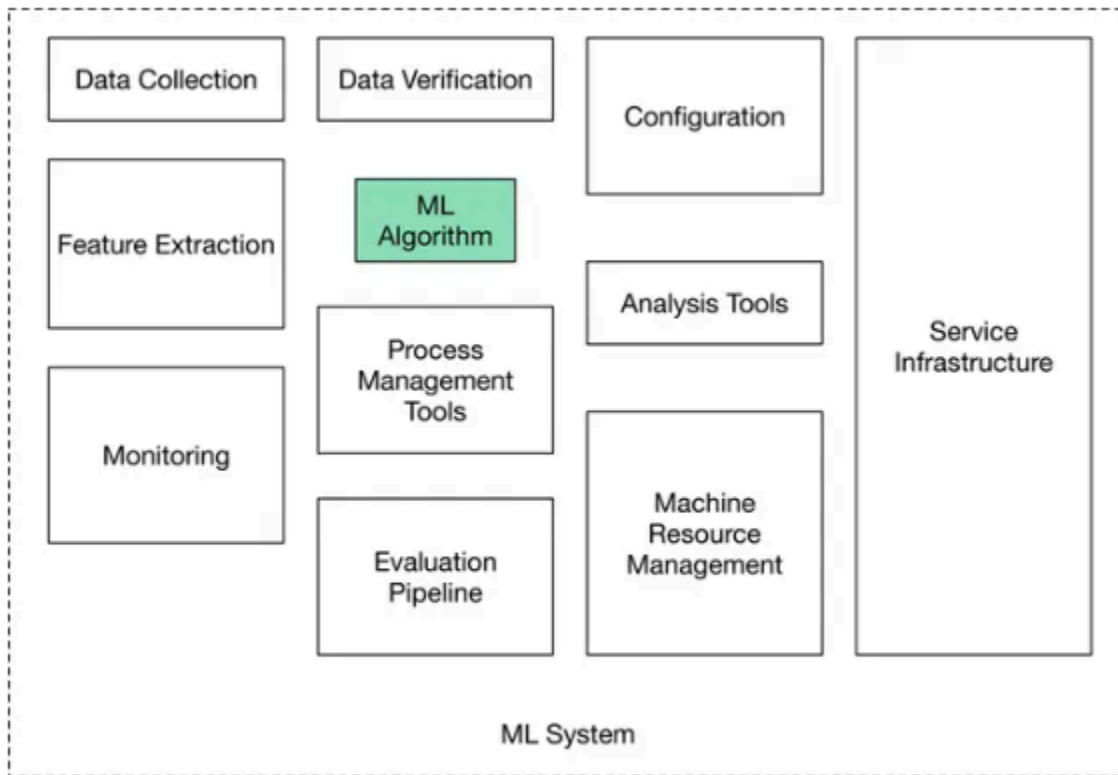


Figure 1.1 Components of a production-ready ML system

1. Clarify requirements

- a. Business objectives and metrics like DAU, MAU, satisfaction surveys, interactions, prevalence, etc.
- b. Features like interactions, posting, projects, etc.
- c. Data sources like user logs, labeled data, etc.
- d. Scale and Performance like how fast, how many DAU, how many items per DAU, growth rate, etc.

2. Frame as ML task

- a. Examples
 - i. Event ticket selling → maximize number of event registrations, increase registration rate per impression
 - ii. Video recommendation → maximize total watchtime
 - iii. Ad prediction → maximize click through rate

- iv. Harmful content detection → maximize accuracy, PR AUC, decrease prevalence
- v. Friend recommendation → maximize number of friendships formed
- b. Input and output
 - i. Post → ML classifier → prob(harmful)
 - ii. <user, event> → ML classifier → Prob(registration)
 - iii. User → ML classifier → prob(event1) , ... , prob(eventN)
- c. Supervised (classifier, regression), Unsupervised (clustering, association), RL (computer agent learns through trial and error with the environment)

3. Data prep

- a. ETL pipelines from data sources to transform into well-formatted data structures for training, data cleaning
 - i. Structured data typically in SQL DBs
 - 1. Pros: Easy to search, predefined schema
 - ii. Unstructured data from logs, audio, image, text typically in NoSQL (JSON, XML, KVStore), data lakes
 - 1. Pros: Flexible format, supports multiple customers, single source of truth
 - 2. Cons: no context, slow to search, slow for real-time needs
 - iii. Data Ingestion Layer:
 - 1. User uploads → raw text/image/video → Data Lake (immutable copy)
 - iv. Logging & Feedback: All predictions + user appeals logged back to the Data Lake for audit and continuous learning.
 - 1. Debuggability
- b. Feature engineering (manual, curated or automatic, NN style)
 - i. Handling missing values rows → filtering is simple but depends on how much you would lose, so consider doing statistical imputation or using a new category to encode the missing data. Would go upstream from there and actually figure out what's causing the missingness in the first place though. Usually a sign of bad logging, etc.
 - ii. Log scaling, z-score normalization, min-max normalization
 - iii. Represent categorical features as one-hot, ordinal integer, or embedding features

4. Modeling

- a. Establish a simple heuristic baseline → experiment with quick models to establish the end-to-end model training and serving pipeline → switch over to neural networks → consider ensembling, meta-prediction-frameworks, multi-layered frameworks for fraud detection.
- b. Factors to consider for model type: Data needs, training time, number of hyperparameters, auto-retraining, interpretability, compute costs
- c. Often times, sampling is critical to achieve good results (upsampling positives, downsample negatives, how to handle in-batch negative sampling if training embeddings via NCE, consider easy negatives and hard negatives e.g. active learning for adversarial environments)
- d. Loss function & optimizers (in DNN)

- e. Dataset, how far back do you go, how big of a model are you training, predictiveness versus staleness → foundational models & finetuning on domain-specific tasks
- f. Distributed training → data parallelism is when you have identical model copies in each worker, split up the data, and do local gradient calculation and then sync up the gradients, but this doesn't work for very very large models, model parallelism is when you split the model's layers or even parts of layers across GPUs, then you need to send a single data batch through a sequence of these devices, so now you have to deal with how the GPUs talk to each other and how to make model partitioning and scheduling efficient. Hybrid approaches can also work, where you split data across clusters of GPUs and split model within each cluster. Basically network communication is the biggest design concern

5. Eval

- a. Offline
 - i. Precision, Recall, F1, ROC-AUC (axes are TPR and FPR, focused on how well the model can distinguish between positive and negative classes), PR-AUC (much more sensitive to class imbalance because the axes are precision and recall, focused on the classifications for the positive class)
 - ii. Precision@K, Recall@K, MAP, NDCG
 - iii. BLUE for translation, ROUGE for summarization, CIDER for captioning
 - iv. Fairness and Bias → require metrics for specific hypotheses, hard to have generalized metrics
- b. Online → A/B test is the most straightforward but requires a lot of infra to set up properly (e.g. doing user salts and multiple overlapping experimentation universes), can also consider shadow deploys or canaries to get system logs or offline simulations to estimate user behavior

6. Deployment & Serving

- a. Cloud vs on-device. Cloud is easy to set up, more accurate due to fewer constraints, faster inference. On-device has no latency, might be cheaper, better for privacy
- b. Model compression → knowledge distillation to produce cheaper student models, quantization either during or post-training, pruning
- c. Online test → shadow test where you run the new model along with existing model, A/B test (sample size, and randomness)
- d. Predictions → batch versus online, online systems require feature systems to also go online, which means logging of feature sets for ML model training or you'll need separate online feature computation and offline feature computation
- e. Caching predictions (semantic caches)

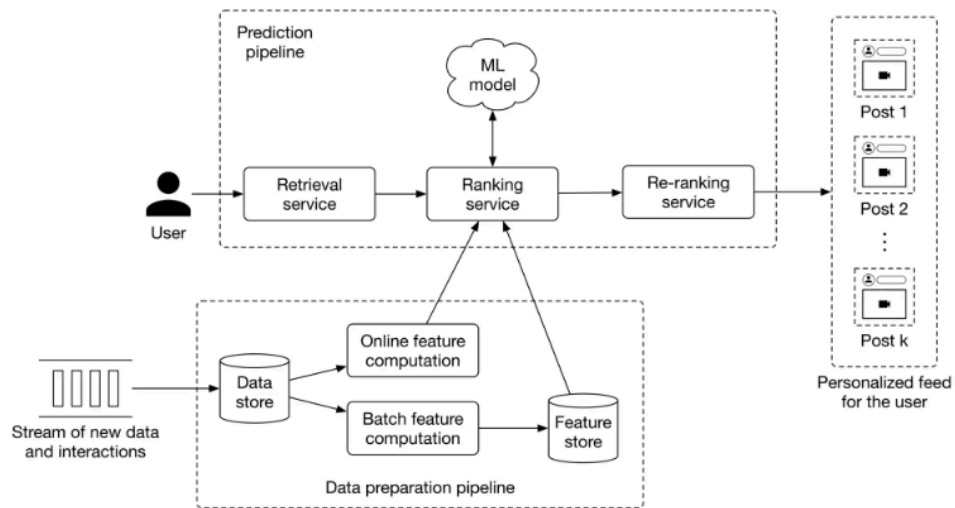


Figure 1.23: ML system design for a personalized news feed system

f.

7. Monitoring

a. Failures

- i. Feature outages (feature distribution drift, NULL values → may lead to NULL predictions)
- ii. Stale model predictions (no retraining, data drift)
- iii. System failures (latency, throughput, versioning)
- iv. Traffic spikes (GPU / CPU utilization)
- v. Code path failures (what's AB tested wasn't what was launched)
- vi. Feedback loops

AI System Design

Modern Generative models include Autoregressive models (Transformers for NLP, time series), diffusion models that do image or video generation, GANs (train both a generator and discriminator in parallel, generator produces realistic data, while discriminator distinguishes)

Recent factors unlocking its improvements:

- Data - SSL nature of GenAI models allow it to tap into Books, Github, Social Media, curated datasets for different domains without human labeling
- Model size - tons more parameters (Claude hundreds of billions), FLOP counts due to model architecture
- Compute - advances in data and model parallelisms, stability allow training over thousands of GPUs for weeks or months at a time, also very costly and power-consuming
- Scaling laws: recent research suggests that model size matters much more than variants of model architecture, though the latter can help significantly with efficiency

1. Clarify requirements

- a. Business objectives and metrics like DAU, MAU, satisfaction surveys, interactions, prevalence, etc.
- b. Features like interactions, posting, projects, etc.
- c. Data sources like user logs, labeled data, etc.
- d. Scale and Performance like how fast, how many DAU, how many items per DAU, growth rate, etc.

2. Frame as ML task

- a. Generative (text, image, video, audio) versus Discriminative (classification, regression)
- b. Inputs (text, image, video, audio) and Outputs
- c. A single model for all modalities or multiple models for more modalities?
- d. Which generative algorithm is best suited and why? (Diffusion, GANs, Autoregressive)

3. Data Prep

- a. Structured data like SQL-like tabular data is likely not the core. Likely unstructured data such as text, images, videos, audio, etc.
- b. Data Collection
 - i. Scraping the internet, social media, books, videos, etc.
 - ii. Can augment with GenAI produced dataset to improve data diversity and have more data, but synthetic data (1) may be less complex than real-world, (2) lower quality thereby spreading errors, (3) not diverse or representative enough
 - iii. Typically you'll want to collect 2 sets of data: (1) for your foundational or pretrained models, (2) for the domain-specific fine-tuning
- c. Data Cleaning
 - i. Clean and remove misinformation, NSFW or harmful materials
 - ii. Dedupe to assure data is representative and to ensure diversity
 - iii. Watch for bias or other low quality content from affecting model performance
- d. Data Efficiency
 - i. **Storage** in Parquet and ORC, which are column-oriented and offer things like data skipping by having statistics like max/min/counts associated with each column in the file. These lead to reduced I/O operations for faster reads. **Homogeneity** in the data leads to better compression like dictionary encoding, etc. which means faster data retrieval and lower storage costs.
 - ii. **Sharding** the data row-wise, or horizontally partitioning the data across multiple devices allows parallel access, easy scalability, and fault tolerance. As with all sharding, we want the key to give us even data distribution.
 - iii. **Indexing** for much faster searches avoids full scans, as well as inverted indexes like Lucene which ("safety" → docs1,2, "classifier" → docs1, 7,8) enables fast full-text keyword search, fuzzy search. Set up HNSW for vector search (FAISS)
 - iv. May want to consider adding a **caching layer** for frequently accessed documents

4. Modeling

- a. Model architecture - Seq2seq, attention, transformers ([section](#))
- b. Model Training and ML datasets
 - i. Training process and datasets: Diffusion models denoise data to generate high-quality samples from noise, GANs rely on adversarial training. LLMs typically follow (1) pretraining on large datasets such as common crawl, (2) supervised finetuning on specific tasks relying on curated labeled datasets, (3) alignment stage to ensure outputs are aligned with values and intended behaviors.
 - ii. ML objectives and loss function: LLM is to predict next token, VAE is original image reconstruction. Loss functions typically follow the same as before.
- c. Objectives and loss functions ([section](#))
- d. Speed up Model Training
 - i. Distributed Training - Data parallelism & model parallelism
 - 1. Data parallelism uses a parameter server to update gradients and model parameters across different GPUs, using the same model copy and different data splits on each GPU

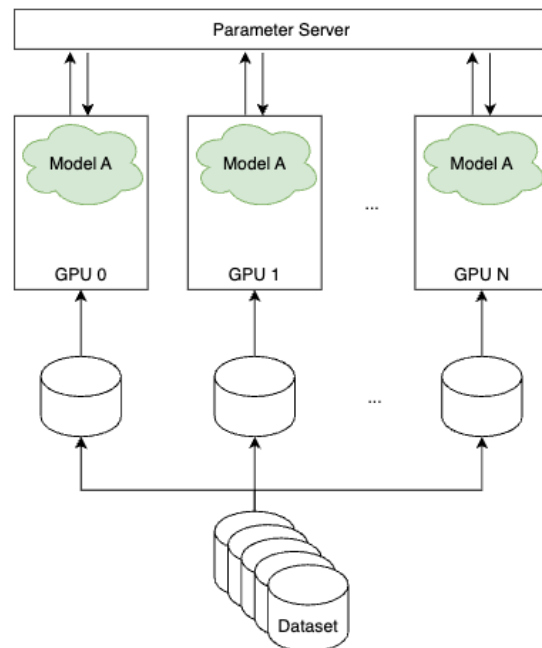


Figure 21: Data parallelism

- 2.
- 3. When updating the model parameters, can do synchronous where PS waits for all gradient updates, aggregate, and send it to all devices. This has to wait for the slowest device. Async updates all other machines whenever PS gets a gradient update and finishes its own computation.
- 4. Model parallelism - each device contains some set of layers, so the devices are passing the activation weights in the forward pass and the gradients in the backward pass. To go deeper, I would imagine you'd have to split the tensor operations across different devices.
- 5. Can also do a hybrid approach.

- ii. Mixed Precision training - uses fp32 only for critical calculations, and fp16 otherwise
- e. Model Sampling to determine how to output from the model
 - i. Greedy search - take top probability token at each step
 - 1. Fast and simple, good for deterministic responses
 - 2. Tends to be hallucinate the least if model is well-trained
 - 3. Lacks diversity and can be repetitive
 - ii. Beam search - keeps track of K most probable sequences, then at next step produce K most likely tokens, and produce the new top K most probable. Keep going until EOS token or max length. Select the sequence with highest cumulative probability.
 - 1. More relevant than greedy search
 - 2. Much more coherent and fluent
 - 3. Very high computational costs
 - 4. Lacks diversity and creativity
 - iii. Top-K sampling: reduce vocab to K most probable tokens, randomly sample one token from reduced set
 - 1. Introduces more diversity via randomness
 - 2. Since we pruned, it prevents gibberish
 - 3. For large K this can still be less relevant
 - iv. Top-P sampling: select the smallest set of tokens such that the cumulative probability exceeds P, then random sample from this set.
 - 1. Adaptive to the probability distribution
 - 2. More human like
 - 3. Still need to tune P
 - v. Add in a temperature term that pushes towards more deterministic versus more diversity / randomness.

5. Eval

- a. Offline Eval
 - i. Standard offline metrics
 - ii. HITL evaluation
 - iii. Hallucination (automated fact checking)
 - iv. Safety and policy compliance
- b. Online Eval (engaging stakeholders on what makes most sense)
 - i. Prompt-response latency
 - ii. Task success rate (e.g. ticket resolved)
 - iii. Average handle time (when human have to intervene)
 - iv. Survey, NPS
 - v. Engagement
 - vi. Retention
 - vii. Prevalence of harms

6. ML System

- a. Components: core model, preprocessing, content filtering, post-processing, upscaling
- b. Scalability → number of users, new features (e.g. image, video understanding and generation)
- c. User feedback and continuous learning → systems in place to retrain models with updated data to improve accuracy and relevance over time

- d. Safety and moderation → filtering out inputs that are harmful or NSFW, preventing outputs that are harmful or NSFW
- e. Security & Robustness → privacy and dealing with sensitive data, protections against prompt injection and other adversarial attacks, ensure outputs are not used for malicious purposes
- f. Bias → proactive detection, auditing, filtering of biased outputs

7. [Deployment](#)

8. [Monitoring](#)

[Design] Harmful Content Detection

Okay, James, this is absolutely the most critical section for your interview at Anthropic, given their explicit focus on AI safety. Your ability to articulate a detailed and nuanced approach to harmful content detection, including the technical design, trade-offs, and critical safety considerations, will be paramount. I'll provide an in-depth breakdown in your preferred notes format, emphasizing the connection to your role and Anthropic's values.

1. Clarify Requirements:

- **Primary Goal:** Proactively monitor new posts, detect harmful content, and remove/demote content violating platform guidelines.
- **Scope:** Focus on **Harmful Content Detection** (violence, nudity, self-harm, hate speech). **Misinformation excluded** for simplicity (due to complexity/controversy). **Bad actors excluded** for simplicity.
- **Input Content:** Heterogeneous and **multi-modal** (text, images, video, or any combination).
- **Language:** Multilingual (assume pre-trained multilingual embeddings available for text).
- **Training Data:**
 - **Scale:** 500 million+ posts daily.
 - **Human Annotation:** Limited availability (e.g., 10,000 posts/day) for high-accuracy labeling (e.g., evaluation data).
 - **User Reports:** Available as a signal for failures/training data.
- **Explainability:** Essential to explain *why* a post is removed (helps users align with guidelines).
- **Latency Requirements:**
 - **Real-time:** For sensitive categories (e.g., violence), immediate blocking.
 - **Batch/Offline:** Acceptable for others (hourly/daily demotion/review).

2. Frame as ML Task:

- **ML Objective:** Accurately predict harmful posts (multi-class, multi-label).
- **System Input:** A post (heterogeneous, multimodal: text, image, video, metadata).
- **System Output:** Probability that the post is harmful for **each specific harmful class** (e.g., $P(\text{violence})$, $P(\text{nudity})$, $P(\text{hate speech})$, $P(\text{self-harm})$).
- **ML Category & Fusion Strategy:**

- **Fusion Method: Early Fusion.**
 - **Rationale:** Crucial for detecting nuanced harmful content (e.g., memes) where individual modalities might be benign but their combination is harmful. Also, eliminates need for separate training data per modality.
 - *(Avoid Late Fusion: Fails on cross-modal harm, requires separate data per modality).*
- **Classifier Type: Multi-Task Classifier.**
 - **Rationale:** Addresses limitations of single binary classifier (no explanation of specific harm) and multiple binary classifiers (high training/maintenance cost). Leverages shared learnings across harm types, efficient training/maintenance, and data sparsity for individual tasks.
 - **Architecture:**
 1. **Shared Layers:** Transform fused input features (multi-modal) into new representations. Benefit multiple tasks.
 2. **Task-Specific Layers (Classification Heads):** Independent ML layers, each optimized for predicting the probability of a specific harmful class (e.g., one head for violence, one for nudity).
 - *(Avoid Single Binary Classifier: Cannot explain specific harm, hard to improve per class. Avoid One Binary Classifier per Harmful Class: Expensive, high maintenance).*
 - *(Avoid Multi-Label Classifier: Less ideal as input transformations might differ for each harm type).*

3. Data Preparation (Data Pipeline):

- **Data Sources:**
 - **Users:** ID, Username, Age, Gender, City, Country, Email.
 - **Posts:** Post ID, Author ID, Timestamp, Textual content, Images/Videos, Links.
 - **User-Post Interactions:** Likes, Comments, Saves, Shares, **Reports** (critical signal), Appeals.
- **Feature Engineering (Early Fusion - all features combined for shared layers):**
 - **Textual Content:**
 - **Preprocessing:** Normalization (lowercasing, punctuation/whitespace removal), Tokenization (Subword, e.g., BPE).
 - **Vectorization:** Use a **pre-trained Transformer-based multilingual language model (e.g., DistilMBERT or a multilingual BERT/LLM encoder)**.
 - **Rationale:** Captures semantic meaning, handles multiple languages, efficient for online prediction (compared to larger BERT).
 - **Image or Video Content:**
 - **Preprocessing:** Decode, Resize, Normalize.
 - **Feature Extraction:** Use **pre-trained models** for embeddings.
 - **Images:** CLIP's visual encoder, SimCLR.
 - **Videos:** VideoMoCo, or a frame-level model (ViT on frames) with aggregation (e.g., average).
 - **User Reactions to the Post (for ambiguous content):**

- **Counts:** Number of likes, shares, comments, reports (scale these numerical values).
 - **Comments:** Use the *same pre-trained multilingual text model* to get comment embeddings, then aggregate (e.g., average) for a fixed-size vector.
- **Author Features (Past Behavior is Predictive):**
 - **Violation History:** Number of past violations, total user reports on author's posts, profane words rate in past content.
 - **Demographics:** Age (numerical), Gender (one-hot or embedding), City/Country (embedding layer preferred over one-hot for many distinct values).
 - **Account Info:** Number of followers/followings, account age (younger accounts more suspicious).
- **Contextual Information:**
 - **Time of Day:** Bucketize (morning/noon/etc.), one-hot encode.
 - **Device:** One-hot encode.
- **Constructing Dataset for Multi-Task Training:**
 - **Input:** Fused features of posts (computed offline and stored in a feature store).
 - **Labels:**
 - **Training Data: Natural Labeling (user reports)** for speed and volume. Acknowledge noise.
 - **Evaluation Data: Hand Labeling (human contractors)** for accuracy on a limited subset (e.g., 10,000 posts/day).
 - **Data Point Example:** Fused_Features, Label_Violence (0/1), Label_Nudity (0/1), Label_HateSpeech (0/1), Label_SelfHarm (0/1).

4. Modeling (Training Pipeline):

- **Model Selection:** Neural Network for Multi-Task Learning.
 - **Architecture:** Shared layers (dense NN blocks) for general feature transformation, followed by task-specific classification heads for each harm type.
 - **Hyperparameter Tuning:** Grid search (or more advanced methods like Bayesian Optimization) for optimal hidden layers, activation functions, learning rate, etc.
- **Loss Function:**
 - **Per Task:** Each task (harm class) uses a standard **Binary Cross-Entropy Loss**.
 - **Formula (for one task):** $L_{task} = -(y \log(y) + (1-y) \log(1-y))$
 - **Purpose:** Optimizes probability prediction for each specific harm type.
 - **Overall Loss:** Sum (or weighted sum) of individual task losses.
 - **Formula:** $L_{overall} = \sum_{k=1}^N L_{task_k}$
 - **Purpose:** Allows the model to learn multiple related tasks simultaneously, leveraging shared features.
- **Challenges & Mitigation (Overfitting in Multimodal Systems):**
 - **Problem:** One modality (e.g., image) can dominate learning if learning speeds vary.
 - **Techniques:**
 - **Gradient Blending:** Adjusts gradients to balance learning across modalities.

- **Focal Loss:** (Or other re-weighting techniques for imbalanced data) can help focus learning on hard-to-classify examples, which might be critical for rare harm types.

5. Evaluation:

- **Offline Metrics (Per Harmful Class):**
 - **PR-AUC (Area Under Precision-Recall Curve): Primary Metric.**
 - **Why:** Shows trade-off between Precision and Recall across different thresholds. More informative for **imbalanced datasets** (harmful posts are rare) and when **positive class performance** (detecting harm) is critical. Higher PR-AUC indicates better accuracy.
 - **ROC-AUC (Area Under Receiver Operating Characteristic Curve): Secondary Metric.**
 - **Why:** Shows trade-off between True Positive Rate (Recall) and False Positive Rate. Useful for overall model performance but less sensitive to class imbalance than PR-AUC.
- **Online Metrics (Focus on Platform Safety & Efficiency):**
 - **Harmful Impressions: Preferred over Prevalence.** Measures how many people were *exposed* to unprevented harmful content. Prioritizes reach over count.
 - $\text{Harmful Impressions} = \text{sum of impressions on unprevented harmful posts}$
 - **Proactive Rate:** Percentage of harmful posts detected by the system *before* user reports.
 - $\text{Proactive Rate} = \frac{\text{System Detected}}{\text{System Detected} + \text{User Reported}}$
 - **Purpose:** Measures system's effectiveness at catching harm proactively. A high proactive rate is a strong indicator of a successful system.
 - **Valid Appeals Rate:** Percentage of removed posts that were *incorrectly* flagged and later reversed by human review.
 - $\text{Valid Appeals} = \frac{\text{Number of Harmful Posts Detected by System}}{\text{Number of Reversed Appeals}}$
 - **Purpose:** Measures **false positive rate** and impact on user experience/trust. A low valid appeals rate is crucial.
 - **User Reports per Harmful Class:** Tracks specific areas of system weakness where users are still finding harm.
 - **Latency:** Real-time (for severe harm) vs. Batch (for others). Crucial to meet differing requirements.

6. Serving (Overall ML System Design):

- **Workflow:** New Post → Harmful Content Detection Service → Confidence Score per harm type.
- **Components:**
 1. **Harmful Content Detection Service (Core):**
 - Receives new post (text, image, video).
 - Performs **Early Fusion** of multi-modal features.
 - Passes fused features to the **Multi-Task Classifier** (Neural Network).
 - Predicts probabilities for each harmful class.

2. Violation Enforcement Service (Real-time Blocking):

- If specific harmful class probabilities (e.g., violence) are **above a high confidence threshold**, immediately blocks/removes the post.
- **Notifies user** why the post was removed (essential for explainability).

3. Demoting Service (Soft Mitigation):

- If harmful class probabilities are **above a lower confidence threshold (but below removal threshold)**, temporarily demotes the post (reduces visibility/spread).
- Sends post to **Manual Review Queue** for human annotators.
- Human review labels are used to **improve future training iterations** (continuous feedback loop).

7. Monitoring:

- **System Health:** Latency, throughput, resource utilization of all services.
- **Model Performance (Offline):** Continuous ROC-AUC/PR-AUC monitoring for each harm class on validation sets.
- **Online Performance (Safety Focus):**
 - **Harmful Impressions:** Track trends, drill down by content type, geography, time of day.
 - **Proactive Rate:** Trend over time, goal is to maximize.
 - **Valid Appeals Rate:** Trend over time, goal is to minimize.
 - **User Reports (by category):** Identify emerging threats or system blind spots.
 - **False Negatives (Misses):** Analyze posts reported by users after being missed by the system.
 - **False Positives (Over-flags):** Analyze posts from valid appeal reversals.
- **Data Quality:** Monitor feature store health, consistency of human labeling vs. natural labeling.
- **Bias Monitoring:** Track model's performance (false positives/negatives) across different demographic groups (age, gender, race) to identify and mitigate fairness issues.

Other Potential Discussion Points (for senior roles / if time permits):

- **Trending Harmful Classes:** Adapting detection for new/evolving threats (e.g., specific misinformation campaigns).
- **Temporal Information:** Leveraging sequence of user actions for bad actor detection.
- **Human-in-the-Loop Optimization:** Efficiently selecting samples for human review (e.g., active learning on borderline cases).
- **Borderline Content:** Strategies for handling content not strictly prohibited but close to guidelines.
- **On-Device Deployment:** Making models efficient enough for client-side detection.
- **Advanced Architectures:** Discussing more efficient Transformers (e.g., Linear Transformers).
- **Proactive Mitigation:** Moving beyond reactive deletion to proactively identifying and removing content before significant exposure. *Strategic: Aligns directly with Anthropic's "safety-by-design" and proactive risk management.*

- **Explainable AI (XAI):** Deeper discussion on how to generate human-understandable explanations for specific removals (e.g., saliency maps for images, attention visualization for text). *Strategic: Aligns with Anthropic's transparency goals.*
- **AI for Moderation:** How LLMs/Generative AI could assist human moderators (e.g., summarizing content, suggesting policy violations). *Strategic: Connects to your interests and Anthropic's work.*

[Design] Multi-Layered Harmful Content Detection

This is a comprehensive request, and it's great that you're thinking through these aspects for your ML system design interview, especially with the focus on AI safety at Anthropic. Let's break down each topic with end-to-end examples, keeping your background and the provided context in mind.

Since you're interviewing for a role focused on building ML Safety classifiers for Anthropic, we'll tailor the examples to that domain. This will allow you to demonstrate your understanding of safety as a core design principle, which is highly valued by companies like Anthropic.

(1) ML Task Formulation for a Given Business Problem: ML Safety Classifier

Let's imagine the business problem is: **"Automatically detect and mitigate harmful content generated by a large language model (LLM) in real-time, specifically focusing on misuse related to CBRN (Chemical, Biological, Radiological, Nuclear) threats."** This aligns directly with Anthropic's ASL-3 standards and your role in building ML Safety classifiers.

End-to-End Example: CBRN Misuse Classifier

1. Define the Business Problem & Goal:

The primary goal is to prevent the LLM from generating responses that could facilitate or promote CBRN misuse. This is a critical safety control to protect against severe, novel, and irreversible harm, aligning with Anthropic's Responsible Scaling Policy.

2. Translate to an ML Problem:

This can be framed as a binary text classification problem, where the input is an LLM-generated text (or potentially a user prompt) and the output is a prediction of whether it is "harmful" (specifically, CBRN-related misuse) or "harmless."

- **Input:** Text string (e.g., LLM response, user prompt).
- **Output:** Binary classification (e.g., **1** for CBRN misuse, **0** for harmless).

3. Data Collection & Labeling Strategy:

- **Data Sources:**
 - **Synthetic Data Generation:** Given the rarity and sensitivity of real-world CBRN misuse data, **synthetic data generation** will be paramount. This is explicitly mentioned in the context for Constitutional Classifiers. We'd use a large, capable

LLM (perhaps an internal Claude model) to generate diverse examples of both benign and harmful (CBRN-related) text. This would include:

- Benign conversations about chemistry, biology, physics.
- Simulated attempts at eliciting CBRN information (jailbreaks, subtle prompts).
- Direct and indirect instructions for creating harmful agents or devices.
- Variations in language, style, and cultural contexts.
- **Red Teaming Data:** Incorporate data from ongoing **red teaming efforts** (human and automated) where security researchers attempt to "jailbreak" or prompt the LLM into generating harmful CBRN content. This provides real-world adversarial examples.
- **Publicly Available Safety Datasets:** While limited for CBRN, general safety datasets (e.g., for toxicity, hate speech) can be used for pre-training or as negative examples.
- **Labeling:**
 - **Expert Human Labeling (for initial bootstrapping and auditing):** Given the high-stakes nature of CBRN, initial labeling would require domain experts (e.g., chemists, biologists, security analysts) to ensure accuracy. This is a form of Human-in-the-Loop (HITL) for critical safety tasks.
 - **AI-Assisted Labeling/RLAIF:** For scale, once initial expert labels establish a strong baseline, an "AI judge" (another well-aligned LLM) can be used to generate preference labels (e.g., identifying harmful vs. harmless CBRN-related content). This aligns with Anthropic's RLAIF/Constitutional AI approach. Regular auditing by human experts would be critical to prevent the "alignment paradox" (where the AI judge itself might perpetuate biases or misunderstand nuances).

4. Model Selection (High-Level):

- **Type:** A highly performant **transformer-based text classifier**. Given the real-time inference requirement, we'd aim for a model that is efficient (e.g., smaller, quantized).
- **Architecture:** Potentially a fine-tuned version of a smaller LLM, or a dedicated deep learning model (e.g., BERT, RoBERTa, or a custom architecture trained specifically for this task). The context mentions using "lightweight machine learning classifiers" for token-level filtering.

5. Evaluation Metrics (Crucially Important for Safety):

- **Primary Metric: Recall of Harmful Content (High Priority):** This is paramount for safety. We want to ensure we catch as many actual CBRN misuse attempts as possible. A high recall means fewer harmful outputs slip through.
- **Secondary Metric: Precision of Harmful Content (Balancing Over-refusals):** While recall is key, precision ensures we don't block too many benign queries (over-refusals). The context explicitly mentions that initial prototypes of Constitutional Classifiers had problems with over-refusals, so this is an important balance.
- **F1-Score:** A balance between precision and recall, especially useful for imbalanced datasets (harmful instances are rare).
- **Latency/Throughput:** For real-time applications, the speed of the classifier is critical. This impacts user experience and the feasibility of token-level filtering.

- **Robustness to Adversarial Attacks:** This is a core safety concern. We need to evaluate how well the classifier performs against prompt injection, evasion attacks, and other adversarial manipulations designed to bypass safety filters. This can be measured via specific red teaming metrics (e.g., jailbreak success rate).

6. Deployment & Monitoring:

- **Deployment Strategy: Canary deployments or A/B testing** would be ideal to safely roll out new versions of the CBRN classifier. This limits the "blast radius" if a new version introduces unintended biases or over-refusals, allowing for rapid rollback.
- **Real-time Observability:** Implement robust LLM observability pipelines to monitor:
 - **Refusal Rates:** Track how often the classifier flags content, and investigate spikes or unexpected drops.
 - **False Positive/Negative Rates (Audited):** Continuously sample flagged and unflagged content for human review to assess precision and recall in production.
 - **Latency:** Ensure the classifier adds minimal overhead to the LLM's response time.
 - **Drift Detection:** Monitor input distributions and model predictions over time to detect concept or model drift, indicating the classifier might be becoming less effective.
- **Feedback Loop:** Establish a continuous feedback loop from monitoring, red teaming, and user reports to retrain and improve the classifier. This aligns with the "continuous adaptation" and "arms race" dynamic described in the context.

(2) Different Input Modalities and Handling Them Via Single Versus Multiple Models

For an ML safety classifier, inputs can extend beyond just text.

End-to-End Example: Multi-Modal Safety Classifier for an LLM with Vision Capabilities

Let's assume the LLM can now process image inputs (e.g., a "vision-language model" like GPT-4V or Claude 3 Opus). The business problem remains safety, but now it includes harmful *visual* content or visual prompts that could lead to harmful text generation.

Input Modalities:

- **Text:** User prompts, LLM-generated text.
- **Image:** User-uploaded images, images generated by the LLM (e.g., if it has image generation capabilities).

Handling Strategies:

Option A: Single, Unified Multi-Modal Model (End-to-End Approach)

Description: Train or fine-tune a single large multi-modal model that takes both image and text inputs and produces a safety classification. This model would have a shared understanding across modalities.

Architecture:

- **Vision Encoder:** A pre-trained vision transformer (e.g., ViT, CLIP's vision encoder) to process images and convert them into a numerical representation (embeddings).

- **Text Encoder:** A pre-trained text transformer (e.g., BERT, a smaller LLM) to process text and convert it into embeddings.
- **Fusion Layer:** A mechanism to combine the image and text embeddings (e.g., concatenation, cross-attention, specialized fusion network).
- **Multi-modal Transformer/Classifier Head:** A transformer block that processes the fused embeddings, followed by a classification head (e.g., a simple feed-forward network with a sigmoid activation for binary classification).

Pros:

- **Holistic Understanding:** A single model can potentially understand the *relationship* between modalities (e.g., a benign text prompt combined with a harmful image, or vice versa), which separate models might miss. For example, a picture of a chemical formula combined with a text asking for its synthesis.
- **Simplicity of Deployment (Potentially):** One model to deploy, manage, and monitor for the combined safety task.
- **Emergent Safety Behaviors:** The model might learn subtle safety signals across modalities that humans haven't explicitly programmed.

Cons:

- **Complexity of Training:** Requires vast amounts of multi-modal, safely-labeled data. Training such models from scratch is immensely resource-intensive. Fine-tuning is more feasible but still demanding.
- **Data Scarcity for Harmful Multi-modal Content:** Generating or collecting diverse, labeled multi-modal harmful content (especially for CBRN) is extremely challenging.
- **Interpretability Challenges:** Debugging why a multi-modal model made a certain safety classification can be harder than for a single-modality model due to the intertwined representations.
- **Latency/Resource Overhead:** A large multi-modal model might have higher inference latency and resource requirements compared to smaller, specialized unimodal classifiers.

Option B: Multiple, Specialized Uni-Modal Models with an Orchestrator (Modular Approach)

Description: Employ separate, specialized models for each input modality, and then use an orchestration layer to combine their safety predictions for a final decision. This aligns with a "defense-in-depth" strategy.

Architecture:

- **Text Safety Classifier:** A dedicated model (like the one discussed in point 1) for classifying harmful text (user prompt, LLM output).
- **Image Safety Classifier:** A dedicated computer vision model trained to classify harmful images (e.g., graphic content, images related to dangerous activities, CBRN symbols).
- **Orchestration/Decision Logic:** A simple rule-based system or a lightweight ensemble model that takes the outputs (e.g., probabilities) from the text and image classifiers and makes a final safety decision. This could be:
 - If *either* text or image is flagged as harmful, block.
 - If both are benign, allow.
 - More complex logic for nuanced cases.

Pros:

- **Modularity & Specialization:** Each model can be highly optimized for its specific modality and safety task. Easier to train, update, and debug individual components.
- **Data Efficiency:** Can leverage existing large unimodal datasets (e.g., for general image safety, text toxicity) more easily.
- **Faster Iteration:** Changes or improvements to one modality's classifier don't necessarily require retraining the others.
- **Clearer Interpretability:** When a specific input triggers a flag, it's easier to pinpoint *which* modality and *which* classifier caused it.
- **Lower Latency (Potentially):** Smaller, specialized models can often achieve lower inference latency. Classifiers can run in parallel.
- **Resilience:** Failure in one classifier doesn't necessarily bring down the entire safety system if the orchestrator has fallback logic.

Cons:

- **Lack of Cross-Modal Understanding:** This approach might struggle with scenarios where the harm arises from the *combination* of modalities, rather than a single one being independently harmful (e.g., a benign image of chemicals combined with a text prompt that makes it dangerous). The orchestration layer has to explicitly encode these cross-modal rules.
- **Increased Orchestration Complexity:** Managing multiple models and their interactions adds complexity at the system level.

Recommendation for Safety Classifiers at Anthropic:

Given the context and the critical nature of safety, the **multiple, specialized uni-modal models with an orchestrator (Modular Approach)** is often preferred for initial deployment and rapid iteration, especially for high-stakes scenarios like CBRN. This allows for:

- **Defense in Depth:** Multiple layers of safety checks.
- **Clearer Auditability:** Easier to trace why a specific input was flagged.
- **Scalability of Training:** Can leverage existing unimodal datasets and expertise.
- **Faster Response to New Threats:** A new image-based jailbreak can be addressed by updating just the image classifier.

However, as multi-modal foundation models become more mature and data-efficient, and as understanding of cross-modal adversarial attacks increases, a **unified multi-modal model** could be integrated as an *additional* layer or for more nuanced, context-aware safety filtering in the future. The design would likely involve a multi-stage moderation pipeline that combines both approaches.

(3) Which Generative Algorithms Are Best Suited for the Task at Hand and Why? Benefits vs. Drawbacks.

This question seems slightly misaligned with your primary role building *safety classifiers*, which are discriminative models. However, it's crucial for you to understand generative models because:

1. **You're classifying their outputs:** You need to know how they work to build effective classifiers against their potential harms.
2. **Synthetic Data Generation:** Generative models are *essential* for creating the training data for your safety classifiers, especially for rare harmful examples (like CBRN misuse).
3. **Adversarial Examples:** Understanding how generative models create outputs can help you anticipate new adversarial attack vectors.
4. **"Policy-as-prompt":** As mentioned in the context, LLMs themselves can interpret policies and act as "AI judges" for RLAIIF, which is a generative task.

Let's focus on **Generative AI for Synthetic Data Generation for ML Safety Classifiers**, as this directly relates to your context.

Business Problem: Generating diverse, high-quality synthetic data (especially adversarial examples and CBRN misuse cases) to train robust ML Safety Classifiers.

Candidate Generative Algorithms:

a. Autoregressive Models (e.g., Large Language Models - LLMs)

Description: These models generate sequences (like text) one token at a time, predicting the next token based on all preceding tokens. They are excellent at capturing long-range dependencies and producing coherent, fluent outputs. This is the foundation of models like GPT, Claude, etc.

Best Suited For:

- **Textual Data Generation:** Perfect for generating diverse, realistic text, including:
 - Benign text in various styles.
 - Simulated harmful content (e.g., CBRN misuse instructions, hate speech, misinformation) by prompting the model to *act as an attacker* or to *simulate harmful user prompts*.
 - Adversarial examples for red teaming (e.g., prompt injections, subtle evasions).
 - Generating "benign" versions of harmful inputs to train classifiers against over-refusals.
- **"Policy-as-Prompt" / AI Judges:** A well-aligned LLM can serve as an AI judge to generate preference data for RLAIIF by critically evaluating other LLM outputs against a "constitution."

Benefits:

- **Quality & Coherence:** Produce highly coherent, grammatically correct, and contextually relevant text.
- **Flexibility & Control (via Prompting):** Highly steerable through prompt engineering to generate specific types of content (e.g., "generate a prompt that tries to get the AI to explain how to make a chemical weapon"). This is crucial for targeted synthetic data generation.
- **Scalability:** Can generate vast quantities of data quickly once configured.
- **Domain Adaptation:** Can be fine-tuned on specific domain data (e.g., scientific texts for CBRN context) to improve relevance.

Drawbacks:

- **Computational Cost:** Training and running large LLMs is very expensive.
 - **"Alignment Paradox":** When generating harmful content for training, there's a risk the model might learn from it, or that the "AI judge" itself might be misaligned. Requires careful sandboxing and validation.
 - **Hallucinations/Inaccuracies:** LLMs can "confabulate" information, which might lead to inaccurate or unrealistic adversarial examples if not carefully curated.
 - **Bias Amplification:** If not carefully prompted and filtered, the LLM can amplify biases present in its training data when generating new content.
-

b. Variational Autoencoders (VAEs) and Generative Adversarial Networks (GANs)

Description:

- **VAEs:** Learn a probabilistic mapping from input data to a latent space, then sample from this latent space to generate new data points. They are good for creating diverse, albeit sometimes blurry, outputs.
- **GANs:** Consist of a generator (creates new data) and a discriminator (tries to distinguish real from fake data), which are trained in an adversarial manner. GANs are known for generating high-quality, realistic samples, especially images.

Best Suited For:

- **Image Data Generation (for Image Safety Classifiers):** If your safety classifier needs to handle image inputs, GANs (and to a lesser extent, VAEs) are strong candidates for generating synthetic images. This could include:
 - Images depicting harmful acts.
 - Adversarial image examples (small perturbations to benign images to make them seem harmful to a classifier).
 - Images with specific safety-relevant features.
- **Structured Data / Tabular Data:** Less relevant for LLM safety classifiers directly, but could be useful if the system involved structured metadata.

Benefits:

- **Realistic Output (GANs):** Can generate very high-fidelity and diverse images that are hard to distinguish from real ones.
- **Latent Space Manipulation (VAEs):** VAEs allow for smooth interpolation in the latent space, enabling the generation of variations of existing data points, useful for creating adversarial examples with controlled perturbations.

Drawbacks:

- **Training Instability (GANs):** GANs are notoriously difficult to train, often suffering from mode collapse (generating limited diversity) or training instability.
- **Interpretability:** VAEs and GANs are often black boxes, making it hard to understand *why* they generated a particular harmful example.
- **Text Generation Limitations:** While there are some text GANs/VAEs, they generally struggle to produce the coherence and long-range dependencies of autoregressive models for natural language. They are not the primary choice for text generation.

- **Data Requirements:** Still require substantial datasets for training.

c. Diffusion Models

Description: These models work by iteratively denoising a random input (pure noise) until it becomes a coherent data sample. They have gained significant traction for their ability to generate incredibly high-quality and diverse images.

Best Suited For:

- **High-Quality Image Data Generation:** Currently the state-of-the-art for generating realistic and diverse images, including:
 - Photo-realistic harmful images for training image safety classifiers.
 - Generating images that could be used in multi-modal adversarial attacks (e.g., an image of a seemingly innocent object that, when interpreted by an LLM, triggers a harmful response).

Benefits:

- **Exceptional Quality:** Produce highly detailed and realistic images.
- **Diversity:** Excellent at generating a wide variety of samples.
- **Controllability:** Often allow for conditioning on text prompts (text-to-image diffusion) or other inputs, offering fine-grained control over the generated content.

Drawbacks:

- **Computational Cost:** Very expensive to train and often slower for inference compared to GANs (though speed is improving).
- **Interpretability:** Similar to GANs, understanding the internal generation process is challenging.
- **Not for Text Generation:** Primarily used for image and audio generation, not directly for text.

Summary for ML Safety Classifier Context:

For **text-based safety classifiers**, **Autoregressive LLMs** are by far the most suitable generative algorithm for synthetic data generation due to their ability to produce high-quality, diverse, and steerable text, including complex adversarial prompts and CBRN misuse scenarios.

For **image-based safety classifiers**, **Diffusion Models** are the current best choice for generating high-quality and diverse synthetic images for training and testing. GANs could also be considered if computational resources are a constraint for diffusion models.

(4) How Easily Can the System Adapt If New Input Modalities or Outputs Are Introduced Later?

This question probes the **flexibility and extensibility** of your system design. For an ML safety classifier, adaptability is crucial because new types of harm, new modalities, and new adversarial attacks are constantly emerging (the "arms race" dynamic).

Let's stick with the multi-modal safety classifier example (text + image).

Current State (Assumption based on Q2):

We've implemented a modular multi-modal safety classifier using specialized uni-modal models for text and image, coordinated by a lightweight orchestration layer.

Scenario A: Introducing a New Input Modality (e.g., Audio/Speech)

Problem: The LLM now has voice capabilities (e.g., it can process spoken commands, or generate speech). We need to classify harmful spoken content (e.g., voice-based jailbreaks, instructions for harm delivered via speech).

Adaptation Strategy:

1. **Develop a New Uni-Modal Safety Classifier for Audio:**
 - **ML Task Formulation:** Speech classification (e.g., `harmful_audio` vs. `harmless_audio`).
 - **Data Collection:** Collect/synthesize audio samples of benign speech, as well as simulated harmful speech (e.g., voice-based phishing attempts, threats, instructions for CBRN delivered verbally). Tools like Whisper can be used for transcription to link audio to textual context for labeling.
 - **Model Selection:** A robust audio classification model (e.g., a fine-tuned Wav2Vec 2.0 or a custom CNN/RNN architecture for audio).
 - **Integration:** This model would ideally be pre-processed (e.g., transcribe speech to text) then fed into an audio-specific safety classifier or directly into a multi-modal encoder if a unified model is used.
2. **Update the Orchestration/Decision Layer:**
 - The existing orchestration logic needs to be updated to incorporate the output of the new audio safety classifier.
 - For instance, the rule could become: "If text, image, *OR* audio is flagged as harmful, block the overall interaction."
 - This might involve adding a new branch in the decision tree or a new input to the ensemble model.

Ease of Adaptation:

- **Relatively Easy (due to modular design):** Our modular approach makes this quite adaptable. We can develop, test, and deploy the audio classifier largely independently. The core text and image classifiers remain untouched.
- **Minimal Impact on Existing Functionality:** New functionality (audio safety) is added without disrupting the established text and image safety pipelines.
- **Main Challenge:** The primary challenge lies in the **data acquisition and labeling** for the new modality, especially for high-stakes harmful content, and ensuring low-latency inference for real-time audio. The orchestration logic itself should be simple to update.

Scenario B: Introducing a New Output Modality (e.g., Video Generation)

Problem: The LLM can now generate short videos based on prompts. We need to ensure these generated videos do not contain harmful content.

Adaptation Strategy:

1. **Develop a New Uni-Modal Safety Classifier for Video:**
 - **ML Task Formulation:** Video classification (e.g., `harmful_video` vs. `harmless_video`). This is significantly more complex than text or image.
 - **Data Collection:** Acquire/synthesize video clips representing various levels of harm. This is a massive undertaking. Might involve using existing video datasets and leveraging generative models (e.g., Stable Diffusion Video, other video-to-video models) to create variations, along with expert labeling.
 - **Model Selection:** A video classification model (e.g., a 3D CNN, a transformer-based video model that processes frames and audio streams). This would likely be a computationally intensive model.
 - **Integration:** The video classifier would run on the generated video output *before* it's shown to the user.
2. **Update the Orchestration/Decision Layer:**
 - The orchestration logic needs to incorporate the new video safety classifier's output. The system would wait for the video generation and its subsequent safety classification before releasing it.

Ease of Adaptation:

- **Moderately Challenging (but feasible with modularity):** While the *system architecture* supports adding a new modular component, the **complexity of the new video classifier itself** and the **difficulty of acquiring/generating high-quality labeled video data** are the primary challenges.
- **Latency Impact:** Video processing for safety classification can introduce significant latency, especially for real-time applications. This would require careful optimization (e.g., running the classifier asynchronously, using smaller models for initial checks).
- **Resource Demands:** Video models are computationally very demanding.

Scenario C: Adapting to New Output Semantics / Policy Shifts (e.g., "Policy-as-Prompt" for Moderation)

Problem: Instead of explicit binary classifiers for every type of harm, the company wants to move towards a more flexible "policy-as-prompt" approach where the moderation logic is defined by natural language guidelines that LLMs can interpret.

Adaptation Strategy:

1. **Shift Classifier Paradigm:**
 - Instead of training a fixed text classifier for CBRN, you might use a powerful, well-aligned LLM (the "AI Judge" or "Moderation LLM") to directly evaluate

outputs based on a dynamically updated "constitution" or set of policies provided as a prompt.

- This moderation LLM would analyze the generated content and provide a confidence score or a categorical output (e.g., "violates CBRN policy," "harmless").

2. Update the Orchestration/Decision Layer:

- The orchestration layer would call this moderation LLM instead of the traditional discriminative classifier.

Ease of Adaptation:

- **High (in terms of policy iteration):** This approach offers unparalleled flexibility for rapid policy iteration. New safety policies (e.g., "model should not engage in political advocacy") can be incorporated by simply updating the prompt of the moderation LLM, without retraining a full discriminative classifier.
- **Challenges:** The "alignment paradox" is a key concern here. Ensuring the moderation LLM itself is perfectly aligned and doesn't introduce its own biases or hallucinate when interpreting policies is critical. This requires robust interpretability and observability on the moderation LLM itself. Computational cost of running a large LLM for every moderation step can also be high.

Overall Conclusion on Adaptability:

The ease of adaptation heavily depends on the **initial system design's modularity and extensibility**.

- **Modular, Layered Architectures (like the one discussed for Q2):** These are inherently more adaptable. New components can be added without major overhauls, supporting the "defense in depth" strategy crucial for AI safety.
- **Loose Coupling:** Ensuring that components are loosely coupled (e.g., classifiers expose a simple API) allows for easier swapping or addition of new modules.
- **Standardized Data Formats:** Using consistent data representations (e.g., embeddings, standardized text formats) across modalities simplifies integration.
- **Flexible Orchestration Layer:** A simple, easily configurable orchestration logic (rule-based or lightweight ensemble) can adapt quickly to incorporate new signals.
- **Emphasis on Synthetic Data & AI-Assisted Labeling:** The ability to rapidly generate diverse, labeled data for new modalities using existing LLM capabilities (as discussed in Q3) is a huge enabler for quick adaptation.

The primary bottlenecks for adaptation are typically **data acquisition/labeling for the new modality/harm type** and the **computational resources/latency required** for the new safety models. Your ability to speak to these practical challenges and how to mitigate them (e.g., through synthetic data, efficient model architectures, asynchronous processing) will demonstrate a mature understanding of ML system design.

VAEs are great for anomaly detection → train a VAE on "safe" data to be used as a pre-filter. Now inputs that cannot be reconstructed well or latent representation deviates significantly can

be flagged as an anomaly. These can help understand latent representations for interpretability purposes.

Tiered System

1. Tier 1 Defense (Fast Rules & Filters):
 - a. Keyword list, regex filters, potentially simple on-device classifier
 - b. Very low latency
 - c. Goal: Drop obviously harmful content fast
2. Tier 2 Defense (Lightweight Models):
 - a. Features fetched from pre-computed embeddings in a NoSQL store
 - b. Models trained using data prepared from the Data Lake
 - c. Near-real-time scoring
3. Tier 3 Defense (Heavy ML Inference):
 - a. Deep models for nuanced cases (e.g., multimodal harmful meme detection)
 - b. Training on multi-terabyte datasets prepared from the Data Lake
 - c. Possibly async or deferred scoring

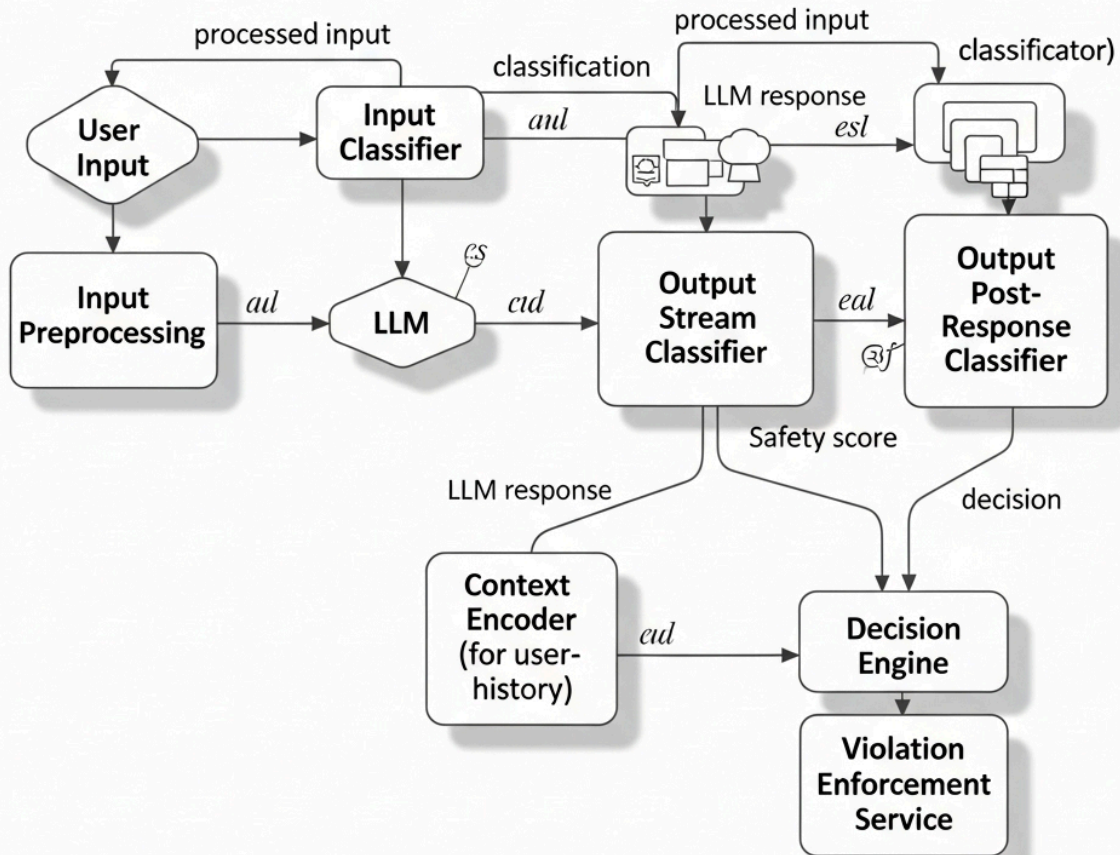
For Generating Synthetic Harmful Data (Red Teaming): "We'd likely use **Top-P Sampling with a higher temperature** on our internal LLMs. This is because we need to generate a *diverse range* of subtle, creative, and novel 'jailbreak' attempts or harmful scenarios that humans might not easily conceive. Maximizing diversity helps our safety classifiers learn to detect a broader spectrum of threats, rather than just the most obvious ones. We'd want the generative model to explore the 'edges' of harmful content generation."

For Internal Debugging/Reproducibility: "If we're debugging a specific safety incident or trying to reproduce a harmful output, we might temporarily use **Greedy Search or Beam Search** to see the most probable (and thus often most common) failure modes of the model. This offers deterministic outputs for analysis."

For Real-time User-Facing Interactions (where safety filters are applied): "The core LLM generating content for users would likely use a combination of **Top-P sampling with an optimally tuned temperature**. The goal here is to balance user experience (creativity, helpfulness) with adherence to alignment (harmlessness). Our real-time **Constitutional Classifiers and token-level filtering** would act as a crucial 'post-sampling' safety layer, analyzing the generated sequence regardless of the sampling method used, potentially even blocking or re-sampling problematic tokens on the fly, as per Anthropic's multi-layered defense strategy."

[Design] Brief: Defense-in-Depth

AI Safety Inference Stack



1. Clarify Requirements (My Strategic Approach)

- **Business Objective:** Ensure AI system **safety and integrity**, maximizing user trust and adherence to platform guidelines (like Anthropic's Responsible Scaling Policy).
 - **KPIs:**
 - **Jailbreak Prevention Rate:** Target >95% (critical for input safety).
 - **False Positive Rate (Unnecessary Refusal):** <0.5% (crucial for user experience).
 - **Time-to-Mitigation for New Attacks:** <24 hours (adaptive learning).
 - **Harmful Content Blocking:** 100% for high-confidence, severe cases (e.g., CBRN, child safety).
- **Core Features:**
 - **Multi-modal:** Text (prompts, responses), Image (if model is multi-modal), Audio (if model handles speech).
 - **Real-time Filters (Different Latencies):**

- **Pre-prompt (Input):** Block malicious user input before LLM inference.
 - **Mid-generation (Streaming/Token-level):** Intervene during LLM output generation.
 - **Post-response (Output):** Final check on complete LLM response.
- **Adaptive Learning:** Continuous updates for new attacks.
- **Hierarchical Escalation:** Automated → Human Review → Expert.
- **Explainability:** Provide reasons for blocking/demotion to users.
- **Data Sources:**
 - **Synthetic Adversarial Data:** Crucial for zero-shot detection of new threats (e.g., **Constitutional AI generated** for diverse harmful/benign pairs).
 - **Red Team Outputs:** From ongoing adversarial testing.
 - **Production Logs:** User interactions, blocked content, false positive reports.
 - **Threat Intelligence:** External feeds on new attack patterns.
- **Scale & Performance:**
 - **Volume:** 50M+ daily interactions, 100K+ QPS peak (massive scale).
 - **Latency:**
 - **Input classification:** <50ms (to prevent prompt injection before LLM compute).
 - **Output classification:** <100ms (for post-processing, potentially higher for full video analysis).
 - **Token-level (mid-generation):** <10ms per token (for blocking/steering).
 - **Total overhead:** <200ms (minimal impact on core LLM response time).
 - **Availability:** 99.99% uptime, graceful degradation.

2. Frame as ML Task (My "Catalytic Clarity" Framing)

- **Primary Task: Multi-Task, Multi-Modal Discriminative Classification.**
 - Input: User prompt (text, image, audio), partial/full LLM response.
 - Output: Probabilities for specific harm categories (e.g., Violence, Nudity, Hate Speech, Self-Harm, Fraud), with confidence scores.
 - Goal: Pinpoint *type* of harm for explainability and specific mitigation.
- **Secondary Tasks:**
 - **Generative Synthetic Data Creation:** Using LLMs (e.g., Constitutional AI principles) to create training data for classifiers.
 - **Anomaly Detection:** Identify novel jailbreak patterns or emerging threats.

3. Data Preparation (Operational Excellence)

- **Strategy: Continuous, Automated Data Pipelines** supplemented by expert human feedback.
- **Key Data Sources:**
 - **Constitutional Synthesis:** Automatically generate vast, diverse harmful/benign examples (prompts/responses) based on defined safety principles using internal LLMs. This is the **scalable** solution for data.

- **Adversarial Augmentation:** Apply known jailbreak techniques (e.g., base64 encoding, role-play, prompt injection variations) to benign data to create adversarial training examples.
- **Red Team Outputs:** High-quality, real-world adversarial examples from dedicated security teams and bug bounty programs.
- **Production Feedback:** User reports (positive and negative), model predictions, human review outcomes.
- **Data Quality & Safety:**
 - **Deduplication:** Remove near-duplicates across modalities/languages.
 - **Bias Detection:** Proactively audit datasets for demographic, linguistic biases. Crucial for fair system.
 - **Privacy:** PII scrubbing, anonymization (GDPR, etc.).
 - **Version Control:** Ensure reproducibility of datasets.
- **Efficiency:**
 - **Columnar Storage (Parquet):** For efficient storage and querying of features and large datasets.
 - **Semantic Indexing (FAISS/Elasticsearch):** For fast similarity search of embeddings (for prompt/response vectors) and full-text search of attack patterns.
 - **Caching:** Hot classification results (Redis), frequently accessed model artifacts, historical attack patterns.

4. Modeling (Strategic Realignment)

- **Overall Architecture: Defense-in-Depth Ensemble.**
 - Multiple specialized models, each with specific roles and latency profiles.
 - Final decision through a meta-classifier or rule-based engine.
- **Key Model Types & Latency Constraints:**
 - **Input Classifier (Pre-Prompt):**
 - **Purpose:** Detect immediate jailbreaks, policy violations in user's input *before* the LLM even processes it. Prevents costly LLM inference on malicious prompts.
 - **Latency:** <50ms. Must be extremely fast.
 - **Type:**
 - **Lightweight Transformer (e.g., DistilMBERT/custom small BERT-like model):** Optimized for low latency, multilingual.
 - **Rule-based System:** For immediate blocking of known explicit keywords/patterns (high precision, low recall but very fast).
 - **Simple ML (e.g., SVMs/XGBoost on extracted features):** For high-precision, low-compute cases.
 - **Output:** Binary flag (block/allow) + confidence score.
 - **Output Stream Classifier (Mid-Generation / Token-Level):**
 - **Purpose:** Detect harmful content *as it's being generated*, allowing for immediate interruption or steering of the LLM. Prevents exposure to full harmful responses.
 - **Latency:** <10ms per token. Extremely critical for real-time interaction.
 - **Type:**

- **Very Lightweight ML Classifiers (e.g., small RNNs, shallow feed-forward nets) or Rule-based filters:** Operate directly on token logits or small contextual windows.
 - **Quantized Models:** For maximal speed.
 - **Output:** Binary flag (block/continue) per token, or probability adjustment.
 - **Output Post-Response Classifier (Full Response):**
 - **Purpose:** Final, comprehensive check on the complete LLM generated response (text, and potentially images/videos if multi-modal). Catches complex, subtle harms.
 - **Latency:** <100ms. More compute-intensive than input/stream.
 - **Type:**
 - **Robust Transformer-based (e.g., RoBERTa-large, fine-tuned custom LLM) for text.**
 - **Pre-trained Vision/Video Models (e.g., CLIP-based, ViT for images, Temporal/3D ConvNets for video):** For multi-modal outputs.
 - **Multi-Task Classifier:** To classify specific harm types (e.g., violence, nudity, hate speech).
 - **Output:** Probabilities for all harm categories + confidence.
 - **Context Encoder:**
 - **Purpose:** Provide user history, session context to classifiers for personalized or context-aware classification.
 - **Latency:** Flexible (can be pre-computed/cached).
 - **Type:** Transformer model (e.g., fine-tuned BERT/LLM for session context encoding).
 - **Anomaly Detector:**
 - **Purpose:** Identify novel jailbreak patterns or emerging threats that existing classifiers might miss. Operates more offline/batch.
 - **Latency:** Batch, hourly/daily.
 - **Type:** Autoencoders, Isolation Forests, statistical models on embedding space.
 - **Meta-Classifer / Decision Engine:**
 - **Purpose:** Combines scores from all classifiers, applies thresholds, business rules, and confidence estimates to make a final block/demote/allow decision.
 - **Latency:** Very low (<5ms, rule-based or shallow NN).
 - **Type:** Weighted voting, stacking, or simple learned NN.
- **Training Objectives & Losses (Multi-Objective Optimization):**
 - **Primary: Weighted Cross-Entropy Loss** per harm category (for multi-task classification), handling class imbalance.
 - **Calibration Loss (e.g., Focal Loss):** Improves confidence calibration, crucial for setting reliable thresholds and managing false positives/negatives.
 - **Robustness Loss (Adversarial Training):** Hardens models against jailbreaks by training on adversarial examples.
 - **Fairness Loss:** Constraints (e.g., demographic parity) to reduce bias in predictions across sensitive attributes.
- **Speed Up Training:**
 - **Distributed Training:** Data, model, and pipeline parallelism across H100 GPUs.

- **Optimization Techniques:** Mixed precision, gradient checkpointing, dynamic loss scaling.

5. Evaluation (Diagnostic Mirroring)

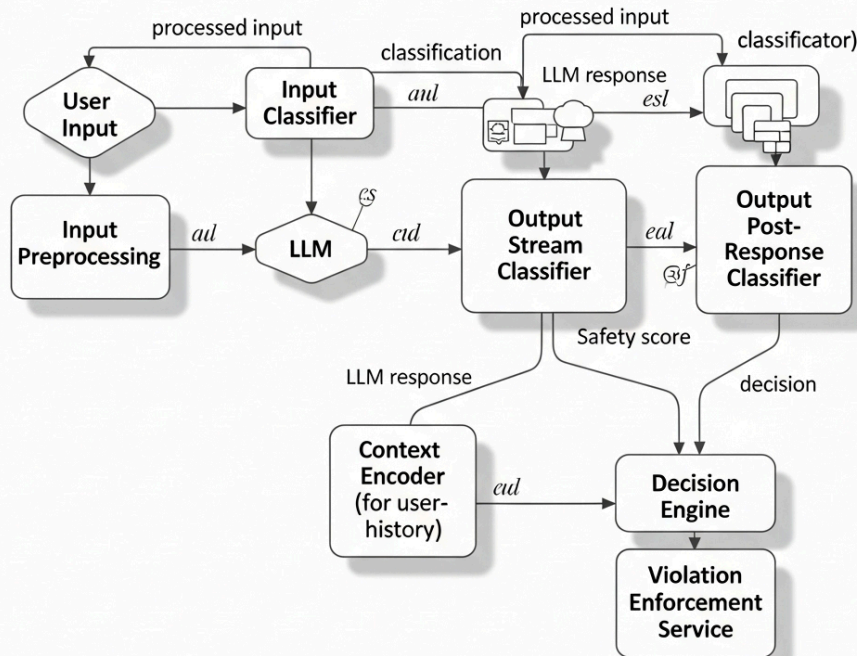
- **Offline Evaluation (Per Harm Category):**
 - **Metrics: PR-AUC and ROC-AUC** for each harm category. PR-AUC is especially critical for rare, positive classes (harm).
 - **Calibration Metrics:** Brier Score, Expected Calibration Error (ECE) for robust thresholding.
 - **Robustness Metrics:** Attack Success Rate (ASR) on new/held-out adversarial test sets (jailbreaks).
 - **Fairness Metrics:** Demographic parity, equalized odds (ensure consistent performance across user groups).
- **Human-in-the-Loop Evaluation:**
 - **Expert Review:** Domain experts review edge cases, provide ground truth, and evaluate model explanations.
 - **Red Team Assessment:** Continuous, structured adversarial testing (with bug bounties) to proactively find system vulnerabilities.
 - **User Studies:** Controlled environments to assess real-world jailbreak attempts.
- **Specialized Safety Evaluation:**
 - **Hallucination Detection:** For LLM-generated explanations or safety reasoning.
 - **Novel Attack Detection:** Evaluate anomaly detector's performance.
- **Online Evaluation (Real-time Feedback & Safety KPIs):**
 - **Harmful Impressions: (Key Metric)** Total exposure to harmful content. Directly measures platform safety impact.
 - **Proactive Rate:** % of harm detected by system vs. user reports. **Higher is better.**
 - **Valid Appeals Rate:** % of correct user appeals. **Lower is better (indicates fewer false positives).**
 - **Latency Monitoring:** P50, P95, P99 for all classification stages.
 - **User Satisfaction:** Surveys, NPS scores (monitor impact of safety interventions).
 - **Attack Detection Time:** Track time from new attack observed to full mitigation deployment.
 - **Escalation Rate:** Monitor frequency of human review/expert intervention.

6. ML System Architecture (Strategic Brevity)

- **Overall System Design:**
 - User Input (Prompt/Response) → Real-time Filters → LLM (for generation) → Post-Processing Filters → Final Output to User
 - **Parallel Monitoring & Feedback Loops:** All stages feed into logging, anomaly detection, and human review.
- **Core Components (Inference Stack):**
 - **Input Preprocessing:** Normalization, tokenization, embedding.
 - **Input Classifier(s):** Fast, lightweight models for immediate prompt safety.

- **LLM:** The core generative model (Claude/ChatGPT).
- **Output Stream Classifier:** Token-level, ultra-low latency.
- **Output Post-Response Classifier:** Comprehensive check on full response.
- **Context Encoder:** Enriches classification with user/session data (features store).
- **Decision Engine:** Combines classifier outputs, applies policy rules.
- **Explanation Generator:** Provides user-facing reasons for blocking.
- **Violation Enforcement Service:** Takes action (block, demote, alert human).
- **Scalability:**
 - **Microservices:** Decoupled classification services for independent scaling.
 - **Auto-scaling (Kubernetes HPA):** Based on latency/queue depth.
 - **Geographic Distribution:** Low-latency regional deployments.
 - **Model Sharding:** Distribute large models (e.g., if LLM is part of classifier itself).
- **Model Management:**
 - **A/B Testing / Canary Releases:** Gradual, safety-guarded rollouts of new classifier versions.
 - **Blue/Green Deployments:** For zero-downtime updates with quick rollback.
 - **Model Versioning:** Clear lineage for audits.
- **Continuous Learning & Adaptation:**
 - **Real-time Learning Pipeline:** Detects new jailbreaks, generates synthetic data, triggers incremental model retraining.
 - **Human Feedback Integration:** Streamlined expert review, user appeal process, red team data feed into retraining loops.
 - **Knowledge Management:** Centralized database of threat intelligence and attack patterns.

AI Safety Inference Stack



7. Safety, Security & Bias Mitigation (High Integrity)

- **Defense in Depth:** Multi-layered approach (input, mid-gen, output, human review) ensures redundancy.
 - **Robustness:** Adversarial training, chaos engineering (testing resilience).
 - **Privacy:** End-to-end encryption, PII masking.
 - **Bias Mitigation:**
 - **Proactive:** Auditing training data for bias (e.g., in constitutional synthetic data), fairness constraints during training, adversarial debiasing.
 - **Ongoing Monitoring:** Real-time demographic breakdown of classifications, regular audits, user community feedback.
 - **Rapid Response:** Protocols for addressing discovered bias issues.
-

[Design] Defense-in-Depth ML Classifier System Design Blueprint

Clarify Requirements

Business Objectives and Metrics

- **Safety KPIs:** Jailbreak prevention rate (target: >95%), false positive rate (<1%), time-to-mitigation for new attacks (<24 hours)
- **User Experience:** Response latency (<200ms additional overhead), user satisfaction (maintain >4.5/5), unnecessary refusal rate (<0.5%)
- **Operational Metrics:** System uptime (99.9%), throughput (support 10M+ daily interactions), cost per classification (<\$0.001)
- **Regulatory Compliance:** CBRN content blocking (100% for high-confidence cases), policy violation detection (>90% recall)

Core Features

- **Multi-modal Classification:** Text, image, audio, video content analysis
- **Real-time Filtering:** Pre-prompt, mid-generation, post-response classification
- **Adaptive Learning:** Continuous retraining on new attack patterns
- **Hierarchical Escalation:** Automated blocking → human review → expert analysis
- **Context-Aware Analysis:** User history, session context, intent understanding

Data Sources

- **Synthetic Adversarial Data:** Constitutional AI generated harmful/benign examples
- **Red Team Outputs:** Human adversarial testing results, bug bounty submissions
- **Production Logs:** User interactions, blocked content, false positive reports
- **Threat Intelligence:** External security feeds, academic research, industry sharing
- **Labeled Datasets:** Human-annotated harmful content, expert domain knowledge

Scale and Performance Requirements

- **Volume:** 50M+ daily user interactions, 100K+ queries per second at peak
- **Latency:** <50ms input classification, <100ms output classification, <200ms total overhead
- **Growth:** 10x scale growth over 2 years, support for new modalities quarterly
- **Availability:** 99.99% uptime, graceful degradation under attack, geographic distribution

Frame as ML Task

Task Classification

- **Primary:** Multi-class discriminative classification (harmful categories: CBRN, fraud, harassment, etc.)
- **Secondary:** Generative synthetic data creation for training augmentation
- **Tertiary:** Anomaly detection for novel attack pattern identification

Input/Output Design

- **Inputs:**
 - Text prompts (raw, encoded, multi-language)
 - Generated responses (partial, complete)
 - Context vectors (user history, session data)
 - Multi-modal content (images, audio embeddings)

- **Outputs:**
 - Classification probabilities per harm category
 - Confidence scores and uncertainty estimates
 - Explanatory features for human review
 - Recommended actions (block, flag, allow)

Model Architecture Strategy

- **Multiple Specialized Models:** Domain-specific classifiers (CBRN, fraud, etc.) rather than single multi-class
- **Ensemble Approach:** Constitutional classifiers + traditional ML + rule-based systems
- **Modality-Specific:** Separate models for text, vision, audio with fusion layer
- **Multi-Scale:** Token-level, sentence-level, document-level classification

Algorithm Selection

- **Constitutional Classifiers:** Transformer-based models trained on synthetic constitutional data
- **Traditional ML:** Gradient boosting (XGBoost) for structured features, SVMs for high-precision cases
- **Deep Learning:** BERT/RoBERTa variants for text, ConvNets for images, specialized architectures for audio
- **Ensemble Methods:** Weighted voting, stacking, or learned combination strategies

Data Prep

Data Collection Strategy

- **Constitutional Synthesis:** Use target LLM to generate diverse harmful/benign examples following constitutional principles
- **Adversarial Augmentation:** Transform benign examples with known jailbreak techniques (encoding, role-play, injection)
- **Real-world Sampling:** Carefully curated production examples (anonymized, consent-based)
- **Expert Annotation:** Domain experts label edge cases, provide ground truth for complex scenarios
- **Continuous Ingestion:** Real-time pipeline for new attack patterns, threat intelligence feeds

Data Cleaning and Quality

- **Deduplication:** Semantic similarity detection to remove near-duplicates across languages/encodings
- **Quality Filtering:** Remove low-quality synthetic examples, ensure human expert validation
- **Bias Detection:** Systematic analysis for demographic, linguistic, cultural biases in training data
- **Privacy Protection:** PII scrubbing, differential privacy techniques, data anonymization
- **Version Control:** Data lineage tracking, reproducible dataset creation, audit trails

Data Efficiency and Storage

- **Columnar Storage:** Parquet format with bloom filters for fast harmful content queries
- **Hierarchical Sharding:**
 - Tier 1: By harm category (CBRN, fraud, etc.)
 - Tier 2: By language/region
 - Tier 3: By time period for temporal analysis
- **Semantic Indexing:**
 - FAISS vector indices for embedding-based similarity search
 - Elasticsearch for full-text search across harmful content patterns
 - Specialized indices for jailbreak pattern matching
- **Caching Strategy:**
 - L1: Hot classification results (Redis)
 - L2: Frequently accessed model artifacts
 - L3: Historical attack pattern database

Modeling

Model Architecture

Constitutional Classifier Design

- **Input Classifier:** RoBERTa-large fine-tuned on constitutional synthetic data
- **Output Classifier:** Similar architecture, optimized for generated text analysis
- **Context Encoder:** Transformer model encoding user/session context for personalized classification
- **Fusion Layer:** Multi-head attention mechanism combining input, output, and context signals

Ensemble Architecture

Input → [Constitutional Classifier] → Score₁
→ [Traditional ML Pipeline] → Score₂
→ [Rule-based System] → Score₃
→ [Anomaly Detector] → Score₄
→ [Meta-Classifer] → Final Decision

Training Process and Datasets

Multi-Stage Training Pipeline

1. **Foundation Training:** Large-scale pretraining on diverse text corpus
2. **Constitutional Training:** Fine-tuning on synthetic harmful/benign examples
3. **Adversarial Training:** Hardening against known jailbreak techniques
4. **Domain Adaptation:** Specialized training for CBRN, fraud, etc.
5. **Continuous Learning:** Online learning from production feedback

Dataset Composition

- **Constitutional Synthetic:** 70% (balanced harmful/benign across categories)
- **Real-world Samples:** 20% (carefully curated and anonymized)

- **Adversarial Examples:** 10% (red team outputs, jailbreak attempts)

ML Objectives and Loss Functions

Multi-Objective Optimization

- **Primary Loss:** Weighted cross-entropy with class balancing for harm categories
- **Calibration Loss:** Focal loss component to improve confidence calibration
- **Robustness Loss:** Adversarial training loss to improve jailbreak resistance
- **Fairness Loss:** Demographic parity constraints to reduce bias

Specialized Loss Components

$$\begin{aligned} \text{Total_Loss} = & \lambda_1 * \text{CrossEntropy_Loss} + \\ & \lambda_2 * \text{Focal_Loss} + \\ & \lambda_3 * \text{Adversarial_Loss} + \\ & \lambda_4 * \text{Fairness_Loss} + \\ & \lambda_5 * \text{Uncertainty_Loss} \end{aligned}$$

Speed Up Model Training

Distributed Training Strategy

- **Data Parallelism:** Split constitutional synthetic data across 8-16 GPUs
- **Model Parallelism:** Large transformer layers distributed across multiple devices
- **Pipeline Parallelism:** Stage different training phases across device clusters
- **Gradient Accumulation:** Handle large effective batch sizes with memory constraints

Optimization Techniques

- **Mixed Precision Training:** FP16 for forward pass, FP32 for loss computation
- **Gradient Checkpointing:** Trade compute for memory in large models
- **Dynamic Loss Scaling:** Prevent gradient underflow in mixed precision
- **Learning Rate Scheduling:** Warmup + cosine decay for stable convergence

Model Sampling and Inference

Real-time Classification Strategy

- **Greedy Classification:** Argmax for high-confidence decisions (>95% certainty)
- **Ensemble Sampling:** When confidence <95%, sample from multiple model predictions
- **Temperature Scaling:** Calibrate output probabilities for better uncertainty estimation
- **Threshold Optimization:** ROC curve analysis to set optimal decision boundaries per harm category

Uncertainty Quantification

- **Monte Carlo Dropout:** Multiple forward passes with dropout for uncertainty estimation
- **Ensemble Disagreement:** Measure variance across ensemble member predictions
- **Calibration Metrics:** Reliability diagrams, Expected Calibration Error (ECE)

Evaluation

Offline Evaluation

Standard Metrics per Harm Category

- **Classification Metrics:** Precision, Recall, F1-score, AUC-ROC per category
- **Calibration Metrics:** Brier score, Expected Calibration Error, reliability diagrams
- **Robustness Metrics:** Attack Success Rate (ASR) on adversarial test sets
- **Fairness Metrics:** Demographic parity, equalized odds across user groups

Human-in-the-Loop Evaluation

- **Expert Review:** Domain experts evaluate edge cases and model explanations
- **Red Team Assessment:** Structured adversarial testing with bounty incentives
- **User Study:** Representative users attempt to jailbreak system in controlled environment
- **Cross-validation:** Multiple expert annotators for inter-rater reliability

Specialized Safety Evaluation

- **Hallucination Detection:** Automated fact-checking of generated explanations
- **Jailbreak Resistance:** Systematic testing against known attack patterns
- **Novel Attack Detection:** Anomaly detection evaluation on held-out attack methods
- **Multi-modal Robustness:** Cross-modal attack resistance (text→image, etc.)

Online Evaluation

Real-time Performance Metrics

- **Latency Monitoring:** P50, P95, P99 classification latency across geographic regions
- **Throughput Tracking:** Queries per second, concurrent user handling
- **Error Rate Monitoring:** Classification failures, timeout rates, system errors
- **Resource Utilization:** GPU/CPU usage, memory consumption, bandwidth

User Experience Metrics

- **False Positive Rate:** User appeals, unnecessary blocking reports
- **User Satisfaction:** Post-interaction surveys, NPS scores
- **Task Success Rate:** Completion rate for legitimate user requests
- **Engagement Impact:** Session length, user retention after safety interventions

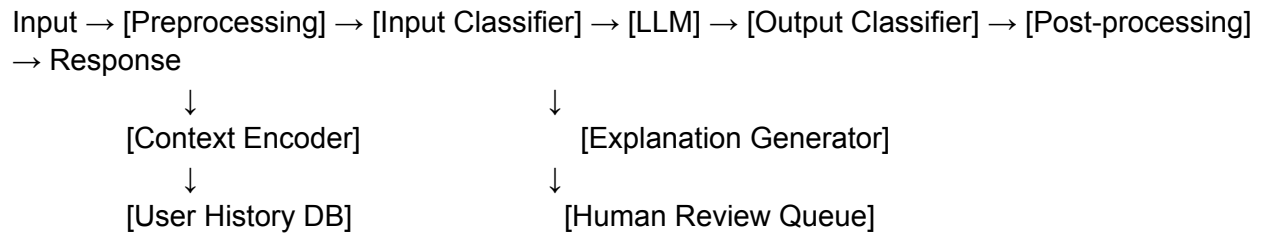
Safety Effectiveness Metrics

- **Harm Prevention:** Blocked attacks per day, prevented policy violations
- **Attack Detection Time:** Time from new attack pattern to successful blocking
- **Escalation Rate:** Cases requiring human review, expert intervention
- **Prevalence Tracking:** Harmful content trends, emerging threat patterns

ML System Architecture

Core System Components

Classification Pipeline



Real-time Inference Stack

- **Load Balancer:** Geographic routing, model version management
- **Model Serving:** TensorRT optimized models, batch inference optimization
- **Feature Store:** Real-time context features, user embeddings
- **Decision Engine:** Ensemble combination, threshold application, action determination
- **Monitoring:** Performance tracking, drift detection, alert systems

Scalability Architecture

Horizontal Scaling Strategy

- **Microservices:** Independent scaling of input/output classifiers, context encoders
- **Auto-scaling:** Kubernetes HPA based on queue depth, latency targets
- **Geographic Distribution:** Regional deployments for latency optimization
- **Model Sharding:** Distribute large models across multiple inference servers

Model Management

- **A/B Testing:** Gradual rollout of new model versions with safety guardrails
- **Blue/Green Deployment:** Zero-downtime model updates with instant rollback capability
- **Model Versioning:** Semantic versioning, compatibility tracking, rollback procedures
- **Canary Releases:** New models tested on small traffic percentage before full deployment

Continuous Learning and Adaptation

Real-time Learning Pipeline

- **Attack Detection:** Continuous monitoring for novel jailbreak patterns
- **Synthetic Data Generation:** Automated creation of training data for new attacks
- **Model Retraining:** Incremental learning, catastrophic forgetting prevention
- **Performance Monitoring:** Model drift detection, performance degradation alerts

Human Feedback Integration

- **Expert Annotation:** Streamlined workflow for expert review of edge cases
- **User Feedback:** Appeal process integration, false positive correction
- **Red Team Integration:** Continuous adversarial testing, bug bounty programs
- **Knowledge Management:** Centralized threat intelligence, attack pattern database

Safety and Security

Defense in Depth Strategy

- **Input Validation:** Schema validation, input sanitization, rate limiting
- **Model Security:** Adversarial robustness, prompt injection resistance
- **Output Filtering:** Multi-layer content filtering, explanation generation
- **Audit Trail:** Complete request logging, decision justification, compliance reporting

Robustness and Reliability

- **Graceful Degradation:** Fallback to rule-based systems during model failures
- **Circuit Breakers:** Automatic failover when classification accuracy drops
- **Chaos Engineering:** Regular testing of system resilience under attack
- **Disaster Recovery:** Multi-region backup, rapid recovery procedures

Privacy and Compliance

- **Data Privacy:** End-to-end encryption, PII detection and masking
- **Regulatory Compliance:** GDPR, SOC2, industry-specific requirements
- **Access Control:** Role-based permissions, audit logging, principle of least privilege
- **Data Governance:** Retention policies, right to deletion, data lineage tracking

Bias Detection and Mitigation

Proactive Bias Prevention

- **Training Data Auditing:** Systematic bias detection in constitutional synthetic data
- **Fairness Constraints:** Demographic parity enforcement during model training
- **Adversarial Debiasing:** Adversarial training to reduce biased predictions
- **Multi-stakeholder Review:** Diverse expert panels for bias assessment

Ongoing Bias Monitoring

- **Real-time Metrics:** Demographic breakdown of classification decisions
- **Regular Auditing:** Periodic bias assessment across user populations
- **Feedback Loops:** User community input on biased system behavior
- **Mitigation Strategies:** Rapid response protocols for discovered bias issues

[Design] Gmail Smart Compose

Questions: 1) personalized to their writing style? 2) only suggest when there's a certain level of confidence or need to suggest all the time? 3) can we use past emails or just whatever text is in the current draft? 4) language that needs to be supported? 5) controlling for bias and NSFW content? 6) how many users? Is cost a concern? Latency constraints?

Frame as ML: <Input partial text> → Smart Compose System → <output rest of the text>

Seq2seq models (RNN such as GRUs, LSTMs versus Transformers)

Feature	RNN (GRU [4], LSTM [5])	Transformer
Architecture	Simple	Complex
Training efficiency	Inefficient due to sequential processing	Efficient due to parallel processing
Effectiveness	Low as it struggles with long sequences	High as it handles long sequences
Scalability	Limited scalability	Highly scalable
Applications	Simple tasks such as time series modeling	Complex tasks such as language completion or translation

Table 1: Comparison of RNN and Transformer architectures

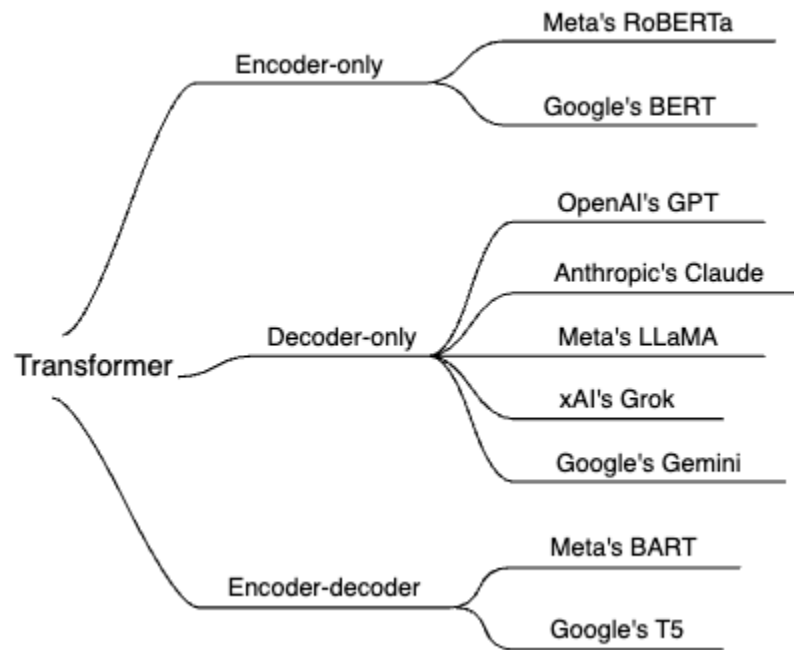
FlashAttention and GroupAttention can help mitigate the $O(N^2)$ attention computation

Data Prep:

1. Data Choice
 - a. General text data
 - i. Probably want to use this to train a foundational LLM model, since it's a lot more diverse and has a lot more data, so we can get a much more powerful model.
 - b. Email data
 - i. <Email ID, Sender, Recipient, Subject, Body>
 - ii. This is critical to adapt the specific domain of email text in general → brevity, singular focus, common phrases and styles, etc.
2. Cleaning the data
 - a. Need to clean the data text and normalize it to get rid of noise, then we need to tokenize the text and index the tokens to create the vocabulary.
 - b. Remove Unique Identifiers → jamesli@gmail.com strip out the name, phone numbers, etc.
 - c. Remove irrelevant characters or symbols like emojis in the first pass and focus more on the words, for websites you'll want to remove irrelevant text to the content, (affiliate links, etc.)
 - d. Remove duplicated data
 - e. Tokenization
 - i. Character-level, word-level, or sub-word level
 - ii. Character level will be too conflated with meaning for each character, too overloaded, so it's hard to learn a meaningful representation
 - iii. Word level typically may be too big of a vocab (e.g. hundreds of thousands), long training times, costly, etc. And OOV happens quite often
 - iv. Subword level will require more complexity to implement how to split, but can be worth the tradeoffs. Benefits are that you can decompose OOV into seen subwords, smaller vocab (sub 100k). Typical algorithms are Byte-pair Encode (BPE) or SentencePiece.
 - f. Token indexing
 - i. Token, ID → the, 0 | | <EOS>, 50,003
 - ii. Now can apply this to the raw text

Modeling:

1. Encoder, Decoder, or both?
 - a. Encoder is good for discriminative tasks (e.g. raw text → encoder → prediction on sentiment)
 - b. Decoder is good for transforming input sequence into a new output sequence
 - c. Encoder and decoder good for translation



- d.
- e. Given Smart Compose is taking in partial text and returning remainder of text, decoder-only seems good
2. Decoder-only components that need to be trained:
 - a. Token embedding → word2vec style improves semantic relevance and helps with sparsity versus one-hot encoding
 - b. Position embedding → add the position embedding of the tokens to the token embeddings element wise to ensure different embeddings for each position even for the same tokens. You can do fixed position embedding like the original transformer paper, which uses sine-cosine functions, or you can learn the position embeddings. The latter may give better performance but at more compute cost and may not generalize as well.
 - c. Multi-head attention or self-attention → basic attention, Query/Key/Value, which processes the sequence into a set of hidden states and performs cross-attention for each output token, which is a weighted sum of value vectors. Self-attention does this H times for learning different relationships, and concatenates each attention head into the linear heads. There's also a residual layer to allow for more efficient learning.
 - d. Prediction head → projects into the vocabulary space using the token probability distribution
3. Pretraining using next token prediction task with cross-entropy loss, this is using the large corpus of diverse text dataset
4. Finetuning the same model (or a pre-trained model) on email dataset
 - a. Same prediction task, same loss function, calculate loss based on the email data and prediction

- b. Can add in context from the email itself (e.g. recipient email, sender email, subject)
 - i. Dear ... → Dear John
 - c. System prompt is very important → prompt engineering
- 5. Model output sampling ([link](#))
 - a. Prefer more deterministic methods such as greedy and top P / top K since we likely would want consistency, more common phrases, and reduce the risk of inappropriate suggestions.

Eval

1. Offline
 - a. Perplexity, which gets at how accurately the model predicts the exact sequence of tokens present in text data. Mathematically, this is the average negative log-likelihood of the predicted probability given the previous tokens
 - b. ExactMatch@K, which is the percentage of generated phrases that are exactly N words long and match the first N words of the ground-truth text. Can have multiple versions of K to understand how well the model captures short, medium, and long responses.
 - c. Standard BLEU and ROUGE scores based on human evaluation can also be used here
2. Online
 - a. Acceptance rate
 - b. Usage rate
 - c. Average completion time of emails
 - d. System response latency
 - e. Feedback
 - f. Human eval for qualitative assessments

ML System design

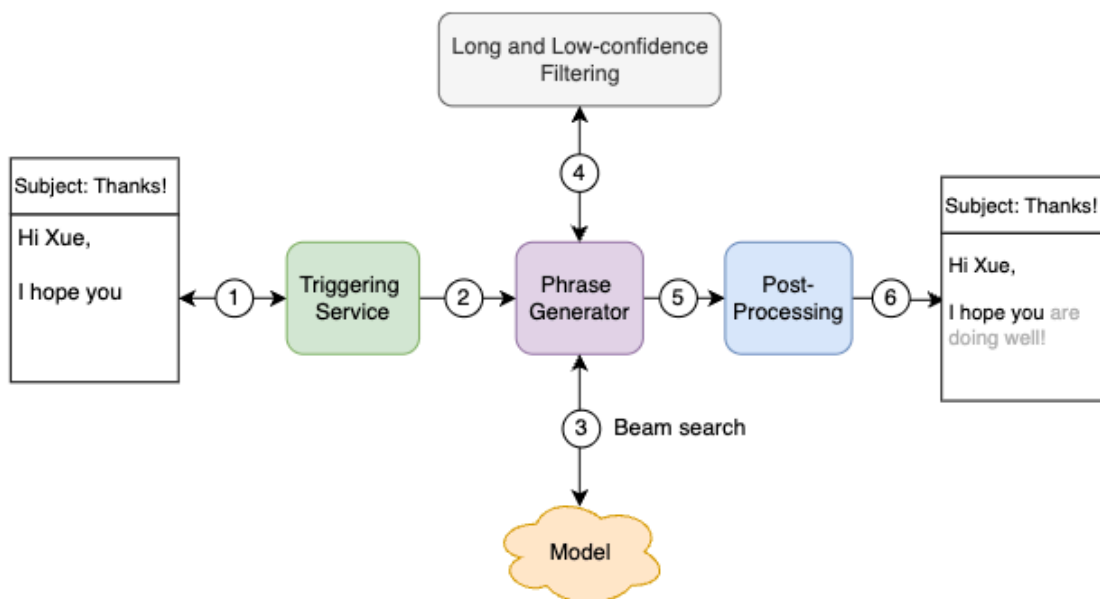
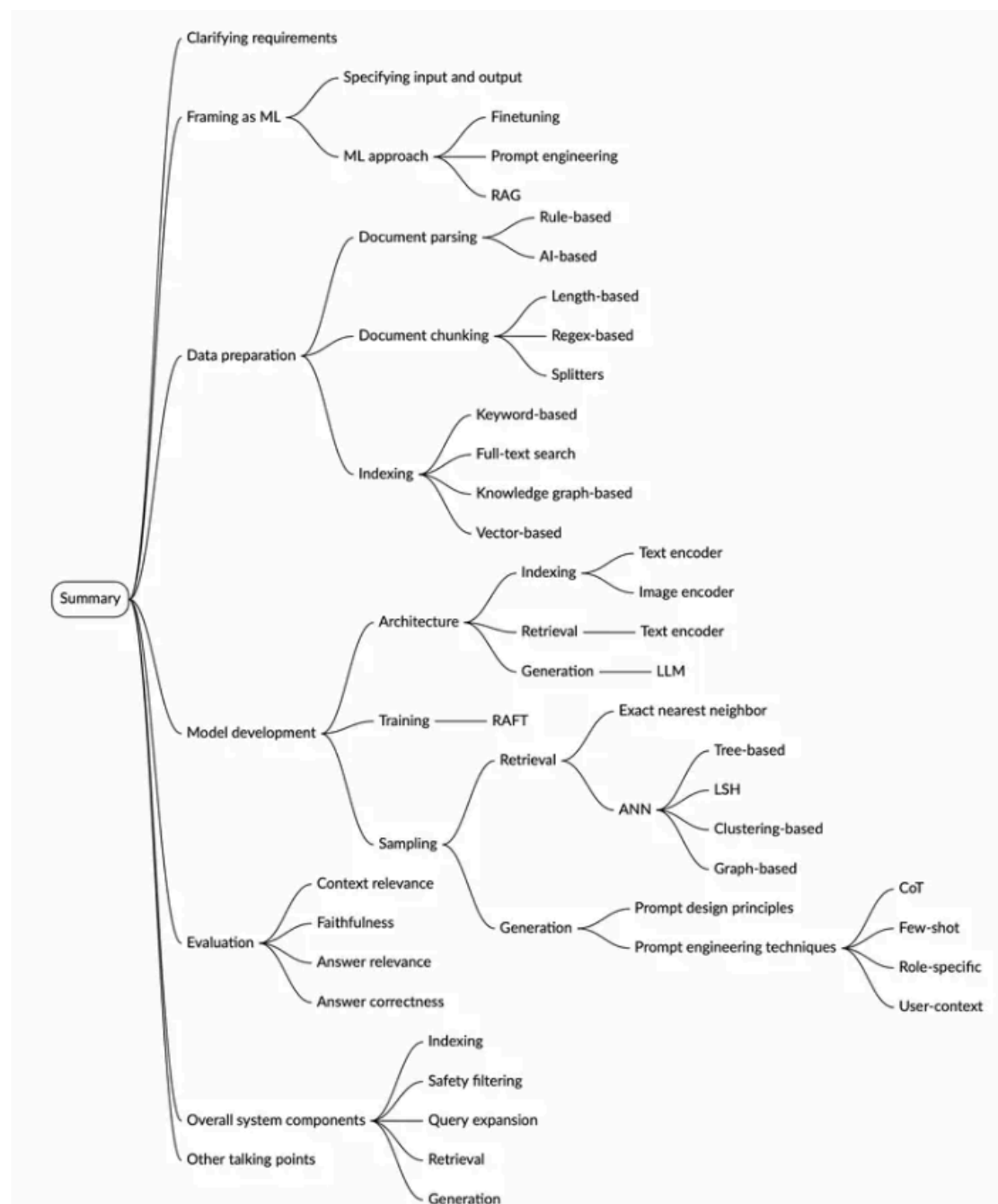


Figure 43: Smart Compose feature overall design

1. Monitoring & Triggering: The service triggers the phrase generator once it identifies specific patterns such as specific keywords or number of characters typed

2. Calls Model: The phrase generator employs beam search to get top-k potential completions from the trained model.
3. Filtering: The phrase generator interacts with the filtering component to remove long suggestions and those with low confidence scores.
4. Post-processing: The completion with the highest score is picked and passed to the post-processing service. The service replaces gender-specific pronouns and adjusts sensitive terms.
5. Display suggestion: The suggestion is displayed to the user for their consideration.

[Design] RAG System



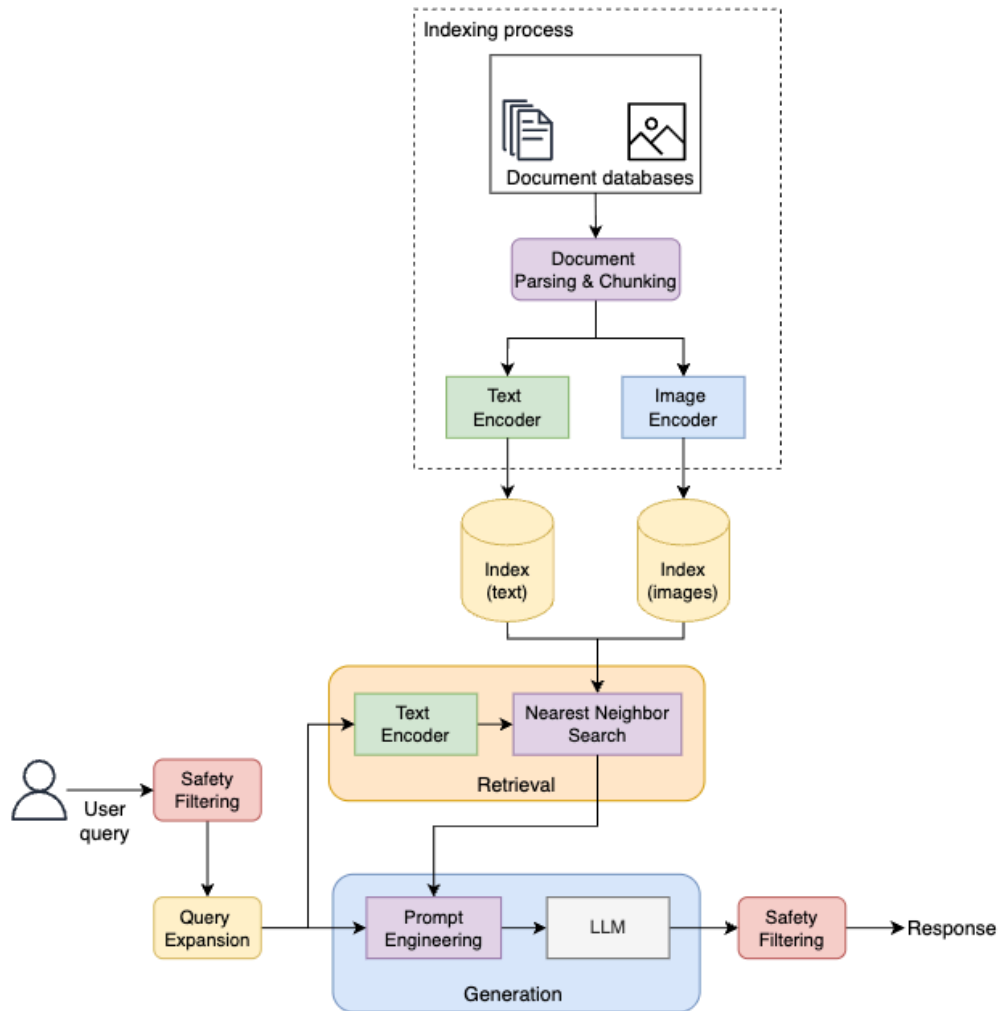


Figure 27: RAG system overall design

1. Clarifying questions:

- What does the external knowledge base consist of? Does it change over time?
- Text, images, or other modalities? PDF or some other format?
- How many pages in total?
- Do we expect any growth in pages?
- Do we need to cite document references?
- How much latency is acceptable?
- Should it be Multi-lingual to start?
- Any user feedback or follow-up questions?
- I assume we need to address safety concerns such as preventing harm, bias, and NSFW responses?

2. Frame as ML:

- To accommodate external data sources, there are 3 types of approaches (with a pre-trained LLM):
 - Finetuning → updating the weights of the pre-trained LLM
 - Pro: enhanced accuracy, handling of niche topics, domain-specific
 - Con: expensive to train, requires frequent retraining whenever there is new information, no referencing
 - Prompt engineering → better prompts to adjust the outputs
 - Pro: easy to do, cost effective, flexible

- 2. Con: Limited customization, Limited to LLM's existing knowledge
- iii. RAG → combines real-time retrieval system with a LLM generation component
 - 1. Pro: access to up-to-date information (retrieval now a separate system), contextual relevance
 - 2. Con: implementation complexity, retrieval quality now matters
- b. RAG offers a balanced solution between prompt engineering and finetuning
- 3. Data Prep
 - a. Document Parsing (e.g. PDF parser)
 - i. Layout detection using object detection → segmented regions
 - ii. Within each region run OCR if it's detected to be text, or run some image encoder into an embedding
 - iii. Structured output
 - iv. Usually handled by some external service
 - b. Document Chunking
 - i. Too long of a doc loses semantic relevance easily, and hard to pick up on key details, so want to break them up into smaller chunks. Need to think about chunk size and overlap, smaller chunk size results in more fragmented knowledge making it harder to return enough context to the final LLM but returned context may be more relevant, while too large of chunk size end up losing key details again.
 - ii. Can do length-based, regular expression-based, or markdown/html
 - iii. LangChain typically handles this
 - c. Indexing
 - i. Organize the chunked data that enables efficient and accurate search
 - ii. Inverted index is a mapping of words to Document IDs in which these words appear. This enables efficient full-text search by mapping each term to a list of documents. It's foundational in search engines. Updates require carefully handling index maintenance for insertions and deletions.
 - iii. Semantic Vector Based Index is a mapping of ID to high dimensional vectors. It typically allows for fast similarity search using HNSW, IVF, PQ using ANN. These trade exact recall for speed. At query time we return top K highest cosine similarity vectors.
 - iv. Suppose each page contains roughly 1,500 characters and includes three images. Using length-based chunking with a chunk size of 500 characters and a 200-character overlap, each page will generate 5 text chunks and 3 image chunks. Therefore, the total number of chunks the RAG system is dealing with is $5M(1500 / (500-200) + 3) = 40M$. This figure is expected to grow by roughly 20 percent each year ...
- 4. Modeling
 - a. Inside the RAG you would use models in 3 places. Indexing to have text and image encoders transform chunked text images into embeddings, in retrieval to transform the user query into embeddings, and the LLM in Generation.
 - b. For text encoding, you might want to consider the encoder only Transformer, or some LSTM/GRU RNN network, though the former have been shown to be far more effective.
 - c. For image encoding, you might want to consider an encoder only transformer or CNN. CNN are great at capturing spatial hierarchies through convolution filters.

Transformers would require turning the image into a sequence by patching, then sent into the multihead attention with positional encodings.

- d. You will want both decoders to map to the same space. You can use encoders that already do that like CLIP, or you can do image captioning. In the latter you would have an image encoder and a text decoder.
 - e. Training: Typically these models are already pretrained, but since there are multiple components, you'll likely have to tune the parameters associated with each step of the process. We can look at the top ones.
 - i. Finetuning via RAFT. RAFT is to train a model based on (Question, Retrieved Docs, Answer) golden sets, where the Retrieved Docs are labeled as relevant or irrelevant. This teaches the LLM to discriminate between relevant and irrelevant sources, and force it to extract and cite evidence.
 - ii. When finetuning, you can do full on model updates which can require extremely large amounts of domain-specific data, or do something like Low Rank Adaptation (LORA), where you train a delta weight matrix that is a projection from two much smaller matrices. $\Delta W = B \cdot A$, and you would update W with ΔW .
 - f. Sampling: ANN can be improved by clustering the embeddings together and then doing both inter-cluster search and intra-cluster search, or HNSW to speed up computation.
 - g. Generation: Here is where we would need effective prompt engineering
 - i. Lots of depth here. Some reasonably effective practices I've seen.
 - ii. Start simple and iterate from there. Break down complex tasks. Use specific and clear instructions. Give example outputs with the degree of specificity you want (e.g. few-shot prompting). Role-specific prompting to get the output language in the style and level you want.
5. Eval - every part of the system, as well as the end-to-end response
- a. Offline Eval
 - i. Retrieval evaluation → Precision@K, Recall@K, Hit Rate, Mean Reciprocal Rank (MRR), NDCG
 - ii. How well the LLM summarizes the documents → faithfulness or groundedness, how much hallucination, reliability and consistency (across multiple runs)
 - 1. Human Eval
 - 2. LLM-Eval with Human in the Loop to audit and improve the Eval instructions
 - 3. Fact-checking against some knowledge base of curated
 - iii. Search Relevance techniques on the final output and input query
 - 1. NDCG based on Human evals of relevance, correctness, completeness, conciseness, understandability
 - 2. F1 score which measures the overlap of the response and the reference answer at the token level
 - 3. Correctness measures from Machine Translation since it's sequence of tokens in and sequence of tokens out (BLEU, ROUGE)
 - b. Online Eval
 - i. User feedback (thumbs up, thumbs down)

- ii. Rephrasing the query and attempting it again (can differentiate this by time passed since last query or semantic relevance of consecutive queries)

6. ML System

- a. Query expansion to increase the chances of having better retrieval
- b. Safety filtering of the input and of the potential output

7. Monitoring

- a. Failures
 - i. Feature outages (feature distribution drift, NULL values → may lead to NULL predictions)
 - ii. Stale model predictions (no retraining, data drift)
 - iii. System failures (latency, throughput, versioning)
 - iv. Traffic spikes (GPU / CPU utilization)
 - v. Code path failures (what's AB tested wasn't what was launched)
 - vi. Feedback loops

[Design] Image Captioning

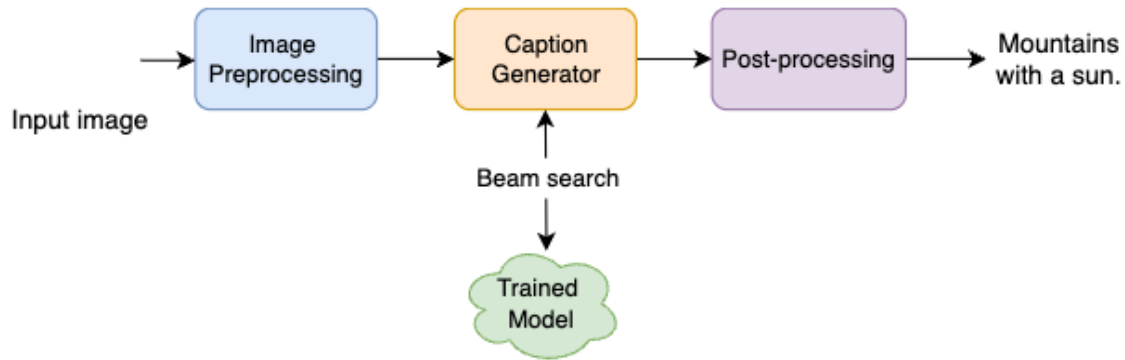
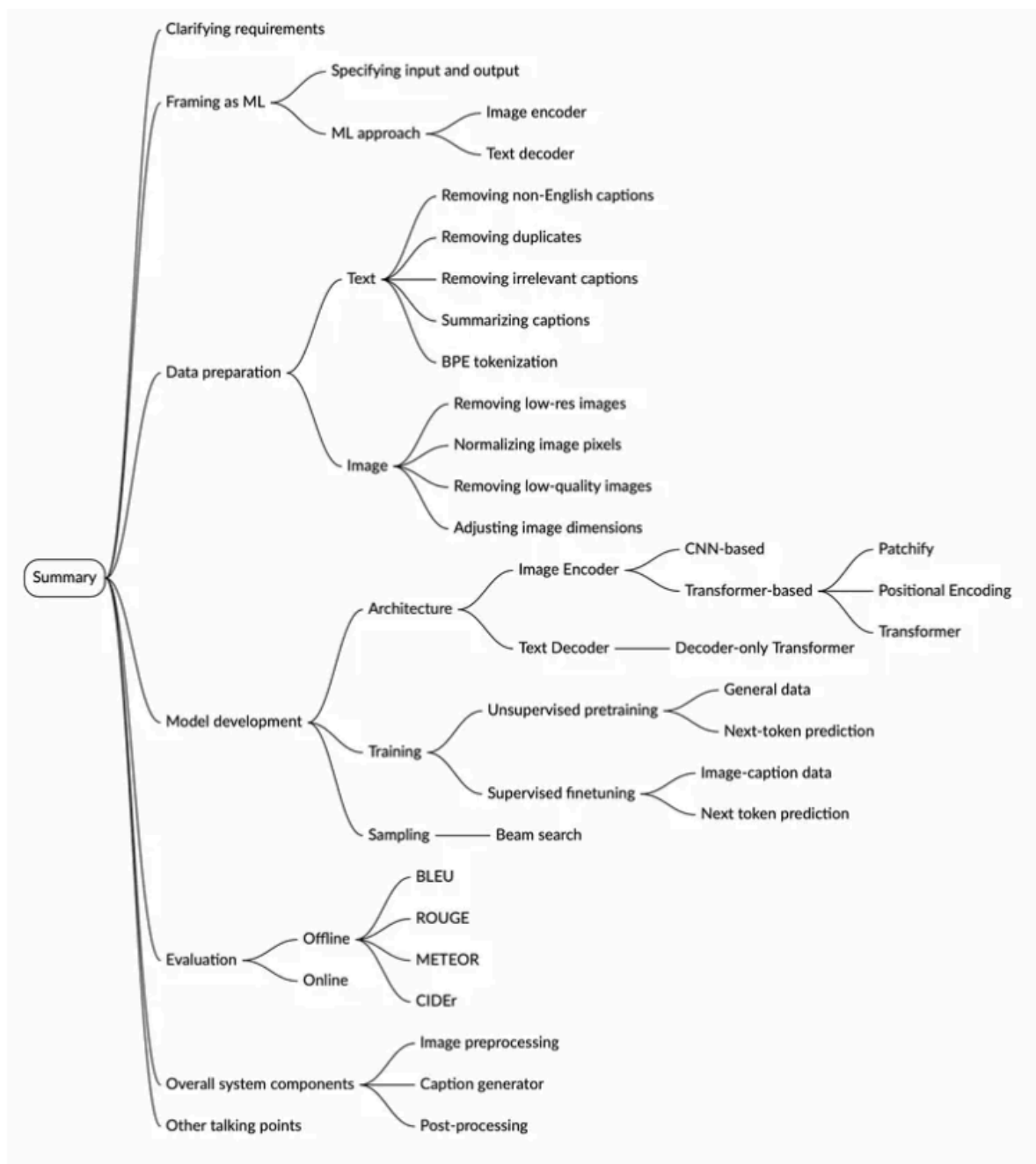


Figure 17: Image captioning overall design



1. Clarify Requirements:

- **Primary Goal:** Generate short, descriptive, and clear text captions (name suggestions) for general everyday images, specifically for designers uploading assets.
- **Input/Output:** Image → Text Caption.
- **Image Type:** General everyday images (not domain-specific like medical). Min resolution 256x256 pixels.
- **Caption Length/Detail:** Short, descriptive, clear. Not long/detailed.
- **Language:** English only.
- **Dataset:** Large (400M image-caption pairs).
 - **Data Quality:** Raw dataset may contain non-English, duplicates, irrelevant, missing captions.
- **Latency:** Quick generation (1-2 seconds acceptable). Not strict real-time.
- **Ambiguous Content:** System should **skip** suggesting a caption if image content is unclear/ambiguous (confidence threshold).
- **Safety/Fairness:** Crucial to **avoid biased or offensive captions**.

2. Frame as ML Task:

- **ML Objective:** Generate text captions that accurately describe image content, suitable for asset naming.
- **System Input:** Image.
- **System Output:** Text caption.
- **ML Category: Multi-modal Language Generation** (Image-to-Text).
 - **Approach: Encoder-Decoder Framework.**
 - **Image Encoder:** Understands visual content, encodes image into a lower-dimensional representation.
 - **Text Decoder:** Uses encoded visual info to generate the descriptive caption.
 - *(Note: Acknowledge alternatives like BLIP-2/3, InternVL, but focus on Encoder-Decoder for this design).*

3. Data Preparation:

- **Data Choice:** 400 million image-caption pairs.
- **Caption Preparation:**
 - **Remove Non-English Captions:** Filter out non-English captions to align with English-only requirements.
 - **Remove Duplicates:**
 1. **Images:** Use perceptual hashing or image similarity models (e.g., CLIP image encoder).
 2. **Captions:** Exact match or semantic similarity (e.g., CLIP text encoder). Prevents overfitting, improves diversity.
 - **Remove Irrelevant Captions:** Use a pre-trained **vision-language model** (e.g., CLIP) to assess relevance (image-text similarity score). Remove pairs below a threshold (e.g., 0.25). Ensures high-quality relevant data.
 - **Summarize Long Captions:** Use an LLM (e.g., Llama) to summarize excessively long captions into concise descriptions. Ensures generated captions are short as required.
 - **Normalize Captions:** Lowercasing, trim whitespaces for consistency.

- **Tokenize Captions: Subword-level tokenization (e.g., BPE)** for vocabulary efficiency and OOV handling. Convert tokens to IDs.
- **Image Preparation:**
 - **Remove Low-Resolution Images:** Filter out images < 256x256 pixels as per requirements.
 - **Normalize Images:** Scale pixel values (e.g., 0-1) for stable training.
 - **Remove Low-Quality Images:** Filter blurry, over/underexposed images using image quality assessment methods (e.g., LAION Aesthetics Predictor).
 - **Adjust Dimensions (Resize & Center-Crop):**
 1. **Resize:** Scale image so *smaller* dimension matches target size (e.g., 256).
 2. **Center-Crop:** Crop the resized image to the final fixed square target size (e.g., 256x256), preserving aspect ratio.

4. Modeling:

- **Architecture:** Encoder-Decoder framework.
- **Image Encoder:**
 - **Purpose:** Process image, encode visual information.
 - **Output Type: Sequence of tokens (patches)** preferred over single token.
 - **Rationale:** Captures granular local details and global features. Aligns well with attention mechanism of text decoder, enabling selective focus on image regions for detailed captions.
 - *(Contrast with single token: Simpler, less compute, but loses local detail, leads to generic captions).*
 - **Architecture Choice: Transformer-based (e.g., ViT).**
 - **Why:** Excels at capturing both local and **long-range global relationships** in images using self-attention. Crucial for descriptive, context-aware captions. (CNNs struggle with long-range dependencies).
 - **Components:**
 - **Patchify:** Divide image into fixed-size patches → flatten → linear projection into embeddings.
 - **Positional Encoding:** Assigns position info to each patch.
 - **Choice:** 2D (preserves spatial structure) or 1D. Learnable vs. Fixed (e.g., sine-cosine). *(ViT uses learnable 1D, but experimentation is key).*
 - **Transformer Blocks:** Apply self-attention and feed-forward networks.
- **Text Decoder:**
 - **Purpose:** Generate the caption one token at a time.
 - **Architecture Choice: Decoder-only Transformer** (standard for text generation).
 - **Input:** Sequence of embeddings (from Image Encoder).
- **Training Strategy (Two-Stage):**
 - **Unsupervised Pre-training:**
 - **Text Decoder:** Pre-train on general text data (e.g., using next-token prediction with Cross-Entropy loss).
 - **Leverage:** Use existing pre-trained decoder-only Transformers (e.g., GPT-2, Llama) to save compute.

- **Image Encoder:** Leverage existing pre-trained vision models (e.g., CLIP, ViT).
 - **Supervised Fine-tuning:**
 - Train both Image Encoder and Text Decoder jointly on the 400M image-caption pairs.
 - **ML Objective:** Next-token prediction.
 - **Loss Function: Cross-Entropy Loss.**
 - **Purpose:** Guides the text decoder to accurately predict the next token in the ground truth caption sequence, given the image encoding and previously generated tokens.
- **Sampling:**
 - **Method: Beam Search**
 - **Rationale:** Prioritizes **quality, consistency, and coherence** for asset name suggestions.
 - **Quality:** Higher-quality captions.
 - **Consistency:** Deterministic output for same image. Crucial for asset management where consistent naming is preferred.
 - **Coherence:** Avoids nonsensical or ungrammatical phrases.
 - *(Avoid stochastic methods like Top-K/Top-P, as they prioritize creativity/diversity over the deterministic coherence needed for asset naming).*

5. Evaluation:

- **Offline Metrics:**
 - **Validation Data:** Image + multiple human-annotated reference captions (for robust training & comprehensive evaluation).
 - **Primary Metric: CIDEr (Consensus-based Image Description Evaluation).**
 - **Why:** Specifically designed for image captioning. Emphasizes consensus with multiple references and weights important words (via TF-IDF). Robust to variations.
 - **Process:**
 1. Represent generated and reference captions using TF-IDF.
 2. Calculate **Cosine Similarity** between generated and each reference caption's TF-IDF vector.
 3. Aggregate (average) similarity scores.
 - **Pros:** Consensus-based (reliable), sensitive to important words, robust to variations.
 - **Cons:** Computationally complex, sensitive to reference quality, can penalize novel but accurate captions, relies on TF-IDF (might lack full semantic understanding).
 - **Secondary Metrics:** BLEU, ROUGE, METEOR (for broader perspective on fluency, recall, precision).
- **Online Metrics:**
 - **Primary Focus:** Not primary evaluation method due to difficulty in collecting direct user feedback/interaction data for backend systems.
 - **If Applicable (e.g., if integrated into UI):** Engagement metrics (e.g., adoption rate of suggested names), user feedback mechanisms for qualitative assessment.

6. Overall ML System Design (Serving):

- **Components:**
 1. **Image Preprocessing:**
 - Takes raw input image.
 - Resizes, normalizes pixels, adjusts dimensions (resize & center-crop). Ensures image is consistent with model input expectations.
 2. **Caption Generator (Core):**
 - Receives preprocessed image.
 - Passes image through trained **Image Encoder** to get sequence of embeddings.
 - Feeds embeddings to **Text Decoder**.
 - Employs **Beam Search** to generate the caption token by token.
 - **Confidence Thresholding:** If cumulative probability of generated caption falls below a predefined threshold, suggestion is *skipped* (to handle ambiguous images).
 3. **Post-processing:**
 - Receives generated caption.
 - **Bias/Safety Check:** Identifies and replaces biased terms/phrases with neutral alternatives. Checks for offensive words and **disables suggestion** if found. (*Strategic: Direct alignment with Anthropic's safety imperative and your strength in "power-with-softness" and "high integrity"*).

7. Monitoring:

- **System Health:** Latency (critical for 1-2s target), throughput, error rates of preprocessing, generator, and post-processing components.
- **Model Performance (Offline):** Continuous CIDEr (and other metrics) monitoring on validation/test sets to track model quality and detect degradation.
- **Safety/Bias Monitoring:** Log captions flagged/modified by post-processing. Develop internal metrics/audits for bias detection in generated captions.
- **Ambiguity Handling:** Monitor the rate of "skipped suggestions" due to low confidence.
- **Data Drift:** Monitor characteristics of incoming images (e.g., resolution, content themes) to ensure they match training distribution.

Other Potential Discussion Points (for senior roles / if time permits):

- **Multi-Lingual Captioning:** Extend to other languages.
- **Domain Adaptation:** Adapting the model for more specific image types beyond everyday images.
- **Edge Device Optimization:** For low-latency or offline use cases.
- **Multiple Captions:** Generating and ranking several plausible captions.
- **Visual Question Answering (VQA):** How the system's components could form a foundation for other V+L tasks.

[Design] Text to Image

Okay, James, let's convert this comprehensive "Text-to-Image Generation" system design into your preferred "Notes format." This is an incredibly relevant topic, especially for a Pinterest-like platform and a company like Anthropic, given its focus on generative AI and safety. I'll maintain your style, highlight key technical choices, and emphasize the safety and fairness aspects you raised.

1. Clarify Requirements:

- **Primary Goal:** Generate high-resolution, detailed images from text prompts.
- **Input/Output:** Text prompt → 1024x1024 pixel image.
- **Text Prompt:** Max 128 words, handle detailed and complex prompts.
- **Language:** English only (initially), but adaptable architecture for multi-language future.
- **Dataset:** ≈ 500 million image-caption pairs (from user assets, mostly captioned). Some captions noisy/non-English.
- **Generation Speed:** Near real-time, aim for ≤ 10 seconds per image.
- **Image Diversity:** Generate wide range of images (realistic, portraits, abstract, conceptual).
- **Safety/Fairness: Crucial** to ensure fairness (avoid bias by age, race, gender) and prevent generation of offensive, inappropriate, or harmful images (filters/checks needed).

2. Frame as ML Task:

- **ML Objective:** Generate visually detailed images that accurately adhere to the text prompt.
- **System Input:** Text prompt.
- **System Output:** Generated image (1024x1024 pixels).
- **ML Category: Multi-modal Generative AI** (Text-to-Image).
 - **Approach Chosen: Diffusion Models.**
 - **Rationale:** Prioritizes **exceptional image quality** and realism, offers flexibility in trading off sampling speed vs. quality. (Acknowledges Autoregressive models as alternative but highlights their complexity/statistical inefficiency for this task).
 - **Mechanism:** Iterative refinement process: starts with random noise and gradually transforms it into a clear image, guided by a text encoder.

3. Data Preparation (Data Pipeline):

- **Data Source:** 500 million image-caption pairs.
- **Image Preparation:**
 1. **Filtering Inappropriate Images:**
 - Remove small images (< 64x64 pixels - often poor quality).
 - **Deduplicate images:** Remove identical or perceptually similar images (e.g., perceptual hashing). Prevents bias from redundant data.
 - **Remove Inappropriate Images:** Employ **harm detection / NSFW detection models** to filter out violence, nudity, etc. **CRITICAL for safety requirement.**

- **Remove Low-Aesthetic Images:** Use specialized ML models to filter images that aren't aesthetically pleasing.
 - 2. **Standardizing Images:**
 - **Adjust Image Dimensions:** Resize (smaller dim matches target, e.g., 128) then **center-crop** to fixed input size (e.g., 128x128).
 - **Normalize Images:** Scale pixel values (e.g., [0,1] or [-1,1]).
- **Caption Preparation:**
 1. **Handle Missing/Non-English Captions:** Use an **image captioning model** (e.g., BLIP-3) to auto-generate descriptive English captions.
 2. **Enhance Captions:** Use pre-trained CLIP to score image-caption relevance. If score below threshold, replace original with auto-generated caption (from BLIP-3).
 3. **Remove Poorly Matched Pairs:** After enhancement, remove pairs with low CLIP similarity scores. Ensures high-quality text-image alignment for training.
 4. **Precompute Caption Embeddings:** Use a **pre-trained text encoder** (e.g., T5) to pre-compute and cache caption embeddings. Reduces training computation (conditioning input).
- **Continuous Data Flow:** Collect and process newly generated data (user prompts, generated images, user feedback) to add to training set for continuous improvement.

4. Modeling (Training Pipeline):

- **Architecture (for Noise Prediction): U-Net** (chosen for educational purposes in context, but note DiT is also strong and often preferred in recent state-of-the-art like Sora).
 1. **U-Net Components:**
 - **Downsampling Blocks:** Convolution, Batch Norm, Activation, Max-pooling.
 - **Cross-Attention:** Crucial layer here. Queries from image features, Keys/Values from text embeddings (from text encoder). Integrates text prompt influence.
 - **Upsampling Blocks:** Transposed Convolution, Batch Norm, Activation.
 - **Cross-Attention:** Continues text influence during upsampling.
 - **Skip Connections:** (Implicit in U-Net) Pass high-res features from downsampling to upsampling path for detail preservation.
- **Diffusion Training Process:**
 1. **ML Objective:** Minimize the difference between true noise (ϵ) and predicted noise (ϵ_θ).
 2. **Loss Function: Mean Squared Error (MSE) / L2 Loss.**
 - **Formula:** $L = E_{t, x_0, \epsilon} (|| \epsilon - \epsilon_\theta(x_t, t) ||^2)$
 - **Purpose:** Guides model to accurately predict noise at each step of the reverse diffusion process.
 3. **Steps:**
 - **Noise Addition:** Randomly sample timestep t , apply noise directly to original image x_0 to get noisy image x_t using noise schedule ($x_t = \alpha_t x_0 + (1 - \alpha_t) \epsilon$).



 $x_0 + 1 - \alpha_t$



 ϵ

- **Conditioning Signals:** Prepare image caption embedding and sampled timestep t for model input.
 - **Noise Prediction:** U-Net/DiT model predicts noise ϵ_θ given x_t , t , and caption embedding.
 - **Loss Calculation:** MSE between predicted noise and actual noise.
- **Challenges & Strategies for Training:**
 1. **Resource-Intensive Training:** (Mitigation strategies due to billions of parameters, high-res images, large data)
 - **Mixed Precision Training:** Use 16-bit and 32-bit floats to reduce memory/increase efficiency.
 - **Model and Data Parallelism:** Distribute training across 6000+ H100 GPUs using frameworks like FSDP/Deepspeed. **CRITICAL for scale.**
 - **Latent Diffusion Models (LDM):** Operate in lower-dimensional latent space (already chosen as primary approach).
 2. **Slow Image Generation (during inference, but design impacts training):**
 - **Parallel Sampling:** Parallelize generation across multiple devices.
 - **Model Distillation:** Train a smaller "student" model to mimic a larger "teacher" model for faster inference.
 - **Model Quantization:** Reduce precision of weights (e.g., 8-bit) for smaller size, faster generation.
 - **Optimized Algorithms:** Use faster sampling algorithms (e.g., DDIM) to reduce required diffusion steps.

5. Evaluation (Evaluation Pipeline):

- **Offline Metrics:**
 - **Benchmark:** DrawBench (curated prompts for object composition, interaction, context).
 - **Key Areas:**
 - **Image Quality:**
 - **Metrics:** Inception Score (IS), FID (Fréchet Inception Distance). (Common for assessing realism/diversity).
 - **Image Diversity:** (Measured by FID, also qualitatively).
 - **Image-Text Alignment:**
 - **Metric:** CLIPScore.
 - **Process:** Cosine similarity between CLIP embeddings of generated image and text prompt.
 - **Purpose:** Quantifies how well generated image matches the text description. Higher score = better alignment.
 - **Human Evaluation:** Complement automated metrics.
 - **Image Quality:** Raters compare generated vs. reference image for photorealism.
 - **Text Alignment:** Raters score "Does caption accurately describe image?" (Yes/Somewhat/No).
- **Online Metrics:** (Measure real-world performance & user satisfaction)
 - **Click-Through Rate (CTR):** Clicks on generated images.
 - **Time Spent on Page:** User engagement.
 - **User Feedback:** Direct feedback on quality/alignment.

- **Conversion Rate:** Desired user action after interaction.
- **Latency:** Actual generation time in production ($\leq 10s$ goal).
- **Throughput:** Images generated per second.
- **Resource Utilization:** CPU, GPU, memory usage for cost optimization.
- **Average Cost per User per Month:** Track cost-effectiveness, especially with repeated generations from unhappy users.

6. Overall ML System Design (Inference Pipeline):

- **Components:**
 1. **Prompt Auto-Complete Service:** Suggests phrases as user types, improves UX.
 2. **Prompt Safety Service:**
 - **CRITICAL for Safety.**
 - Uses a **text classification model** to process user prompts.
 - **Rejects prompts** violating usage policies (violence, hate, nudity, etc.).
 3. **Prompt Enhancement:**
 - Refines user prompts for clarity, coherence, detail.
 - Helps diffusion model produce better outputs.
 4. **Image Generation (Core):**
 - Encodes enhanced text prompt (e.g., with T5 text encoder).
 - Passes tokens to the **Diffusion Model** to generate one or multiple images.
 5. **Harm Detection:**
 - **CRITICAL for Safety (Defense-in-Depth).**
 - Applies **image classification models** to generated images.
 - Flags/blocks inappropriate images (violence, nudity, bias) *before* display, even if previous prompt safety checks were passed.
 6. **Super-Resolution Service:**
 - Increases resolution of generated images (e.g., from 64x64 base to 1024x1024).
 - Often a cascade of multiple SR models.

7. Monitoring:

- **System Health:** Latency, throughput, resource utilization for all services.
- **Model Performance (Offline):** Continuous monitoring of CLIPScore, FID, IS on test sets.
- **Safety/Bias Monitoring:**
 - **Quantify Bias:** Track generations related to age, race, gender (e.g., if a prompt for 'engineer' consistently generates images of one race/gender).
 - Log prompts rejected by Prompt Safety Service.
 - Log images flagged/blocked by Harm Detection.
 - Regular **red-teaming** for new bypasses or unintended harmful generations.
- **User Feedback:** Analyze explicit feedback on safety and quality.
- **Cost Management:** Monitor average cost per user/generation.

- **Prompt Distribution Analysis:** Track common prompts, emerging trends, and any problematic prompt patterns.

Other Potential Discussion Points (for senior roles / if time permits):

- **Consistency Models:** For even faster generation.
- **RLHF for Quality/Alignment:** Applying human feedback (or AI feedback) to refine generated image quality and alignment beyond simple loss functions. *Strategic: Directly links to Anthropic's core alignment work.*
- **Inpainting/Outpainting:** Extending functionality.
- **Personalization:** Fine-tuning models to specific user styles/concepts.
- **Advanced Controls:** ControlNet for precise control over generated images.
- **Ethical AI:** Deep dive into the ethical challenges of generative AI (deepfakes, misinformation, copyright, representational harm). *Strategic: Your strengths in high integrity and analytical thinking.*

[Design] Text to Video

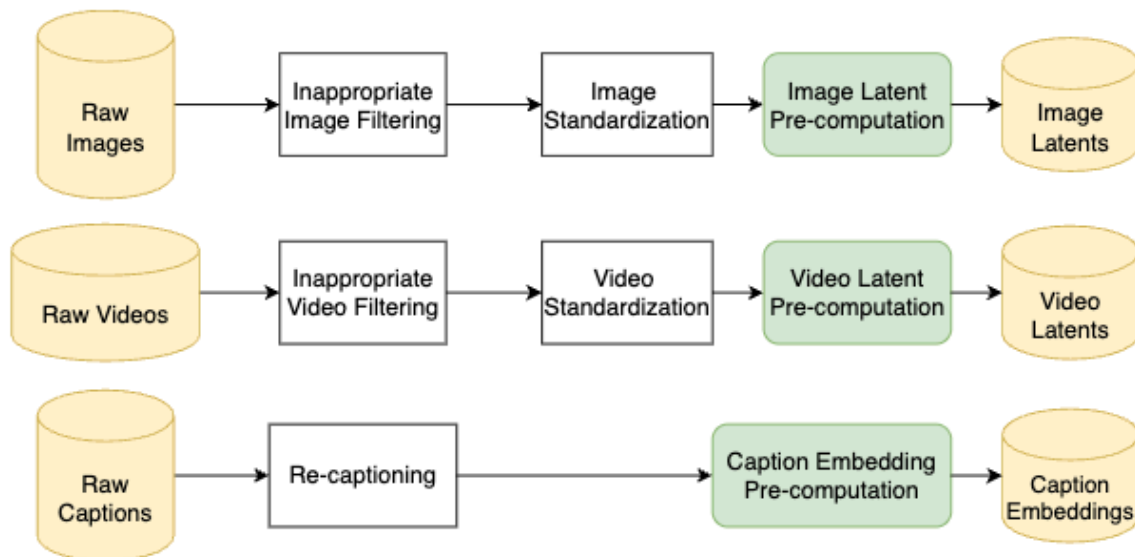


Figure 23: Data pipeline

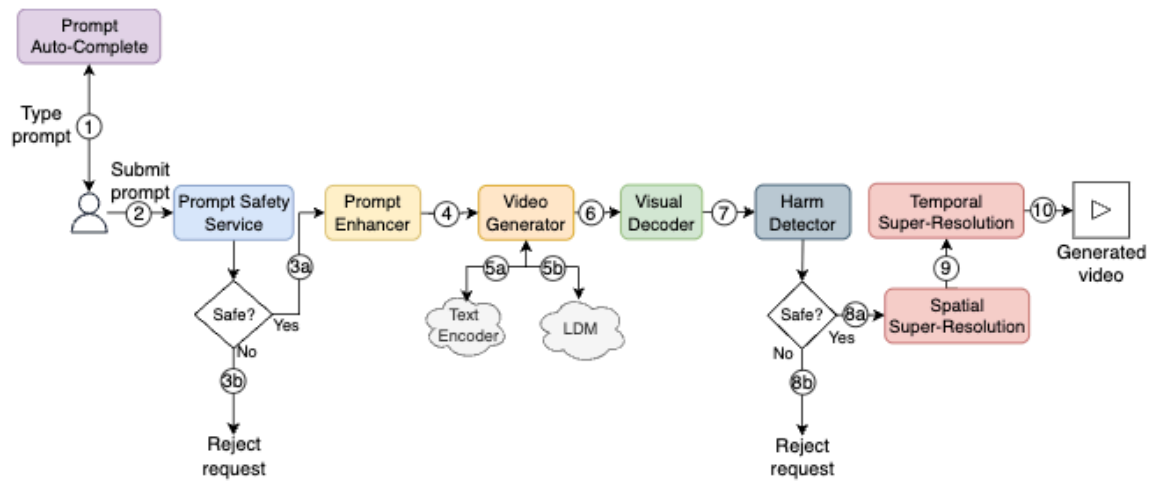
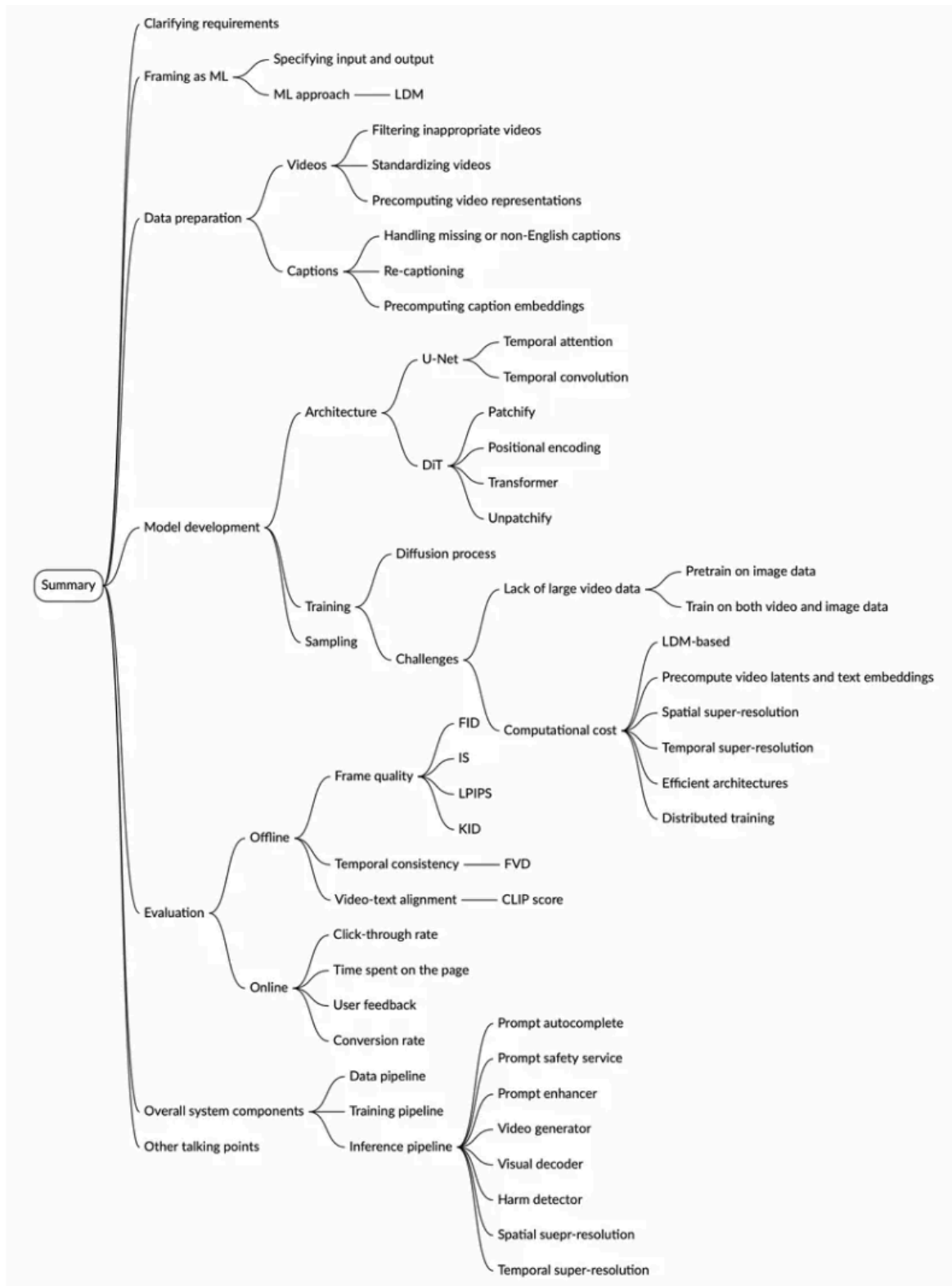


Figure 24: Inference pipeline components



1. Clarify Requirements:

- **Primary Goal:** Generate 5-second, 720p (1280x720 pixels), 24 FPS videos from text descriptions.
- **Input/Output:** Text prompt → Video.
- **Latency (Generation):** A few minutes initially (acceptable), with future optimization.
- **Content Category:** General, diverse genres/subjects.
- **Language:** English text input only (initially).
- **Audio Output:** Silent videos (initially).

- **Training Data:** 100 million diverse video-caption pairs (some noisy/non-English).
- **Pre-existing Assets:** Access to a pre-trained **text-to-image model** (assumed for extension).
- **Compute Budget:** Over 6000 H100 GPUs (significant resources available).
- **Safety/Fairness:** Crucial to prevent generating offensive or harmful videos.

2. Frame as ML Task:

- **ML Objective:** Generate videos that visually and temporally align with a given text prompt.
- **System Input:** Descriptive text prompt.
- **System Output:** 5-second, 720p, 24 FPS video.
- **ML Category: Multi-modal Generative AI (Text-to-Video).**
 - **Approach: Latent Diffusion Model (LDM).**
 - **Rationale:** Extends successful text-to-image diffusion to video. Crucially, operates in a **lower-dimensional latent space** (instead of directly in pixel space) to reduce computational complexity and memory (e.g., 512x efficiency gain from 8x spatial and 8x temporal compression).
 - **Core LDM Components:**
 1. **Compression Network (VAE):**
 - Visual Encoder: Maps raw video pixels → compressed latent representation (reduces frame count + resolution).
 - Visual Decoder: Reconstructs original video frames from latent space.
 2. **Diffusion Model (within LDM):** Learns to denoise these lower-dimensional latent representations.

3. Data Preparation (Data Pipeline):

- **Data Source:** 100 million diverse video-caption pairs.
- **Video Preparation:**
 1. **Filter Inappropriate Videos:**
 - Remove low-quality/short/slow-motion/distorted videos (e.g., as per Movie Gen).
 - **Deduplication:** Eliminate identical videos (e.g., perceptual hashing) for diverse training.
 - **Harmful Videos:** Use **harm-detection models** to identify and remove explicit/inappropriate content. **CRITICAL for safety requirement.**
 2. **Standardize Videos:**
 - Adjust video length: Split longer videos into 5-second clips.
 - Standardize frame rate: Re-encode to 24 FPS.
 - Adjust video dimensions: Resize/crop to 1280x720 pixels.
 3. **Precompute Latent Representations:**
 - Pass all standardized videos through the **pre-trained VAE's Visual Encoder** to obtain latent representations.

- **Cache/Store these latent representations:** Avoids on-the-fly computation during training, significantly speeding up diffusion training. (Calculation: ~2MB/video, 200TB for 100M videos - manageable).
- **Caption Preparation:**
 1. **Handle Missing/Non-English Captions:** Use video captioning models (e.g., LLaMa3-Video, LLaVA) to automatically generate descriptive captions.
 2. **Re-captioning:** Improve existing captions (even if present) using pre-trained video captioning models to generate longer, more detailed versions. Improves quality and text alignment (Sora team finding).
 3. **Precompute Caption Embeddings:** Use a **text encoder** (e.g., from a pre-trained text-to-image model) to precompute and store embeddings for all captions. Speeds up LDM training (conditioning input).

4. Modeling (Training Pipeline):

- **Architecture Choice: DiT (Diffusion Transformer) for videos.**
 1. **Rationale:** Based on Sora's success, higher scalability with data/compute due to Transformer nature, flexible for multi-modal adaptation. (Acknowledging U-Net as a viable alternative but choosing DiT).
 2. **DiT Components for Video:**
 - **Patchify:** Divides video into 3D fixed-size patches → flatten → linear project to patch embeddings. (Similar to image patchify but 3D).
 - **Positional Encoding:** Assigns spatio-temporal position info to each patch. (e.g., 3D coordinates, or RoPE from OpenSora). Can be learnable or fixed.
 - **Transformer:** Processes sequence of patch embeddings + conditioning signals (text prompt, timestep) to predict noise for each patch.
 - **Unpatchify:** Converts predicted noise vectors back to original video dimensions.
- **Training Process (Diffusion Model):**
 1. **ML Objective:** Minimize reconstruction loss (accurate noise prediction).
 2. **Loss Function: Mean Squared Error (MSE) / L2 Loss.**
 - **Formula:** $L_{\text{MSE}} = ||\text{predicted_noise} - \text{actual_noise}||^2$.
 - **Purpose:** Guides the DiT model to accurately predict the noise added to the latent representation, enabling high-quality video reconstruction.
 3. **Steps per Iteration:**
 - **Noise Addition:** Randomly sample timestep, add noise to precomputed latent video representation.
 - **Noise Prediction:** DiT model predicts noise from noisy latent video + text embedding + timestep.
 - **Loss Calculation:** MSE between predicted and actual noise.
- **Challenges & Strategies for Training (Addressing Scale/Data Scarcity):**
 1. **Lack of Large-Scale Video-Text Data:**
 - **Strategy 1 (Chosen):** Train DiT on **both image-text and video-text data** (treat images as single-frame videos) in a single stage. Leverages hundreds of millions of image-text pairs.
 - *(Alternative: Pretrain on image data, then finetune on video data - 2 stages).*
 2. **Computational Cost of High-Resolution Video Generation:**

- **LDM Approach:** Training in lower-dimensional latent space (already chosen).
- **Precompute Video Representations:** Cache latent representations to avoid redundant computations (already chosen).
- **Spatial Super-Resolution Model:** Generate lower-resolution videos with DiT, then use a separate model to upscale to 720p/1080p/4K.
- **Temporal Super-Resolution Model:** Generate at lower FPS (e.g., 12 FPS), then interpolate frames to achieve 24 FPS.
- **Efficient Architectures:** Use optimized attention implementations (e.g., **FlashAttention**) and Mixture of Experts (MoE) to accelerate training.
- **Distributed Training: Tensor parallelism** (or other techniques) to parallelize training across 6000 H100 GPUs. Critical for managing memory and compute for large models and data. *Strategic: Aligns with your "scaling through TLs/EMs" and "operational excellence" strengths.*

5. Evaluation:

- **Offline Metrics (Focus on VBench/Movie Gen Bench for comprehensive testing):**
 - **Frame Quality:** Assess individual frames.
 - **Metrics:** FID (Fréchet Inception Distance), Inception Score (IS). (Average over all frames).
 - **Limitations:** Don't capture temporal consistency.
 - **Temporal Consistency:** How smoothly content transitions between frames.
 - **Metric: FVD (Fréchet Video Distance).**
 - **Process:** Generate videos → extract features (e.g., using pre-trained I3D model) → calculate mean/covariance of features for generated vs. real videos → compute Fréchet distance.
 - **Goal:** Lower FVD indicates more realistic and temporally consistent videos.
 - **Video-Text Alignment:** How accurately generated video reflects text description.
 - **Metric: CLIP Similarity Score.**
 - **Process:** Extract frame-level visual features (CLIP image encoder) and textual features (CLIP text encoder) → compute cosine similarity per frame → aggregate (average/max).
 - **Goal:** High CLIP similarity indicates strong alignment.
 - **Human Evaluation:** Crucial for subjective assessment. Compare pairs of videos from different models, assess quality, consistency, and alignment.
- **Online Metrics:**
 - **Click-Through Rate (CTR):** User clicks on generated videos.
 - **Time Spent on Page:** Engagement with generated video content.
 - **User Feedback:** Likes, shares, explicit feedback on quality/relevance.
 - **Conversion Rate:** (If applicable for e.g., ad generation).

6. Overall ML System Design (Inference Pipeline):

- **Components:**
 1. **Text Prompt Input:** User provides text.

2. **Text Encoder:** Converts text prompt → text embedding (precomputed/cached).
3. **Diffusion Model Inference:**
 - Starts with pure noise in latent space.
 - Iteratively denoises using the trained DiT model (conditioned on text embedding and timestep).
 - Outputs a fully denoised latent video representation.
4. **Visual Decoder:**
 - Uses the VAE's Visual Decoder (from the compression network) to convert the latent video representation back into pixel space.
 - Outputs a lower-resolution, lower-FPS video.
5. **Temporal Super-Resolution (Optional/Future):** Interpolates between generated frames for smoother motion.
6. **Spatial Super-Resolution (Optional/Future):** Upscales video resolution (e.g., from 720p to 1080p/4K).
7. **Post-processing (CRITICAL for Safety):**
 - Applies **harm-detection models/filters** to generated video frames and overall content.
 - Checks for **biased terms/visuals**, offensive content.
 - If harmful, **blocks or modifies** the generated video. *Strategic: Directly addresses the safety requirement and your expertise.*

7. Monitoring:

- **System Health:** Latency (critical to meet 1-2min target, future faster), throughput, GPU utilization, error rates.
- **Model Performance (Offline):** Continuous monitoring of FVD, CLIP similarity, FID/IS.
- **Safety/Bias Monitoring:**
 - Track frequency of harmful/biased generations flagged by post-processing.
 - Set up alerts for new types of harmful content.
 - Regular **red-teaming** exercises to find vulnerabilities in safety guards (as per Anthropic context).
- **User Feedback:** Collect and analyze explicit user feedback on safety and quality of generated videos.
- **Data Drift:** Monitor characteristics of incoming text prompts and potential shifts in generated video content over time.

Other Potential Discussion Points (for senior roles / if time permits):

- **Sampling Flexibility:** Supporting variable durations, resolutions, aspect ratios.
- **Downstream Applications:** Inpainting, video-to-video stylization, animating images.
- **Control Mechanisms:** Desired motion level, camera vs. object motion (e.g., ControlNet for text-to-video).
- **Efficiency:** Progressive distillation, different noise schedulers, noise conditioning augmentation.
- **Personalization:** Fine-tuning model to a particular subject or user.
- **Ethical Implications:** Broader discussion on responsible AI development for generative video.

[Design] Youtube Search (Text → Video)

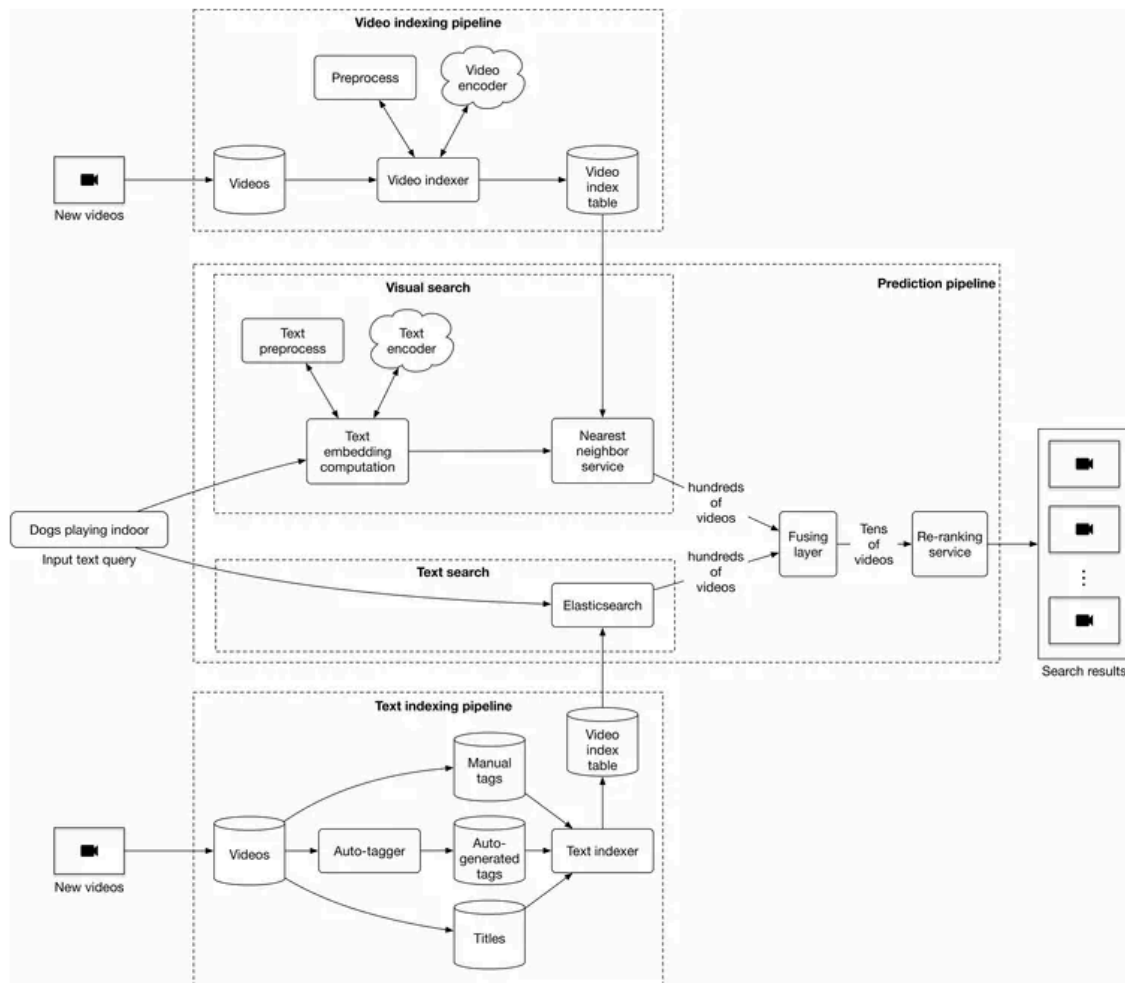


Figure 4.19: ML system design

1. Clarify Requirements:

- **Primary Goal:** Given a text query, retrieve a ranked list of relevant videos.
- **Input/Output:** Text query → Ranked list of relevant videos.
- **Media Type:** Platform serves only videos.
- **Relevance:** Determined by video's **visual content** and **associated textual data** (title, description). No other metadata for simplicity.
- **Training Data:** 10 million pairs of ⟨video, text query⟩ available.
- **Language:** English only (for simplicity).
- **Scale:** 1 billion videos on the platform.
- **Personalization:** None required (opposed to recommendation systems).

2. Frame as ML Task:

- **ML Objective:** Rank videos based on their relevance to the text query.

- **System Input:** Text query.
- **System Output:** Ranked list of videos.
- **ML Category:** **Ranking Problem** solved via **Representation Learning** (for visual search) and **Inverted Index** (for text search).
 - **Representation Learning (Visual Search Component):**
 - Train a **Text Encoder** (Text query $\rightarrow$ Text Embedding) and a **Video Encoder** (Video $\rightarrow$ Video Embedding).
 - Goal: Learn embeddings such that the **dot product** (similarity score) between a text embedding and a video embedding reflects their relevance.
 - Rank videos by similarity scores to the text query embedding.
 - **Inverted Index (Text Search Component):**
 - Non-ML approach (no training cost).
 - Uses an inverted index (e.g., Elasticsearch) to perform efficient full-text search on video titles, descriptions, and tags.
 - Finds videos with keyword overlap with the text query.

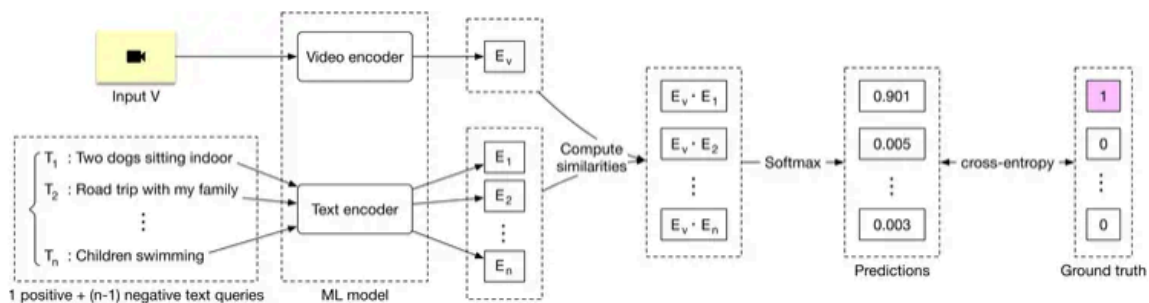


Figure 4.18: Loss computation

3. Data Preparation:

- **Data Choice:** Annotated dataset of 10M $\langle$ video, text query $\rangle$ pairs.
 - Example: Video Name, Query, Split Type (Train/Val/Test).
- **Feature Engineering (Preparing Text Data):**
 - **Text Normalization (Cleanup):**
 - Lowercasing.
 - Punctuation removal.
 - Whitespace trimming.
 - Normalization Form KD (NFKD) / Strip accents.
 - Lemmatization/Stemming (e.g., "walking" $\rightarrow$ "walk").
 - **Tokenization:** Convert text into tokens.
 - **Subword Tokenization (e.g., BPE, SentencePiece):** Preferred.
 - **Pros:** Handles OOV (out-of-vocabulary) words by decomposing them into seen subwords; smaller vocabulary size than word-level; balances expressiveness with manageable vocabulary.
 - **Cons:** More complex to implement than word/char level.
 - (Avoid Character-level: too granular; Word-level: too large vocab, frequent OOV).

- **Tokens to IDs:** Convert tokens to numerical IDs.
 - **Lookup Table:** Map each unique token to an ID (1:1 mapping).
 - **Pros:** Quick conversion, easy ID to token lookup.
 - **Cons:** Memory-intensive for large vocab, cannot handle unseen tokens, no collisions.
 - **Hashing (Feature Hashing):** Use a hash function to get IDs, no lookup table needed.
 - **Pros:** Memory efficient, handles unseen tokens.
 - **Cons:** Slower conversion (hash computation), no ID to token lookup, potential for collisions.
 - **Chosen Approach:** Lookup table is standard for Transformer-based models; hashing is for very large vocabularies or memory constraints. For 10M pairs, lookup table is fine.
- **Feature Engineering (Preparing Video Data):**
 - **Workflow:** Raw Video → Sample Frames → Image Preprocessing (Resizing, Scaling, Normalization) → Frame Embeddings.

4. Modeling:

- **Text Encoder Model:**
 - **Purpose:** Convert text query into a meaningful embedding vector.
 - **Choice: Transformer-based models (e.g., BERT).**
 - **Why:** Captures semantic and contextual meaning (word order, different embeddings for same word in different contexts). Powerful for NLP tasks.
 - **Avoid:** Statistical methods (BoW, TF-IDF) due to limitations in capturing word order/semantic meaning, and sparse representations. Word2vec/Embedding layers (simple, but less contextual than Transformers).
 - **Key Components (Deep dive if asked):**
 - **Token Embedding:** Converts token IDs to dense vectors.
 - **Position Embedding:** Adds positional information (fixed or learned) to token embeddings for sequence order.
 - **Multi-Head Attention/Self-Attention:** Allows tokens to weigh importance of other tokens in the sequence. Output of multiple heads are concatenated and linearly transformed.
 - **Prediction Head:** (Not relevant for encoder's primary output, but generally part of pretext task training).
- **Video Encoder Model:**
 - **Purpose:** Convert video into a meaningful embedding vector.
 - **Architectural Options:**
 - **Video-level Models:** Process whole video (e.g., 3D CNNs, Video Transformers).
 - **Pros:** Captures full temporal context (actions, motions).
 - **Cons:** Computationally expensive, slower.
 - **Frame-level Models:** (Chosen Approach)
 - **Process:** Sample frames from video → run image model on frames → aggregate frame embeddings (e.g., average) into video embedding.
 - **Model:** ViT (Vision Transformer) or ResNet applied per frame.

- **Pros:** Faster training/serving speed, computationally less expensive.
 - **Cons:** May not fully understand temporal aspects (actions/motions).
- **Chosen Approach: Frame-level model (e.g., ViT)** due to speed and computational efficiency, as temporal understanding is not *crucial* for basic video search here.
- **Model Training (Contrastive Learning):**
 - **Objective:** Optimize parameters of both Text and Video Encoders simultaneously.
 - **Goal:** Ensure **relevant video-text pairs have high similarity (dot product)** in the embedding space, and irrelevant pairs have low similarity.
 - **Loss Function:** Similar to the Visual Search system: Contrastive Loss.
 - Compute dot product similarity between text embedding (ET) and each video embedding (EV).
 - Apply **Softmax** over these similarities to get probabilities.
 - Compute **Cross-Entropy Loss** against the ground truth (the one relevant video among the candidates).
 - **Formula:** $L = -\log(P_{\text{relevant_video}})$ (where $P_{\text{relevant_video}}$ is the softmax probability assigned to the true relevant video).

5. Evaluation:

- **Offline Metrics:**
 - **Evaluation Dataset:** Given $\langle \text{text query, associated relevant video} \rangle$ pairs.
 - **Choice: Mean Reciprocal Rank (MRR).**
 - **Why:** Addresses shortcomings of Precision@k and Recall@k for this specific dataset setup (where only one relevant video per query is given). MRR measures how high the *first relevant item* is ranked.
 - **Formula:** $MRR = \frac{1}{m} \sum_{i=1}^m \frac{1}{\text{rank}_i}$ (where m is total queries, rank_i is rank of first relevant item for query i).
 - (Avoid Precision@k/mAP: Numerator is max 1, leads to low values. Avoid Recall@k: Always 0 or 1, doesn't distinguish between rank 15 vs 50).
- **Online Metrics:**
 - **Click-Through Rate (CTR):** $\text{CTR} = \frac{\text{Total number of suggested videos}}{\text{Number of clicked videos}}$. Good general engagement metric.
 - **Video Completion Rate:** Percentage of videos watched till the end from search results.
 - **Total Watch Time of Search Results:** Sum of time users spend watching videos retrieved from search. **Strong indicator of relevance/engagement.**

6. Deployment & Serving:

- **Prediction Pipeline:**
 1. **Text Query Input:** User enters query.
 2. **Visual Search Component:**

- Text Query → Text Encoder → Text Embedding.
 - Text Embedding → ANN Nearest Neighbor Service → Top-K Video Embeddings.
 - **(Leverage)** ANN algorithms (e.g., Faiss, ScaNN) for billions of videos, as in Visual Search System.
- 3. **Text Search Component:**
 - Text Query → Elasticsearch (using Inverted Index) → Top-K Videos based on title/tag overlap.
- 4. **Fusing Layer:**
 - Combines ranked lists from Visual Search and Text Search.
 - **Implementation:** Weighted sum of predicted relevance scores (simpler, faster at serving than training a new model).
- 5. **Re-ranking Service:**
 - Applies business logic: filters inappropriate content, removes duplicates, enforces policies.
- **Indexing Pipelines:** (Two parallel pipelines)
 1. **Video Indexing Pipeline:**
 - Raw Video → Video Encoder → Video Embedding.
 - Video Embedding → Indexing Service (stores embedding, uses vector quantization for memory efficiency) → Nearest Neighbor Service.
 2. **Text Indexing Pipeline:**
 - Video Titles, Manual Tags, Auto-Generated Tags → Text Preprocessing (normalization, tokenization, IDs) → Elasticsearch (Inverted Index).
 - **Auto-Tagger Component:** Standalone ML model (e.g., a text classifier or generative model) to generate tags for videos without manual tags. Valuable for discoverability, even if noisier.

7. Monitoring:

- **System Health:** Latency, throughput, error rates for all services (encoders, NN, Elasticsearch, fusing, re-ranking).
- **Model Performance (Offline):** Continuous MRR monitoring on evaluation datasets to track embedding quality and retrieval accuracy.
- **Online Performance:** Monitor CTR, Video Completion Rate, and Total Watch Time of search results. Alert on significant deviations.
- **Data Quality:** Monitor text query distribution, video ingestion rates, and tag quality (manual vs. auto-generated).
- **ANN Accuracy:** Periodically evaluate ANN recall/precision against exact NN on a small sample to ensure search quality is maintained.

Other Potential Discussion Points (for senior roles):

- **Multi-Stage Design:** Candidate Generation (faster, broad retrieval) + Re-ranking (slower, more complex ranking logic).
- **Additional Video Features:** Incorporate video length, popularity, freshness, creator reputation, etc., into the ranking.

- **Interaction-based Training:** Transition from annotated dataset to continuous training using real-time user interactions (clicks, watch time, shares) for labels. *Strategic: Aligns with Pinterest's Homefeed approach and your expertise.*
- **Query Understanding:** Spelling correction, query category identification, entity recognition components.
- **Multi-Modal System:** Incorporate speech and audio content of videos for better relevance.
- **Internationalization:** Support for other languages.
- **Near-Duplicate Detection:** Remove highly similar videos from results.
- **Query Taxonomy:** Tailoring search approaches for head, torso, and tail queries.
- **Real-world System Complexity:** Acknowledge the simplified nature and mention areas of added complexity in production systems (e.g., personalization, complex re-ranking).

[Design] Video / Newsfeed Recommendation

Alright, James, let's break down this Video Recommendation System design, specifically a homepage system similar to YouTube's. This aligns very well with your current role at Pinterest in Homefeed recommendations and your ambition to be a Director of Engineering, as it touches on personalization, scale, and complex ML pipelines.

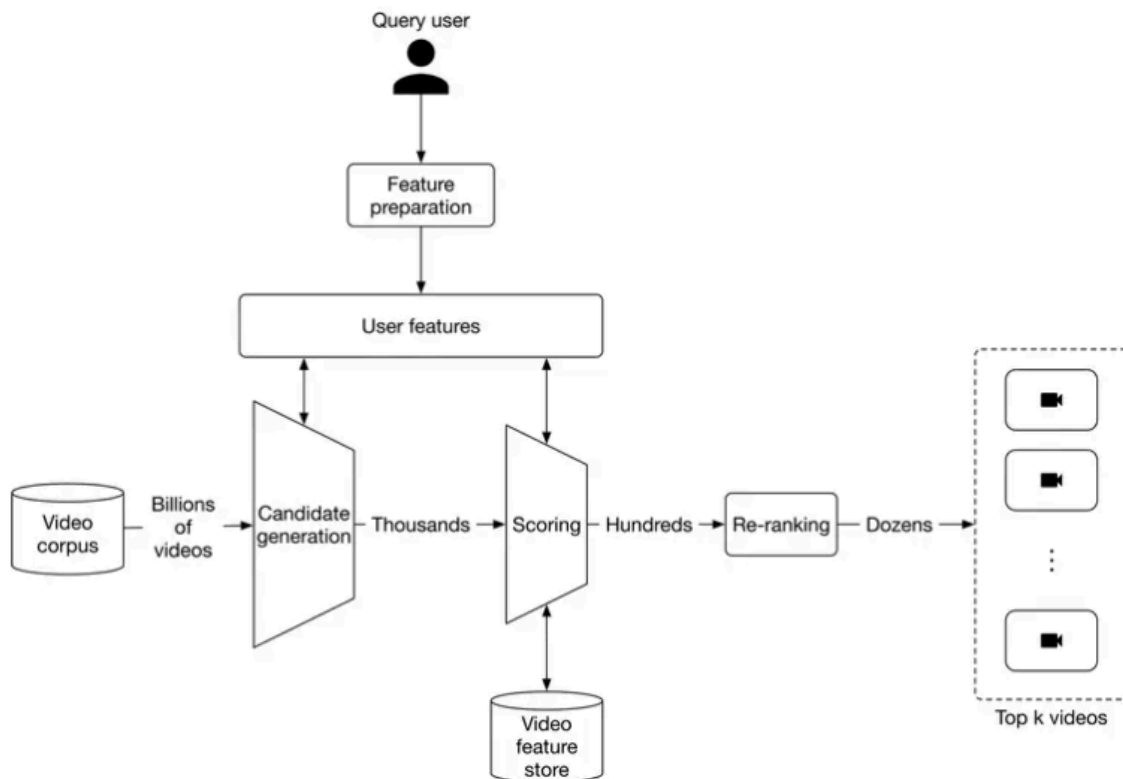


Figure 6.23: Prediction pipeline

1. Clarify Requirements:

- **Primary Goal:** Increase user engagement by recommending personalized videos on the user's homepage.

- **Input/Output:** User profile/context → Ranked list of engaging videos.
- **Media Type:** Only videos.
- **Relevance:** Maximize the **number of relevant videos** (ML Objective choice).
Relevance defined by implicit/explicit user reactions (e.g., explicit like, watch $\geq 50\%$ of video).
- **Training Data:** Construct dataset based on user interactions with video content.
- **Language:** Users and videos worldwide, multi-language support assumed.
- **Scale:** ≈ 10 billion videos on the platform.
- **Latency:** Recommendations ≤ 200 milliseconds.
- **Simplifications:** No playlists feature.

2. Frame as ML Task:

- **Business Objective:** Increase user engagement.
- **ML Objective (Chosen):** Maximize the **number of relevant videos**.
 - **Rationale:** Provides explicit control over signals, avoids "clickbait" issues (max clicks) and "short video bias" (max completed videos).
- **System Input:** User ID and contextual information.
- **System Output:** Ranked list of videos.
- **ML Category: Personalized Recommendation System**, specifically **Hybrid Filtering** (combining Collaborative Filtering and Content-Based Filtering).
 - **Rationale:** Hybrid approach leverages strengths of both: CF for discovering new interests and general trends (no domain knowledge needed), Content-based for new videos (cold-start) and capturing unique niche interests (requires video features/domain). They are complementary.
 - **Implementation Strategy:** Sequential Hybrid Filtering (e.g., CF-based for **Candidate Generation**, Content-based for **Scoring/Ranking**).

3. Data Preparation:

- **Data Choice:**
 - **Videos:** Raw video files, Video ID, Duration, Language, Titles, Tags (manual/auto-generated).
 - **Users:** User ID, Demographics (Age, Gender, City, Country, Language, Time zone).
 - **User-Video Interactions:** Likes, Clicks, Impressions, Watch Time, Past Searches, Comments. Includes contextual info (Location, Timestamp).
- **Feature Engineering:**
 - **Video Features:**
 - **Video ID:** Embedding layer (learned during training).
 - **Duration:** Numerical feature (users prefer certain lengths).
 - **Language:** Embedding layer (categorical).
 - **Titles & Tags:** Map into feature vectors.
 - **Tags:** Lightweight pre-trained model (e.g., CBOW) for feature vectors.
 - **Titles:** Context-aware word embedding model (e.g., pre-trained BERT) for feature vectors.
 - **User Features:**

- **User Demographics:** Age, Gender (embeddings if categorical), City, Country, Language, Time Zone.
- **Contextual Information:**
 - **Time of Day:** User preference variation.
 - **Device:** e.g., shorter videos on mobile.
 - **Day of Week:** Behavioral shifts.
- **User Historical Interactions:** (Key for personalization!)
 - **Search History:** Pre-trained BERT for query embeddings, then average embeddings for fixed-size vector.
 - **Liked Videos:** Video IDs → embedding layer, then average embeddings.
 - **Watched Videos / Impressions:** Similar to liked videos.
- **Constructing Dataset for Training:**
 - **Data Point:** Features from a $\langle \text{user}, \text{video} \rangle$ pair.
 - **Labeling:** "Positive" if user explicitly liked or watched $\geq 50\%$ (combines explicit/implicit feedback for relevance). "Negative" from random videos not interacted with, or explicitly disliked.
 - **Imbalanced Data:** Acknowledge this issue (many more negatives than positives) and note the need for techniques to address it (e.g., negative sampling, re-weighting, data augmentation for positives).

4. Modeling:

- **Embedding-Based Models:** Goal is to learn user and video embeddings such that their distance (e.g., dot product) represents relevance.
 - **A. Matrix Factorization (MF):** (Typically for Candidate Generation / CF-based)
 - **Concept:** Decomposes a sparse user-video feedback matrix into lower-dimensional user and video embedding matrices.
 - **Feedback Matrix Construction: Combination of explicit and implicit feedback** for relevance (e.g., like=1, watch 50%+=1, no interaction=0, dislike=0).
 - **Loss Function: Weighted combination of squared distance** over observed and unobserved $\langle \text{user}, \text{video} \rangle$ pairs.
 - **Purpose:** Addresses MF's weakness of not penalizing unobserved pairs (leading to all-ones embeddings) and avoiding unobserved pairs dominating observed ones (leading to all-zeros predictions).
 - **Formula:** First summation for observed, second for unobserved, with a weight W for balance.
 - **Optimization:** Weighted Alternating Least Squares (WALS) preferred for faster convergence and parallelizability over SGD.
 - **Pros:** Efficient training and serving (embeddings are static after learning).
 - **Cons:** Relies *only* on user-video interactions (no other features like age, language), struggles with new users (cold-start) as no interactions mean no meaningful embedding.
 - **B. Two-Tower Neural Network:** (Preferred for Ranking/Scoring / Hybrid)

- **Architecture:** Two encoder towers: **User Tower** (User features → User Embedding) and **Video Tower** (Video features → Video Embedding). Shared embedding space.
- **Loss Function: Cross-Entropy** (for binary classification of user-video relevance). (See formula in prior responses).
- **Pros:**
 - **Utilizes Rich Features:** Can incorporate *all* user features (demographics, context, history) and video features (duration, title, tags). This enhances predictive capability and personalization.
 - **Handles New Users (Cold Start):** Can make recommendations based on demographic/contextual features even without interaction history.
- **Cons:**
 - **Slower Serving:** User embedding must be computed at query time. If using video features in video tower, video embeddings also need re-computation if not pre-indexed.
 - **More Expensive Training:** More parameters than MF.

5. Evaluation:

- **Offline Metrics:**
 - **Precision@k:** Proportion of relevant videos in top k. (Use multiple k values).
 - **mAP (Mean Average Precision):** Measures ranking quality for binary relevance.
 - **Diversity:** Calculate average pairwise similarity (e.g., cosine similarity) of videos in the recommended list. Lower score = higher diversity.
 - **Note:** Use in conjunction with relevance metrics; diverse but irrelevant is bad.
- **Online Metrics:** (Focus on increasing user engagement - the business objective)
 - **Click-Through Rate (CTR):** Clicks / Impressions. Good for initial engagement, but can be susceptible to clickbait.
 - **Number of Completed Videos:** How many recommended videos users watch to the end.
 - **Total Watch Time:** Total time users spend watching recommended videos. **Strong indicator of relevance and engagement.** (Crucial for YouTube-like systems).
 - **Explicit User Feedback:** Likes/Dislikes. Most accurate reflection of user opinion.

6. Deployment & Serving (Multi-Stage Design):

- **Prediction Pipeline:**
 - **Goal:** Efficiently narrow down billions to thousands (Candidate Gen), then accurately rank thousands (Scoring). Maximize efficiency in stage 1, accuracy in stage 2.
 - **Components:**
 - **Candidate Generation (Stage 1):**
 - **Goal:** Narrow 10B videos → thousands. Prioritize **efficiency** over accuracy.

- **Model: Two-Tower Neural Network** (CF-style: user features → user embedding; video ID → video embedding). Choose this as it handles new users and is relatively lightweight for this stage.
 - **Process:** User ID → User Encoder → User Embedding. User Embedding → ANN Nearest Neighbor Service (against billions of pre-computed video embeddings) → Top-K candidate videos.
 - **Diversification:** Can use *multiple candidate generators* (e.g., based on popularity, trending, user's location) to ensure diversity of recommendations.
- **Scoring (Ranking) (Stage 2):**
 - **Goal:** Rank the thousands of candidates accurately. Prioritize **accuracy** over efficiency.
 - **Model: Two-Tower Neural Network** (Content-based style: uses all available rich user and video features to compute relevance scores). Can be heavier model due to smaller candidate set.
 - **Process:** Takes user and *candidate* video features, computes fine-grained relevance scores for each candidate video.
- **Re-ranking:**
 - Applies business rules/constraints: region restrictions, freshness boosts, misinformation filtering, duplicate removal, fairness/bias adjustments, explicit clickbait filtering models.
- **Indexing Pipelines:**
 - **Video Indexing Pipeline:**
 - Raw Video → Video Encoder (from the two-tower model) → Video Embedding.
 - Video Embedding → Indexing Service (stores embedding, uses **vector quantization** for memory efficiency for NN search) → ANN Nearest Neighbor Service.
 - **User Indexing Pipeline:**
 - User Features → User Encoder (from the two-tower model) → User Embedding.
 - User Embedding → Indexing Service (for quick retrieval during Candidate Generation).

7. Monitoring:

- **System Health:** Latency (critical for <200ms), throughput, error rates for all services.
- **Model Performance (Offline):** Continuous monitoring of Precision@k, mAP, Diversity on validation sets.
- **Online Performance:** Real-time tracking of CTR, Video Completion Rate, Total Watch Time, Explicit Feedback. Set up alerts for unexpected drops or changes.
- **Data Quality:** Monitor incoming user interaction data (clicks, likes, watch time) for integrity and distribution shifts.
- **Cold Start Handling:** Monitor engagement metrics for new users/videos to validate cold-start strategies.
- **Bias Monitoring:** Track metrics across different user demographics or content types to ensure fairness.

Other Potential Discussion Points (for senior roles / if time permits):

- **Exploration-Exploitation Trade-off:** How to balance recommending known relevant videos (exploitation) with recommending novel or new videos to discover new interests (exploration).
- **Bias Mitigation:** Detailed strategies for addressing popularity bias, positional bias, feedback loop biases.
- **Multi-Objective Optimization:** Optimizing for watch time, diversity, freshness, and relevance simultaneously.
- **Negative Feedback:** Explicitly using dislikes or skips to refine recommendations.
- **Sequence Modeling:** Incorporating the *sequence* of watched/searched videos using RNNs or Transformers in user embeddings for richer context. *Strategic: Aligns with your AI/ML interests.*
- **Seasonality:** Adapting recommendations to changes in user behavior over seasons/holidays.

[Design] Similar Content / Visual Search Recommendation

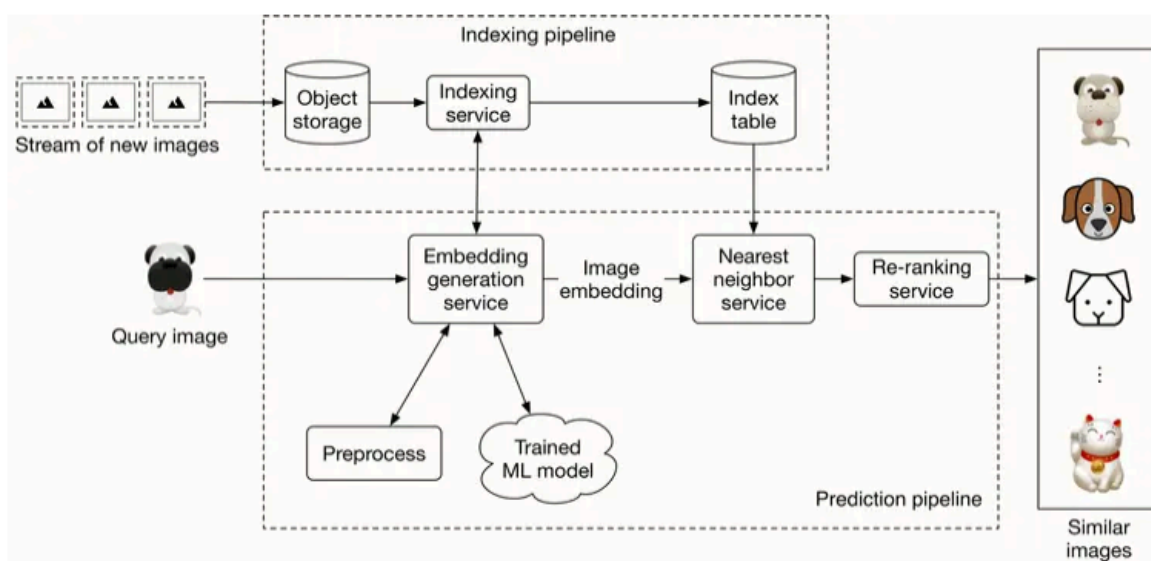


Figure 2.19: Prediction and indexing pipeline

1. Clarify Requirements:

- **Primary Goal:** Retrieve images visually similar to a user-provided query image, ranked by similarity.
- **Input/Output:** Query image (can be a crop) → Ranked list of visually similar images.
- **Media Type:** Only images. No video, no text queries (for simplicity).
- **Personalization:** None. Same query image yields same results for all users.

- **Metadata Usage:** No reliance on image metadata (tags, etc.), only image pixels (for simplicity, but note this is a simplification from real-world).
- **User Actions:** Only image clicks (as proxy for similarity). No save/share/like (for simplicity).
- **Content Moderation:** Out of scope for this design, but acknowledged as critical for a safe platform.
- **Training Data:** Online construction using user interactions (clicks as proxy) is the expected approach.
- **Scale:** 100-200 billion images on the platform. Search must be *fast*.

2. Frame as ML Task:

- **ML Objective:** Accurately retrieve images visually similar to the query image.
- **System Input:** Query image (pixels).
- **System Output:** Ranked list of similar images.
- **ML Category: Ranking Problem** solved via **Representation Learning**.
 - **Representation Learning:** Train a model to transform input images into N-dimensional **embedding vectors** (points in an embedding space). The goal is for **similar images to have embeddings in close proximity**.
 - **Ranking Process:**
 1. Compute query image embedding.
 2. Calculate similarity scores (distances) between query embedding and all other image embeddings in the platform.
 3. Rank images by these similarity scores (most similar first).

3. Data Preparation:

- **Data Choice:**
 - **Images:** Raw image pixel data (including cropped query images).
 - **User-Image Interactions:** Impressions and Clicks (User ID, Query Image ID, Displayed Image ID, Position, Interaction type, Timestamp). Clicks are key for positive signal.
- **Feature Engineering (Image Preprocessing):**
 - **Resizing:** Standardize to fixed dimensions (e.g., 224x224 pixels) for model input.
 - **Scaling:** Normalize pixel values (e.g., 0-1 range).
 - **Z-score Normalization:** Mean 0, Variance 1 for better model convergence.
 - **Consistent Color Mode:** Ensure uniform color representation (e.g., RGB).
- **Constructing Dataset for Contrastive Training:**
 - **Data Point:** (Query Image, Positive Image, N-1 Negative Images). Ground truth is the index of the positive image among the N candidates.
 - **Positive Image Selection Options (and Trade-offs):**
 1. **Human Judgments:** High accuracy, but very expensive, time-consuming, not scalable for 100-200B images.
 2. **User Clicks (Proxy):** Automatic, no manual work. **Pros:** Real-world implicit feedback. **Cons:** Noisy (users click for many reasons), sparse (most images lack clicks), leads to poor performance if used naively.

3. **Self-Supervision (Data Augmentation):** Artificially create similar images by augmenting the query image (e.g., rotation, cropping, color jitter). Frameworks like SimCLR/MoCo use this.
 - **Pros:** No manual work, fully automated, less noisy (augmented image is always similar), scalable to huge datasets.
 - **Cons:** Artificial similarity may not perfectly reflect real-world visual/semantic similarity.
- **Chosen Approach:** Start with **Self-Supervision (Data Augmentation)** due to automation, no upfront cost, and scalability with billions of images (as shown by SimCLR results). Acknowledge that we can iterate and incorporate noisy click data later or use human labeling for refinement if initial results are unsatisfactory. *Strategic point: This demonstrates adaptability and an iterative approach.*

4. Modeling:

- **Model Selection:**
 - **Neural Networks (NNs):** Ideal for unstructured data like images and capable of producing learned embeddings for representation learning.
 - **Architectures:** CNN-based (e.g., ResNet) or Transformer-based (e.g., ViT) are strong candidates for image processing.
 - **Simplified Model Architecture:** (Figure 2.5) Input Image → (Convolutional Layers → Fully Connected Layers) → Embedding Vector. Hyperparameters (layers, neurons, embedding size) are tuned experimentally.
- **Model Training (Contrastive Training):**
 - **Goal:** Optimize model parameters so that similar images have embeddings close to each other, and dissimilar images are far apart.
 - **Process:** Provide (Query Image, Positive Image, N-1 Negative Images). Model learns to make positive closer to query than negatives.
 - **Loss Function (Simplified Contrastive Loss):**
 1. **Compute Similarities:** Measure similarity between query embedding (Eq) and candidate embeddings (Ex) using:
 - **Dot Product:** $E_q \cdot E_x$
 - **Cosine Similarity:** $\frac{E_q \cdot E_x}{\|E_q\| \cdot \|E_x\|}$ (often preferred as it's scale-invariant for embedding vectors)
 - (Note: Euclidean distance generally performs poorly in high dimensions due to curse of dimensionality).
 2. **Softmax:** Apply softmax over the computed similarities to convert them into probabilities:

$$P_i = \frac{\exp(\text{similarity}(E_q, E_i))}{\sum_{j=1}^N \exp(\text{similarity}(E_q, E_j))}$$
 3. **Cross-Entropy:** Compute Cross-Entropy loss between these predicted probabilities and the one-hot ground truth label (index of the positive image).

$$L_{\text{Contrastive}} = -\sum_{i=1}^N y_i \log(P_i) \quad (\text{where } y_i = 1 \text{ for positive image, } 0 \text{ for negatives}).$$
 - **Purpose:** Minimizing this loss ensures the model assigns high probability to the positive image and low probabilities to negative images, indicating good separation in embedding space.
- **Pre-training / Fine-tuning:** Leverage pre-trained contrastive models (e.g., from ImageNet or large web datasets) and then fine-tune on Pinterest's specific interaction

data. This significantly reduces training time and leverages learned general representations. *Strategic point: This aligns with operational excellence and efficient resource utilization.*

5. Evaluation:

- **Offline Metrics:**
 - **Evaluation Dataset:** Query image, candidate images, and continuous ground truth similarity scores (0-5).
 - **Choice: nDCG (Normalized Discounted Cumulative Gain).**
 - **Why:** Unlike MRR, Recall@k, or Precision@k, nDCG explicitly accounts for the *graded relevance* (0-5 scores) and the *position* of relevant items in the ranked list, giving higher scores to highly relevant items placed at the top. This perfectly aligns with the requirement to rank "most similar to least similar."
 - **Formulas:**
 - $DCG_p = \sum_{i=1}^p \log_2(i+1) \text{rel}_i$
 - $nDCG_p = DCG_p / DCG_p^{\text{ideal}}$ (where $IDCG$ is DCG of ideal ranking)
 - *(Avoid MRR, Recall@k, Precision@k shortcomings as discussed in context).*
- **Online Metrics:**
 - **Click-Through Rate (CTR):** $CTR = \frac{\text{Total number of suggested images clicked}}{\text{Total number of suggested images}}$. A high CTR indicates users find suggestions relevant and click frequently.
 - **Average Daily/Weekly/Monthly Time Spent on Suggested Images:** Measures user engagement with search results. Expected to increase with system accuracy.
 - *(Self-reflection: For Pinterest, this aligns with key metrics for user engagement and growth).*

6. Deployment & Serving:

- **Prediction Pipeline:** (Figure 2.19 - right side)
 1. **Embedding Generation Service:**
 - Preprocesses query image (resizing, scaling, normalization, color mode).
 - Passes preprocessed image through the trained embedding model.
 - Outputs the query image embedding.
 2. **Nearest Neighbor Service:**
 - Takes the query embedding.
 - Performs a **Nearest Neighbor (NN) search** against the indexed embeddings of all images on the platform.
 - **Choice: Approximate Nearest Neighbor (ANN) algorithms** (e.g., from Faiss, ScaNN) are mandatory due to **100-200 billion images** (Exact NN / Linear Search is $O(N \times D)$ and too slow).
 - **ANN Categories (High-level understanding):** Tree-based (Kd-trees, Annoy), Locality-Sensitive Hashing (LSH), Clustering-based. These reduce search complexity to sub-linear (e.g., $O(D \log N)$).
 3. **Re-ranking Service:**
 - Applies business logic and policies: filters inappropriate results, removes private/duplicate images, applies other product rules.

- Ensures final display list adheres to platform standards.
- **Indexing Pipeline:** (Figure 2.19 - left side)
 1. **Indexing Service:**
 - Responsible for processing all images on the platform into embeddings (using the *same* trained model as the prediction pipeline).
 - Adds these embeddings to the index table for the Nearest Neighbor Service.
 - Keeps the index updated as new images are uploaded (e.g., near real-time ingestion).
 - **Optimization:** Uses **vector quantization** (e.g., Product Quantization) to reduce memory usage for storing billions of high-dimensional embeddings.

7. Monitoring:

- **System Health:** Latency, throughput, error rates of Embedding Generation, Nearest Neighbor, and Re-ranking services.
- **Model Performance (Offline):** Continuous nDCG monitoring on held-out evaluation sets to detect model/data drift.
- **Model Performance (Online):** Real-time monitoring of CTR and Average Time Spent metrics. Alerting on significant drops or unexpected changes.
- **Data Quality:** Monitor incoming image data for anomalies. Monitor clicks for changes in patterns.
- **Nearest Neighbor Accuracy:** Regularly sample ANN results and compare against Exact NN (for a small subset) to monitor the recall-precision trade-off of the ANN algorithm.

Other Potential Discussion Points (if time permits / for senior roles):

- **Content Moderation:** Briefly mention the integration points for safety classifiers (e.g., after initial retrieval, or as part of the re-ranking service to filter NSFW/inappropriate images). *Strategic: Connects to your expertise and Anthropic's mission.*
- **Image Metadata:** How tags, contextual info, etc., could be used to improve results (e.g., multi-modal embeddings, filtering).
- **Active Learning / Human-in-the-Loop:** For efficiently labeling ambiguous examples or identifying new types of similar images for training data. *Strategic: Aligns with your "scaling through TLs/EMs" and "operational excellence" strength.*
- **Biases:** Discuss potential biases (e.g., positional bias, popularity bias) in click data and how to mitigate them in training or re-ranking.

[Design] Chat System

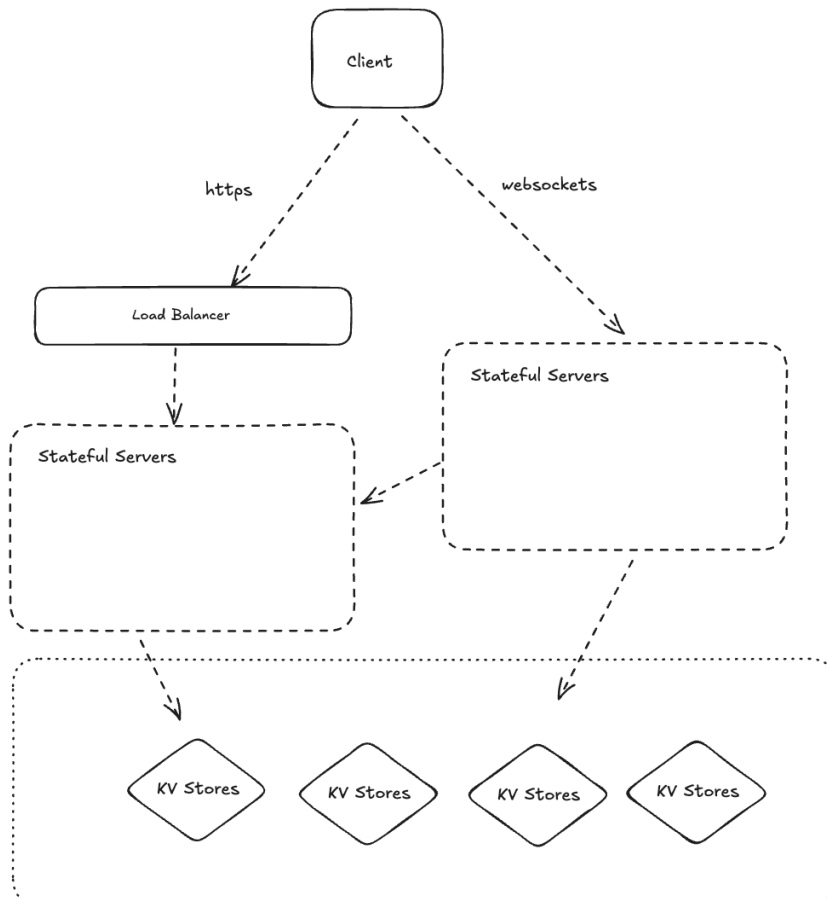
Ask Clarifying Questions:

- Primarily 1:1 or Group? Both

- Group limit? 100
- Mobile or Web app? Both
- Scale? 50m DAU
- Important features (e.g. attachments? Online indicators?) Online, push notification, no attachments
- Message size limit? Yes less than 100k characters
- E2E encryption? Maybe
- Storage time? Forever

High level proposal:

- Sender client → server → receiver client
 - Server does 2 things: 1) receive message, 1) store message, 2) relay message to the right recipient(s)
- Sender request
 - HTTP works well, it's classic
- Receiver request
 - Websockets (bidirectional and persistent) since it's much more efficient, especially for the server → receiver client, downside is that there's additional complexity since you'll have to handle dropped connections, reconnection logic, and heartbeats
 - Polling (client periodically ask) or long polling (hold the connection until timeout or message)
- Stateful requests (client <> server) → WS
- Stateless requests (client → server: signups, profile, et.c) → HTTPS
 - Need a load balancer



- Storage: lots of chat, recent chats access frequently, might require some random access sometimes, read to write ratio is about 1:1
 - Nosql (HBase or Cassandra) and KvStore(dynamoDB or Redis) allow for easy horizontal scaling, low latency reads, long tail access is cheaper (relative to SQL)
- Data Model for 1:1
 - Message_id
 - Sender_id
 - receiver_id
 - Content
 - Created_timestamp
- Data Model for group chats
 - channel_id
 - Message_id
 - Sender_id
 - Content
 - Created_timestamp
- Message queues are designed for async communication that decouples senders from receivers. Fault tolerance, async processing for logging, pipeline audits, handles traffic spikes via buffering. PubSub system for example allows real-time monitoring and offline logging to be multiple subscribers.

Model Sampling

1. Greedy search - take top probability token at each step
 - a. Fast and simple, good for deterministic responses
 - b. Tends to be hallucinate the least if model is well-trained
 - c. Lacks diversity and can be repetitive
2. Beam search - keeps track of K most probable sequences, then at next step produce K most likely tokens, and produce the new top K most probable. Keep going until EOS token or max length. Select the sequence with highest cumulative probability.
 - a. More relevant than greedy search
 - b. Much more coherent and fluent
 - c. Very high computational costs
 - d. Lacks diversity and creativity
3. Top-K sampling: reduce vocab to K most probable tokens, randomly sample one token from reduced set
 - a. Introduces more diversity via randomness
 - b. Since we pruned, it prevents gibberish
 - c. For large K this can still be less relevant
4. Top-P sampling: select the smallest set of tokens such that the cumulative probability exceeds P, then random sample from this set.
 - a. Adaptive to the probability distribution
 - b. More human like
 - c. Still need to tune P
5. Add in a temperature term that pushes towards more deterministic versus more diversity / randomness.

Seq2seq, Attention, Transformers

- Seq2seq: Input sequence—>Output sequence using encoders and decoders
 - Encoder: LSTM cells and layers, each layer unrolls with token embeddings and previous hidden states as input, until EOS
 - Hidden states and cell states gets updated
 - These are fed into the next layer, and instead of word embeddings it's the higher order representation from the layer below
 - The finally layer's final hidden and cell states become the context vector
 - Decoder: Uses the context vector as well as the input word embedding and feeds into the LSTM cells and module, producing the hidden vector for the next word in the sequence. Separately it predicts the output word, and keeps going until it predicts EOS.
- Simple Attention —> decoder does cross attention with encoder tokens
 - Encoder: Processes input sequence into a set of hidden states (representations of each input token).
 - Decoder: Uses the context vector and performs cross-attention with the encoder's hidden states to selectively focus on relevant parts of the input sequence when generating each output token.
- Transformers —> multi-headed self attention + positional encodings + context matrix + (residual connections and layer norm and decoder masking)
 - Encoder: Made of multiple identical layers. Each layer has a multi-head self-attention mechanism followed by a position-wise feed-forward network. Positional encodings are added to the input embeddings to provide sequence order information. The output is a set of context-rich representations for each input token.
 - Decoder: Also made of multiple identical layers. Each layer has three sub-layers: masked multi-head self-attention (to prevent attending to future tokens), multi-head cross-attention (to attend to the encoder's output), and a position-wise feed-forward network. Positional encodings are also used. Residual connections and layer normalization are applied throughout both the encoder and decoder.
 - $\text{Attention}(Q, K, V) = \text{softmax}(QK^T/\sqrt{d_K})V$

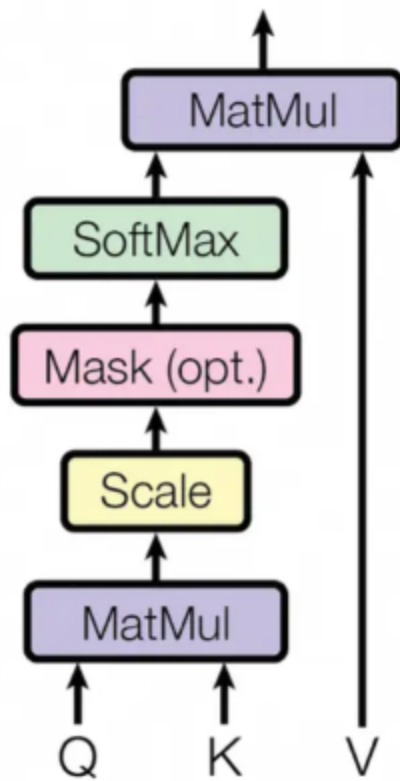


Figure 17: Scaled dot-product attention (Credit: [32])

-
- $\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head1}, \dots, \text{headN})W$

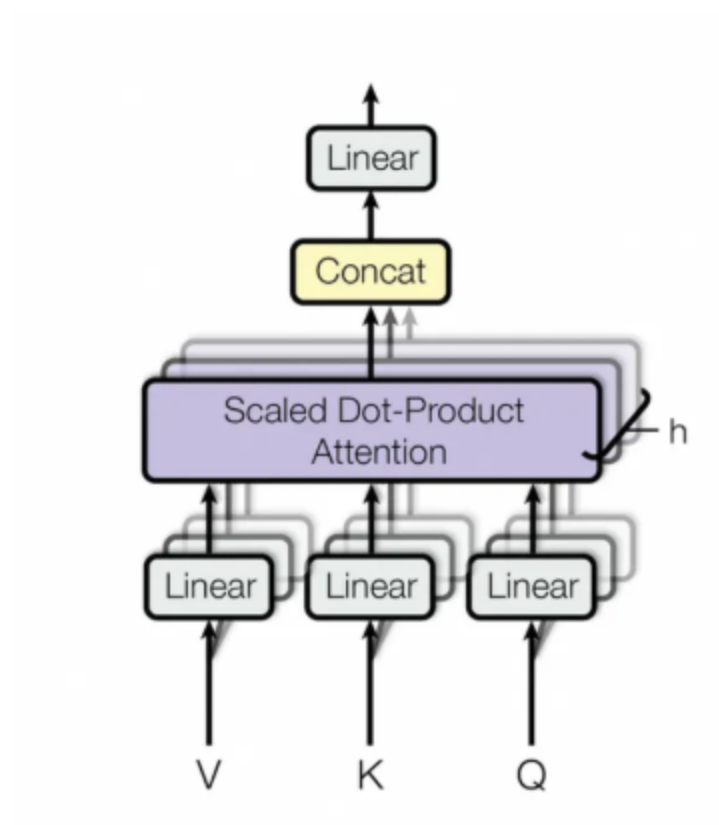


Figure 18: Multi-head attention (Credit: [32])

o

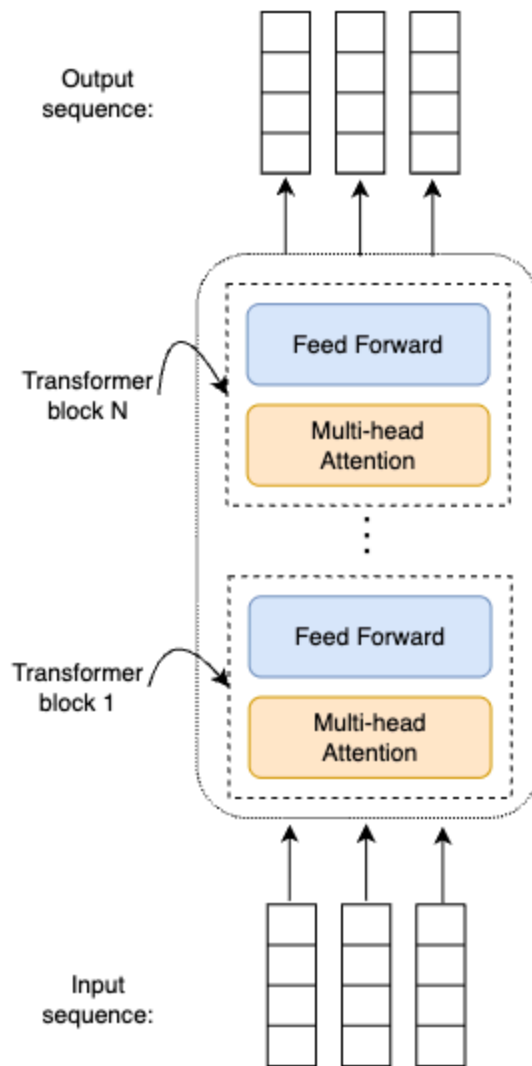


Figure 21: A simplified Transformer structure

○

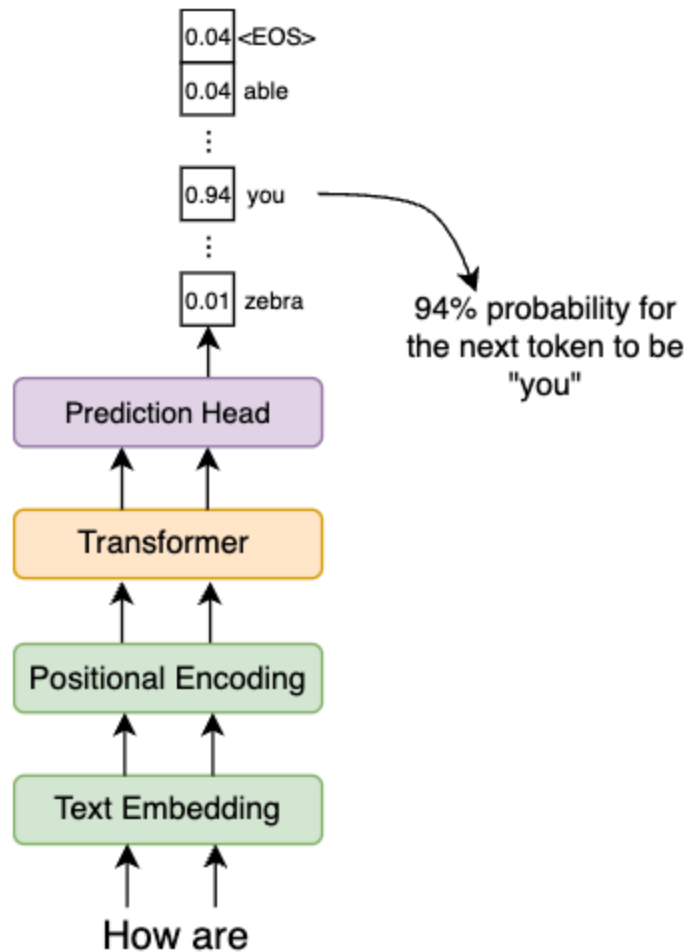


Figure 22: Prediction head output probabilities

○

RLHF / RLAIF / DPO

- Reinforcement Learning from Human Feedback
 - Consists of 3 stages: Supervised Fine Tuning on a small dataset of high-quality human curated demonstrations of desired behavior (input <> output pairs).
 - The LLM gives multiple responses to the same prompt, and then the human annotators compare and rank multiple responses at a time, creating a human preference dataset to train a smaller NN called reward model, which outputs a scalar reward score. Usually pairwise ranking loss.
 - Finetune the LLM using the reward model to maximize rewards, and balance with a KL divergence measure to prevent it from drifting too far from the original distribution (e.g. to maintain fluency).
 - Pro: highly effective, generalizable
 - Con: hard to train, with the final stage being very difficult to tune and can be unstable, and requires extensive, high-quality human labeling, you can get reward hacking (e.g. by being overly verbose or sycophantic)
- Reinforcement Learning from AI Feedback

- Replaces the reward model by human data with a reward model by AI data with constitutional principles in the prompt. These principles can be crowdsourced with company employees, and continuously updated and modified.
- Pros: scalability since we are not as dependent on human labeling to scale, transparency via the constitution → can use chain-of-thought to provide further insights
- Cons: if the AI itself is not well-aligned, then biases or undesired behaviors can propagate, so humans must stay in the loop, the constitution might not be able to encode human values entirely
- Direct Preference Optimization
 - Skips the second stage entirely and directly optimizes policy in the model finetuning of the third step
 - Negative loss likelihood similar to logistic regression, increase the likelihood of the preferred response from the pair and decrease the likelihood of the rejected response from the pair
 - Pros: simpler and more computationally efficient, no need for separate RM model, and comparable performance
 - Cons: highly dependent on the data quality, less flexible for complex rewards

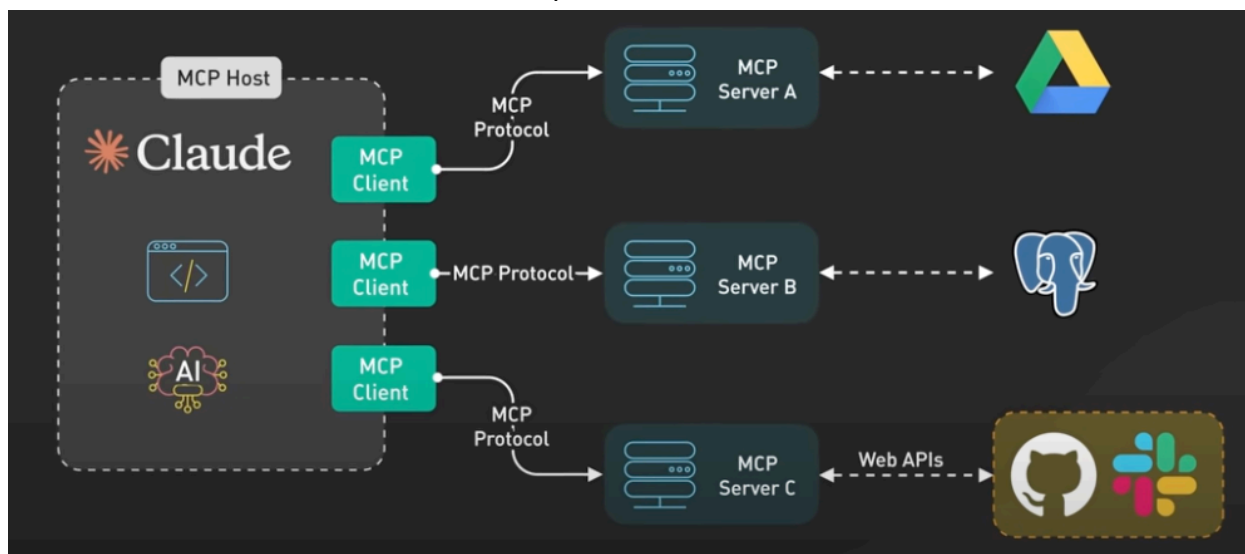
Loss Functions and Optimizers

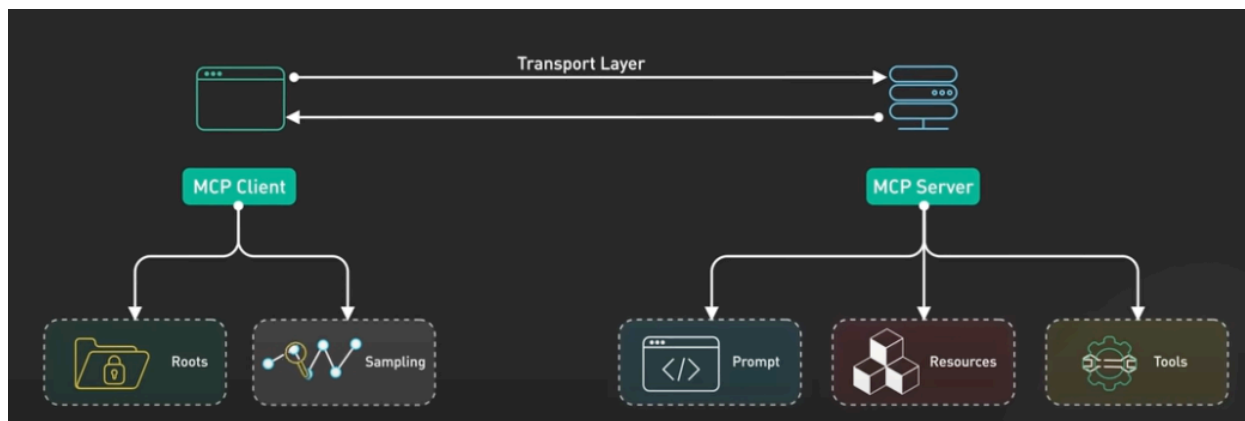
- Loss
 - Cross Entropy ($-\sum_{\text{categories}} (y_i * \log(\hat{y}_i))$) - typically better for mult-class problems since it maps to a value between 0 and 1, thereby encouraging the model to output higher probabilities for correct classes
 - Sensitive to class imbalance, can do weighted cross-entropy
 - Sensitive to mis-labeling which makes labeling and data-cleaning very important
 - Doesn't optimize for overall quality of output in LLMs, so need alignment techniques
 - Mean Squared Error ($\sum_{\text{instances}} (y_i - \hat{y}_i)^2$) - good choice for regression problems
 - Can be very skewed by outliers, not great for classification problems
 - Evidence Lower Bound for VAEs - reconstruction term (MSE or BCE) wants to make the reconstruction accurate, while KL divergence term (distance between learned posterior distribution and prior distribution) wants to regularize the latent space.
 - L1 - alternative choice for regression problems when you don't want to be too skewed by edge cases
 - Non-smoothness, optimization may run into more issues
 - Focal - variant of CE that increases loss for hard-to-classify examples
 - More compute, needs tuning focal parameter
 - Contrastive - encourages similar things to be closer in feature space while pushing dissimilar things further out → great for embedding generation
 - Need to construct negatives carefully, particular in-batch
- Optimizers
 - SGD and Minibatch SGD - efficient, allows you to escape local minima

- Adam - uses exponential decaying averages of first moments in order to generate adaptive learning rates based on second moments, only 3 parameters: step size, decay rates for gradient and squared gradient
- SOAP - introduces one additional hyperparameter from Adam, the preconditioning efficiency, and it doesn't have to compute the full hessian tensor, but use approximations, and can get faster convergence

MCP

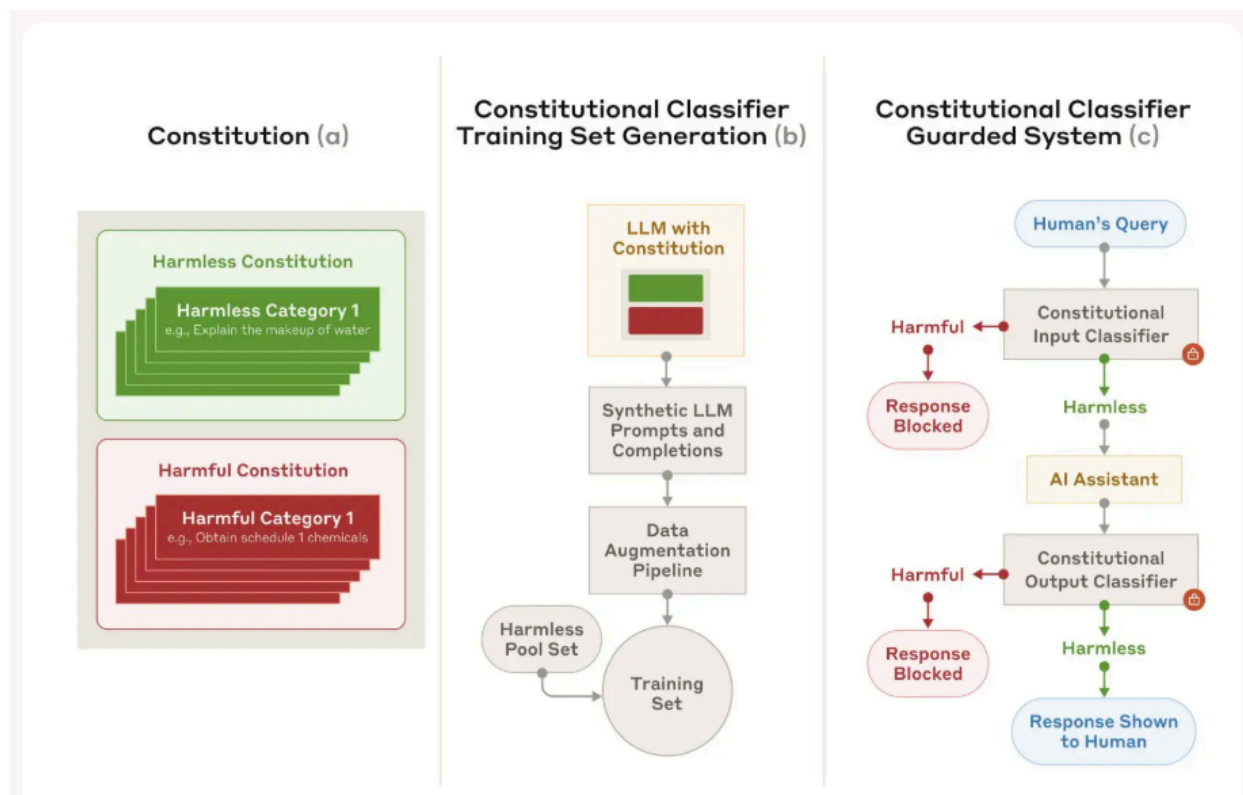
- Allow AI model providers and external data sources and tools to build once instead of N x N times, while maintaining Security and Context
- Hosts (LLM applications like Claude Desktop)
- Client are components within the host that establishes and maintains 1:1 connections with external servers
- Servers provide context, tools, and prompts
- 5 core primitives
 - Servers
 - Prompts → instructions or templates that can be injected into the LLM context, guiding the flow
 - Resources or data → allow the model the reference external information such as memory or knowledge
 - Tools → executable function that allows the model to call to modify files or run queries, etc.
 - MCP clients
 - Roots → creating a secure channel for file access on the local system (reading code, open documents, etc.)
 - Sampling → allows the server to request the LLM's help when needed (e.g. formulating queries), which creates a two-way interaction that makes it more flexible and powerful





AI Safety

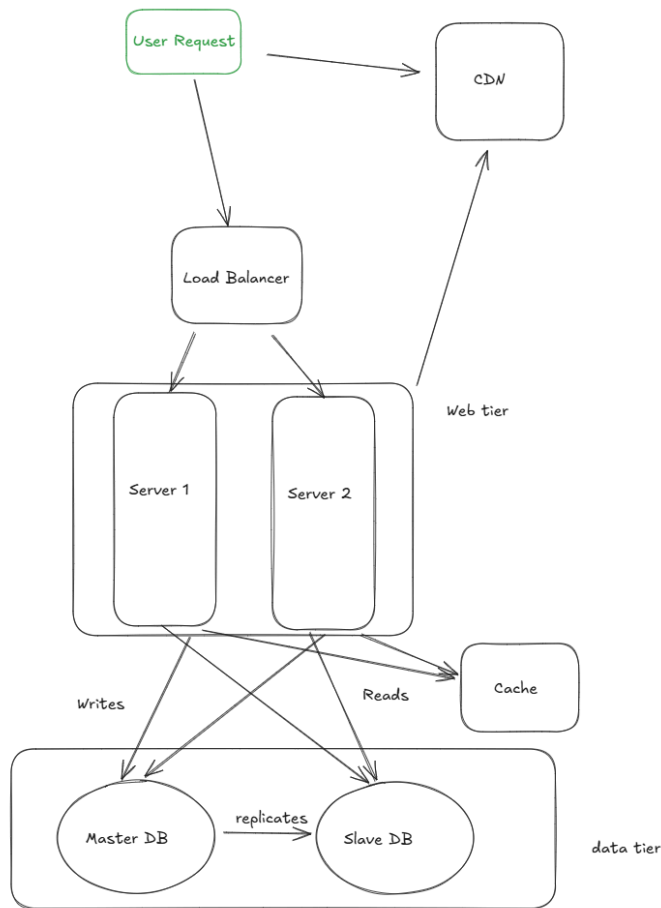
- Transformative AI is on its way if not already here. AI behavior can diverge from what the creators intend: toxicity, bias, dishonesty, hallucination.
- With the rate of progress, the applied systems need to be closely tied to the frontier AI systems, yielding predictable but also surprising developments. Empiricism is key.
- Core approach
 - Capabilities (improving general AI performance)
 - Alignment Capabilities (making AI helpful, honest, harmless)
 - Automated red-teaming
 - Constitutional AI
 - Debiasing
 - RLHF
 - Alignment Science (evaluating whether AI systems are truly aligned)
- Tactics and outcomes
 - Identify risks posed by AI systems and find safe ways to train powerful AI systems
- Defending against Chemical Biological Radiological Nuclear Risks
 - Increasingly strong deployment and security protections to increasingly power AI (ASL 3)
 - Specific targeting of categories of misuse of AI that can be catastrophic
 - Security controls to prevent model weight theft
 - Constitutional Classifiers to make the system much more difficult to jailbreak, minor compute overhead with large reduction in jailbreak success
 - Continuous synthetic jailbreak attempts via automated red-teaming, which is then used to train new classifiers
 - Bring others along with Bug Bounty programs, publication of known attempts to jailbreak (e.g. sophisticated influence operations by networks of people and bots)
 - threat intelligence assessments and operations
- Jailbreak is defined as some way to bypass the normal safeguards of the AI → doing things that are explicitly not desired by the creators of the AI. These can be very long prompts, modifying the style of the input, etc.



Scaling

- Web Tier Horizontal Scaling → more servers to handle failover, traffic increases
 - Vertical just means adding more memory to each computer, which is not sustainable
 - Load Balancer** helps to redirect traffic intelligently, usually pings from outside the network only hits the load balancer, which redirects traffic internally in smart way
- Data Tier Database replication → better performance to allow in parallel queries, better reliability and availability
 - masters handles CRUD while slaves keep replicas and handles reads
 - Data vertical scaling is adding more memory or switching machine types: high costs
 - Data horizontal scaling (aka **sharding**) is adding more servers:
 - Each shard shares the same schema, but the data on each shard is unique
 - Uses hash function to decide which shard to store and fetch data
 - Most important is choosing a **partition or sharding key** that leads to even distribution of data.
 - Challengers with sharding: need to reshard when you have rapid growth, hotspot or hotkey problems (celebrity problems for social networks), etc.
- Caches** are a temporary data store layer that allows better performance because you have less DB workload and you can **scale the cache tier independently** based on use-case
 - Consider when reads are frequent but writes are infrequent. Keep persistent data in DB, not cache

- b. Set an expiration and eviction policies (TTL and least frequently used or FIFO are good examples)
 - c. Prevent the cache from being single points of failure → if cache fails, have backup DB reads, overprovisioning, etc.
- 4. **Stateless servers** → allows for user session and authentication not to be tied to specific servers, but will require moving session data out of the web tier and into the data tier in something like MemCached/Reids/NoSQL. This **enables auto-scaling based on traffic load**.
- 5. CDN (a kind of cache with focus on geography) → controls which servers are closest and can be used to deliver static content like image / video, CSS, etc.



- 6. **Data centers** → **requests are geo-routed, and handle emergencies** like natural disasters, etc.
 - a. Need data syncs, traffic redirection, automated deployment strategies and tools to keep things consistent between data centers
- 7. **Message queues** → supports async communication and acts as **buffer and distributor of requests**, allows for more efficient usage of compute via decoupling of concerns
 - a. Publisher & subscribers
- 8. Logging, metrics, monitoring, CI/CD can be seen as a separate component
- 1. Back of Envelope calculations for various numbers if necessary
 - a. How many users do we expect per day and what is the average QPS? Do we expect to handle failovers?
 - b. Latency

- i. RAM read is 300x faster than disk read (30ms), 100x faster than network read (10ms)
 - ii. Worldwide roundtrip is 150 ms, which is 300x faster than a roundtrip in the same data center
- c. Downtime in a year: 99% or 0 9's → 3.5 days, 2 9's → 1 hour, 3 9's → 5 mins, 4 9's → 30 seconds
- d. Twitter example:
 - i. 300 MAU, 50% DAU → 150m, 2 posts per day avg, 10% has media, Data stored 5 years
 - ii. QPS: $150m * 2 / (24 * 3600) = 3000 - 4000$, peak is 2x so 7000
 - iii. Storage: text less than 0.001% → mostly media, 1mb → 30TB per day → 60 TB for 5 years